ТЕХНОЛОГИИ
РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

ТРПО

(Программная инженерия)

Часть 1. Организация процесса разработки

© Ржавин В.В.

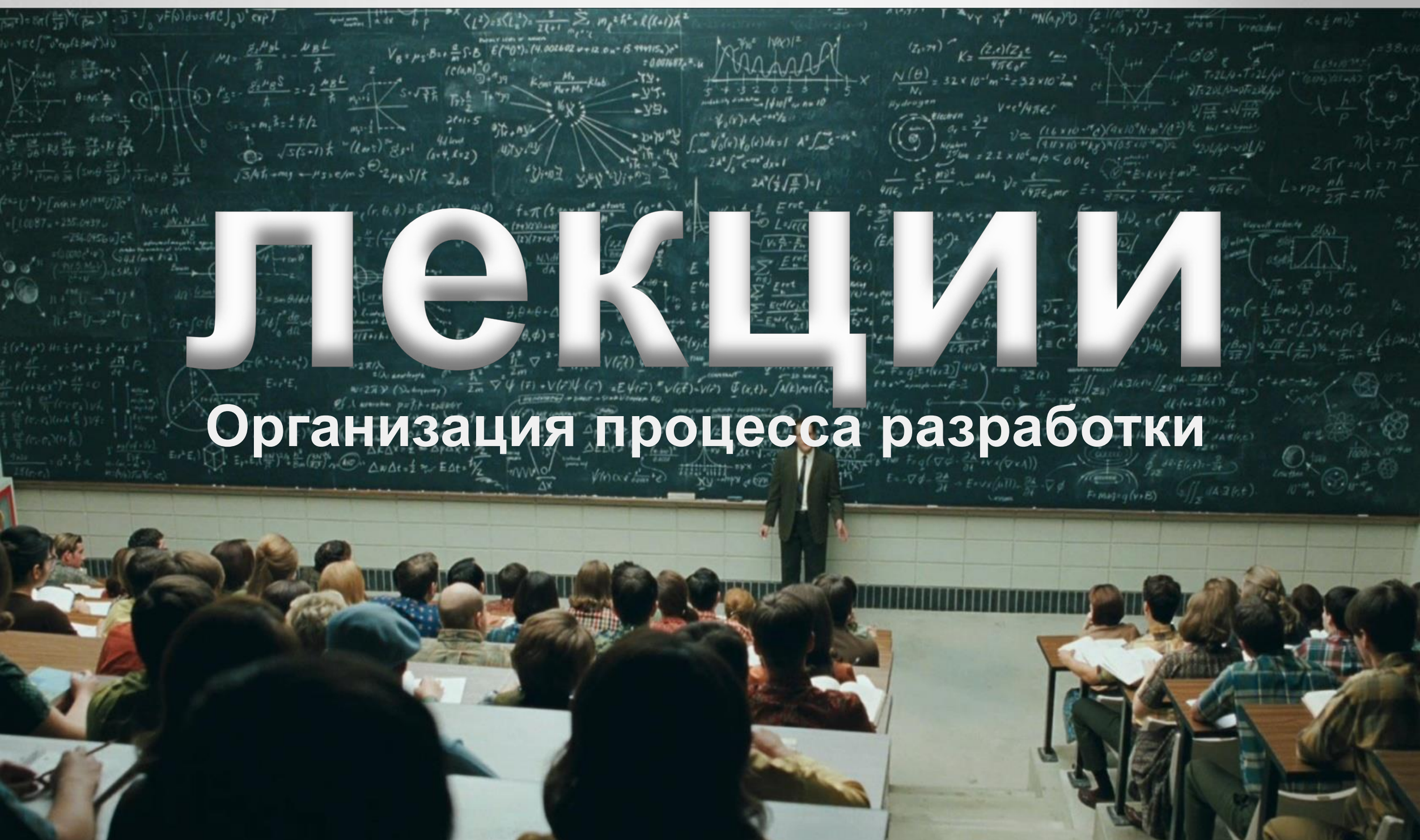
Профессиональная разработка ПО имеет мало общего со «спихиванием лабораторок». Приходя в промышленную разработку программного обеспечения, молодой специалист долго методом проб и ошибок обучается тому, что и как надо на работе делать. И еще больше времени у него уходит на понимание того, почему надо делать именно так, а не иначе.

С.АРХИПЕНКОВ

Лекции по управлению программными проектами

Пишите код так, как будто сопровождать его будет склонный к насилию психопат, который знает, где вы живете.

Стив Макконнелл


лекции
Организация процесса разработки

ОРГАНИЗАЦИЯ ПРОЦЕССА РАЗРАБОТКИ

Основные определения
Модели и подходы

© Ржавин В.В.

Терминология

Сам русский термин «технология программирования» был введен русским академиком Андреем Петровичем Ершовым . Это было очень давно, в те времена термин «программирование» Андрей Петрович толковал в обобщенном смысле, подразумевая все виды деятельности, выполняемые в ходе создания программных систем. И это был революционный прорыв. На Западе для определения этой деятельности использовался (и тогда и сейчас) термин «инжиниринг», вобравший в себя как технологический, так и созидательный аспект деятельности. С тех пор много воды утекло…

С современных позиций обобщенный термин, применимый к созданию программных систем, обозначают как «разработка».

Справедлива формула: «***разработка = анализ + проектирование + программирование (кодирование) + тестирование+ отладка***». Иногда сюда же включают и сопровождение. А чтобы подчеркнуть промышленно производственный аспект, говорят о «технологии разработки».

Буквально русский перевод термина *software engineering* (инжиниринг) нельзя назвать шедевром: звучит он как-то не по-русски. Поэтому долгое время в русскоязычном пространстве бытовал термин *технология разработки программного обеспечения*. Однако такой перевод сужал рамки понятия, оставляя за бортом многие аспекты дисциплины. Именно поэтому в национальных стандартах утвердился перевод *программная инженерия*.

СОДЕРЖАНИЕ

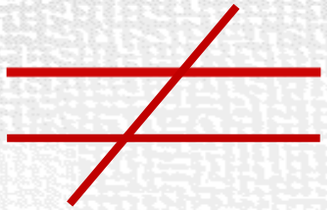
1. Введение
   a. Жизненный цикл программного обеспечения
   b. Свод знаний по программной инженерии (SWEBOK)
   c. Стандартные модели жизненного цикла программного обеспечения
   d. Стратегии разработки и технологичность программной системы
   e. Технологии программирования

2. Планирование проекта
3. Моделирование систем
4. Анализ требований
4. Проектирование
5. Реализация
6. Тестирование
7. Интеграция и развертывание

Программы и Программное обеспечение

Программное обеспечение (Software) – совокупность программ системы обработки информации и программных документов, необходимых для эксплуатации этих программ (ГОСТ 19781-90)

Программное обеспечение



Программа

Программное обеспечение (программная система)

Программные системы состоят из

1. совокупности программ,
2. файлов конфигурации, необходимых для установки этих программ,
3. документации, которая описывает структуру системы, а также содержит инструкции для пользователей, объясняющие работу с системой,
4. адреса Web-узла, где пользователь может найти самую последнюю информацию о данном программном продукте.

- Создается ПО **промышленным способом** коллективом профессионалов-инженеров
- Продается пользователям в виде **программных продуктов**
- Разрабатываются программные продукты в виде **программных проектов**

Кто работает над созданием программного продукта ?

Над созданием программного продукта работают представители разных специальностей:

- Аналитики;
- Архитекторы, Проектировщики;
- Управленцы (менеджеры);
- Тестеры;
- Кодировщики (программисты);
- Технические писатели;
- Системные администраторы;
- Специалисты по повторному использованию;
- Дизайнеры;
- Специалисты по эргономике;
- …

Разработка программного обеспечения

Разработка программного обеспечения (*software engineering*, *software development*) — это род деятельности (профессия) и процесс, направленный на создание и поддержание работоспособности, качества и надежности программного обеспечения, используя технологии, методологию и практики из информатики, управления проектами, математики, инженерии и других областей знания.

СЛОЖНОСТЬ РАЗРАБОТКИ ПО

Как и другие традиционные инженерные дисциплины, разработка программного обеспечения имеет дело с проблемами качества, стоимости и надёжности. Некоторые программы содержат миллионы строк исходного кода, которые должны правильно исполняться в изменяющихся условиях. Сложность ПО сравнима со сложностью наиболее сложных из современных машин, таких как самолеты.

Современное объектно-ориентированное ПО является распределенным (оно может находиться на многих узлах компьютерной сети) и его выполнение случайно и непредсказуемо. Размер современного ПО — сумма размеров его компонентов. Каждый компонент разработан так, чтобы быть ограниченного управляемого размера. В результате размер не является главным фактором в сложности современного ПО.

Сложность современного ПО заключается в «проводах», то есть в связях и коммуникационных путях между компонентами. Связи между компонентами создают зависимости между распределенными компонентами, которые могут быть сложны для понимания и управления. Трудность усугубляется тем, что компоненты часто разрабатываются и управляются людьми и коллективами, даже не известными друг другу.

Разделы дисциплины

1. **Требования к программному обеспечению**: извлечение, анализ, спецификация и ратификация требований для программного обеспечения.
2. **Проектирование программного обеспечения:** проектирование программного обеспечения средствами Автоматизированной Разработки Программного Обеспечения (CASE) и стандарты формата описаний, такие как Унифицированный Язык Моделирования (UML).
3. **Инженерия программного обеспечения**: создание программного обеспечения с помощью языков программирования.
4. **Тестирование программного обеспечения:** поиск и исправление ошибок в программе.
5. **Обслуживание программного обеспечения**: программные системы часто имеют проблемы совместимости и переносимости, а также нуждаются в последующих модификациях в течение долгого времени после того, как закончена их первая версия.

6. **Управление конфигурацией программного обеспечения:** так как системы программного обеспечения очень сложны и модифицируются в процессе эксплуатации, их конфигурации должны управляться стандартизированным и структурированным методом.
7. **Управление разработкой программного обеспечения**: управление системами программного обеспечения имеет заимствования из управления проектами, но есть нюансы, не встречающиеся в других дисциплинах управления.
8. **Процесс разработки программного обеспечения**: процесс построения программного обеспечения горячо обсуждается среди практиков, основными парадигмами считаются agile или waterfall.
9. **Инструменты разработки программного обеспечения:** методика оценки сложности системы, выбора средств разработки и применения программной системы.
10. **Качество программного обеспечения**: методика оценки критериев качества программного продукта и требований к надёжности.

Замечание

Русскоязычному сообществу не повезло: схожее название (проектирование) имеет один из шагов деятельности программного проекта. Так же называется и выполняемая на этом этапе работа. Суть проектирования состоит в получении эскиза концептуального решения требуемого ПО, представляемого обычно с помощью рисунков и пояснений к ним. В английской же терминологией здесь употребляется другой термин - **design**. Будьте внимательны и не называйте проектированием все шаги работы программного проекта.

Бизнес и IT-проекты…

Сейчас **программирование** это не только наука и искусство, но и **бизнес**.
Для того, чтобы этот бизнес был **успешным**, необходимо выполнение многих условий.

Условия?



Необходимые условия **успеха** IT-компании:

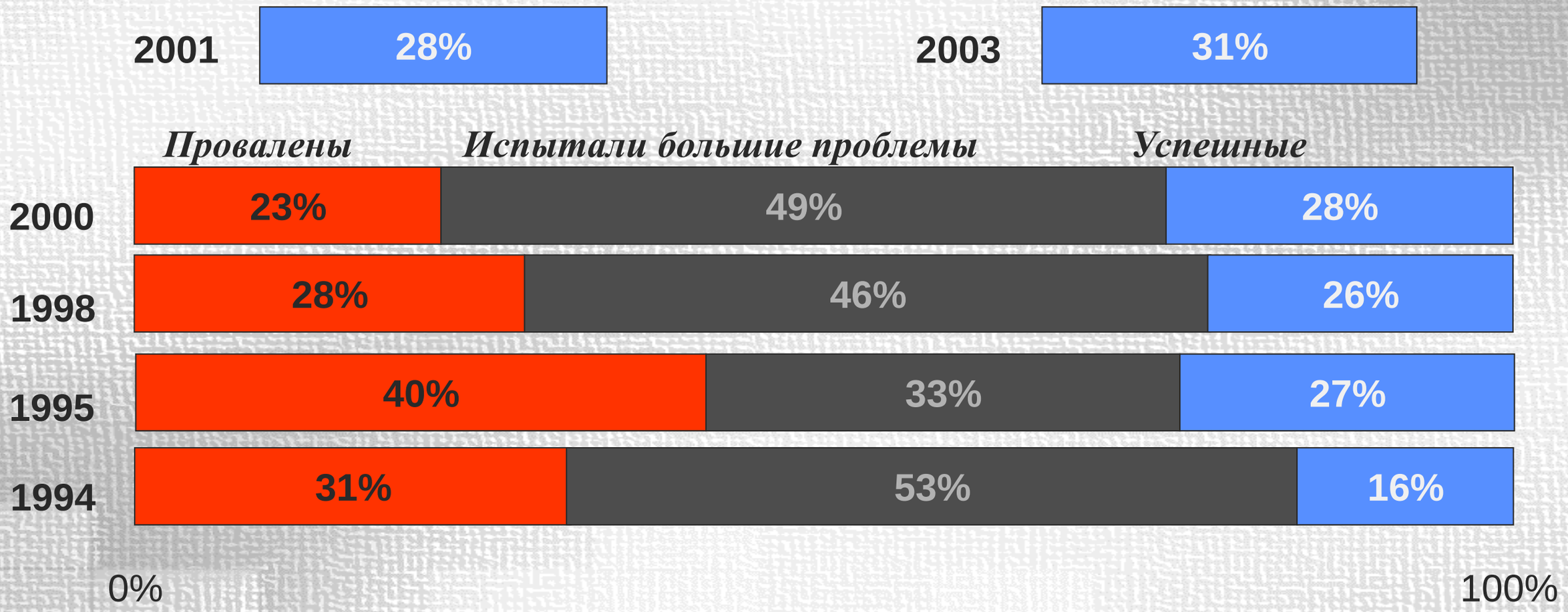
- **выпуск продукта на рынок**
  - надлежащего качества;
  - вовремя;
  - интересного потенциальным пользователям.
- **расходы должны соответствовать изначальному бюджету**.

Немного статистики. IT-проекты...

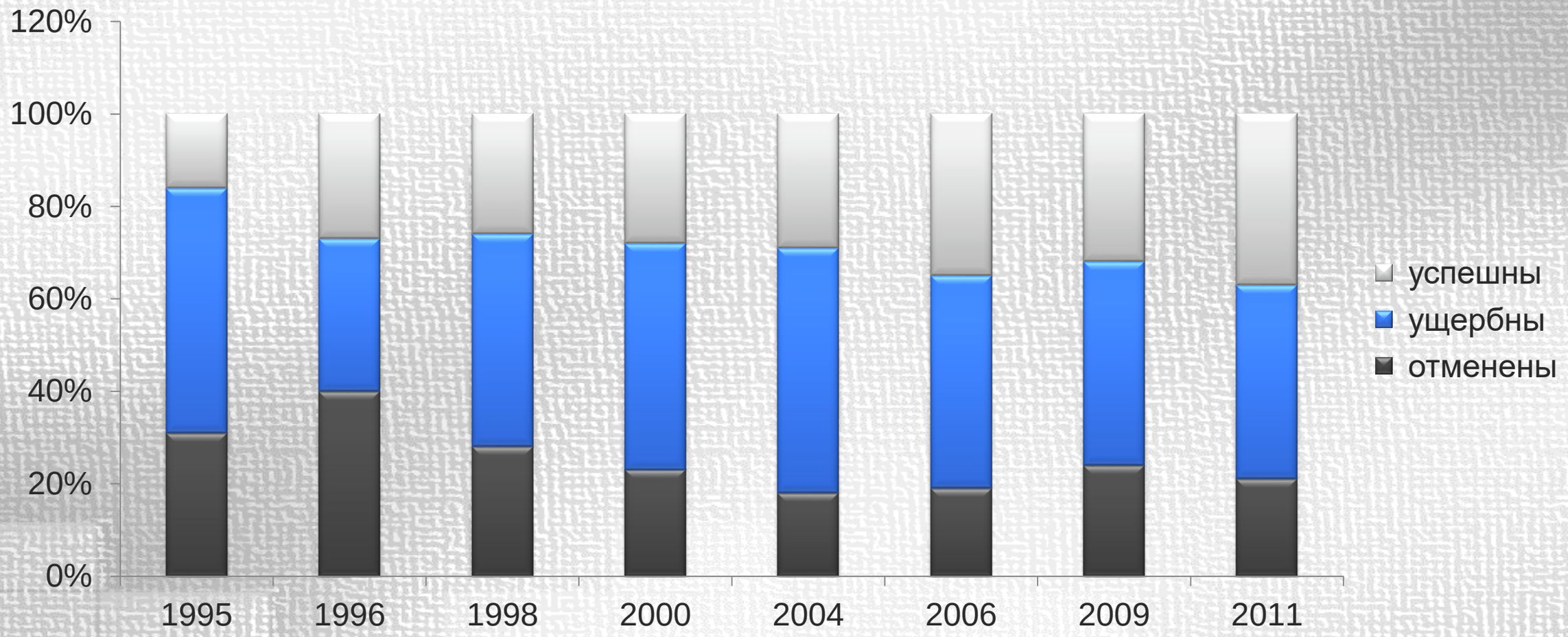
Проваленные: закончились неудачей – цель вообще не была достигнута.

Испытавшие большие проблемы: закончились созданием продукта, но превысили **бюджет** или (и) не уложились во **время** или (и) имеют лишь частичную **функциональность**.

Успешные: закончились созданием продукта, уложились в **бюджет** и **время**. Вся планируемая **функциональность** реализована.


2001
28%
2003
31%
Провалены
Испытали большие проблемы
Успешные
2000
23%
49%
28%
1998
28%
46%
26%
1995
40%
33%
27%
1994
31%
53%
16%
0%
100%

Статистика IT-проектов



Тенденции

Быстрый рост объемов IT-рынка, рынка ПО.
Укрепление позиций российских компаний.
По-прежнему малая доля в мировых объемах.

В чем нуждается отрасль?

В **грамотных специалистах**, способных повысить **успешность** проектов.

Факторы успеха проектов

• Вовлечение пользователей	15.9%
Поддержка руководства	13.9%
• Четкая и ясная постановка требований	13.0%
Хорошее планирование	9.6%
• Реалистичные ожидания	8.2%
Частые контрольные точки	7.7%
Компетентная команда	7.2%
Владение (требованиями)	5.3%

Почему проекты не успешны?

1. Нереалистичные временные рамки.
2. Недостаток количества исполнителей.
3. Размытые границы проекта.
4. Недостаток средств.
5. Нехватка квалифицированных кадров.

Кто виноват?

- Все 5 пунктов относятся к компетенции управленцев различного уровня.
- Проектами в области разработки ПО очень сложно эффективно **управлять**.

Почему?



Откуда берется сложность управления?

- **Человеческий фактор**. Традиционные трудности управления коллективом.
- **Новая молодая область.** Не хватает квалифицированных кадров, методов, технологий, опыта.
- **Сложность задач.** Рост возможностей приводит к росту сложности решаемых задач и наоборот.
- **Трудности в работе с заказчиками и будущими пользователями.** Заказчики не знают, что они хотят. Пользователи плохо владеют ПК и не понимают, как он работает и что от него ждать. Все говорят на разных языках.
- …

КАК БОРОТЬСЯ СО СЛОЖНОСТЬЮ?

- Как бороться со сложностью задач? (1)
- Как бороться со сложностью управления? (2)

Ответ: **разрабатывать и внедрять специальные технологии и методологии:**

1. ***Технологии программирования*** (изучали в курсах Основы программирования, ООП).
2. ***Методологии*** MSF, RUP, XP, …

Только треть программных проектов завершается успешно. Главной причиной большинства неудач в разработке ПО является не столько сложность и масштабность разрабатываемых продуктов, сколько применение **неадекватных методов управления программными проектами**.

Разработка ПО отличается от материального производства, поэтому опыт управления, накопленный в этих отраслях, не всегда эффективен и применим в программных проектах. **Управлять разработкой ПО надо иначе**.

Что надо делать для успеха программного проекта

Стив Макконнелл в своей книге приводит тест программного проекта на выживание. Руководитель программного проекта должен его периодически использовать для внутреннего аудита своих процессов.

Чтобы программный проект стал успешным, необходимо:

1. Четко ставить цели.
2. Определять способ достижения целей.
3. Контролировать и управлять реализацией.
4. Анализировать угрозы и противодействовать им.
5. Создавать команду.

ПРОГРАММИРОВАНИЕ – ЭТО

- Искусство ?
- Наука ?
- Ремесло ?

Программирование – искусство?

Программирование это не искусство, в том смысле, что оно не является “*творческим отражением и воспроизведением действительности в художественных образах*”. Об искусстве в программировании можно и должно говорить только в смысле умения, мастерства, знания дела, как и в любой другой профессии.

Творчество - это интеллектуальная деятельность человека, законы которой нам неизвестны. Если бы мы знали законы творчества, то и картины, и стихи, и музыку, и программы уже давно бы создавали компьютеры. Творческое начало - это то, что роднит программирование с наукой и искусством.

Творчество в программировании начинается с определения целей программы и заканчивается только тогда, когда в ее коде, написанном на каком-либо языке программирования, поставлена последняя точка. *Попытки разделять программистов на творческую элиту, архитекторов и проектировщиков, и нетворческих программистов-кодеров не имеют под собой объективных оснований*.

Программирование – наука?

Программирование это не наука. Наработки математиков в области логики, теории информации, численных методов, реляционной алгебры, теории графов и некоторых других дисциплинах на долю процента не покрывают сложность программистских задач. **В программировании нет системы знаний о закономерностях создания программ**. Даже выдающиеся программисты не возьмут на себя смелость утверждать об архитектуре новой программной системы то, что она будет успешной. Хотя в программировании уже накоплен определенный опыт провалов, который может позволить искушенному программисту увидеть в архитектуре новой системы антипаттерны - источники серьезных будущих проблем. Но не более того.

Программирование – ремесло?

- Программирование - не искусство и не наука – это ремесло (?). Сегодня мы так же далеки от индустриальной разработки программ, как и 50 лет назад.
- А поскольку это ремесло, то человек, научившийся писать программы на C ++, будет так же далек от профессионала, как ученик третьего класса средней школы, научившийся писать по-русски, от А. С. Пушкина или Ф. М. Достоевского.
- **Путь к мастерству в ремесле лежит только через опыт.**

Программирование - инженерная дисциплина

Разобравшись в различиях между ремеслом, научной теорией и инженерным делом, можно понять роль теории в инженерии:

- ***ремесло*** применяет традиционные методы для изготовления изделий и предоставления услуг;
- ***наука*** собирает, проверяет, документирует знания и открытия с помощью исследований;
- ***инженерия*** применяет знания, научно обоснованные методы для разработки и изготовления технических изделий.

Чтобы превратить ремесло в инженерию, нужно применять научные теории для анализа и обоснования методов и технологий. С точки зрения ремесла не важно понимать, почему работают те или иные методы и как их можно обосновать и оптимизировать, достаточно того, что они дают удовлетворительный результат. Для инженерии же, напротив, ключевыми являются вопросы о том, как систематически создавать новые методы, почему те или иные технологии действенны и как с помощью научных методов обосновать, соотнести, доказать и оптимизировать их.

Ремесло и инженерия имеют много общего, поскольку у них схожие цели: создание конструктивных решений практических проблем изготовления изделий и предоставления услуг. Соответственно граница между ремеслом и инженерией размыта, например существует мнение, что разработка ПО сегодня в большой степени ремесло. Однако согласно определению ремесло пользуется традиционными методами, а поскольку разработка ПО еще молодая область, у нее нет достаточного количества таких методов. В некоторых отношениях данная дисциплина эмпирическая, опирающаяся на методики, которые нельзя назвать хорошо проверенными. Эволюция традиционных методов требует времени, а сфера ПО все еще стремительно развивается.

Существующее состояние программной инженерии напоминает большую поваренную книгу с многочисленными описаниями рецептов однажды успешно приготовленных блюд из ингредиентов, которых в будущем уже не будет. Завтра в новой системе будут другие вычислительные машины, технологии, языки программирования, инструменты и окружающее ПО, новые проблемы взаимодействия с которыми обязательно придется решать.

Программист должен обладать способностью первоклассного математика к абстракции и логическому мышлению в сочетании с эдисоновским талантом сооружать все, что угодно, из нуля и единиц. Он должен сочетать аккуратность бухгалтера с проницательностью разведчика, фантазию автора детективных романов с трезвой практичностью экономиста.

ЛЮДИ КАК ГЛАВНЫЙ ФАКТОР В РАЗРАБОТКЕ ПО

Небольшое, но важное замечание…



Доктор Алистер Коуберн (Alistair Cockburn) - специалист в области IT-стратегий, специалист по быстрой разработке и управлению проектами, объектно-ориентированном проектированию. За его новаторские технологии в области анализа сценариев использования (use-case) в 2007 году он был удостоен звания "The All-Time Top 150 i-Technology Heroes". Автор методологии Crystal clear, соавтор манифеста Agile, сертифицированный тренер по методологиям SCRUM.

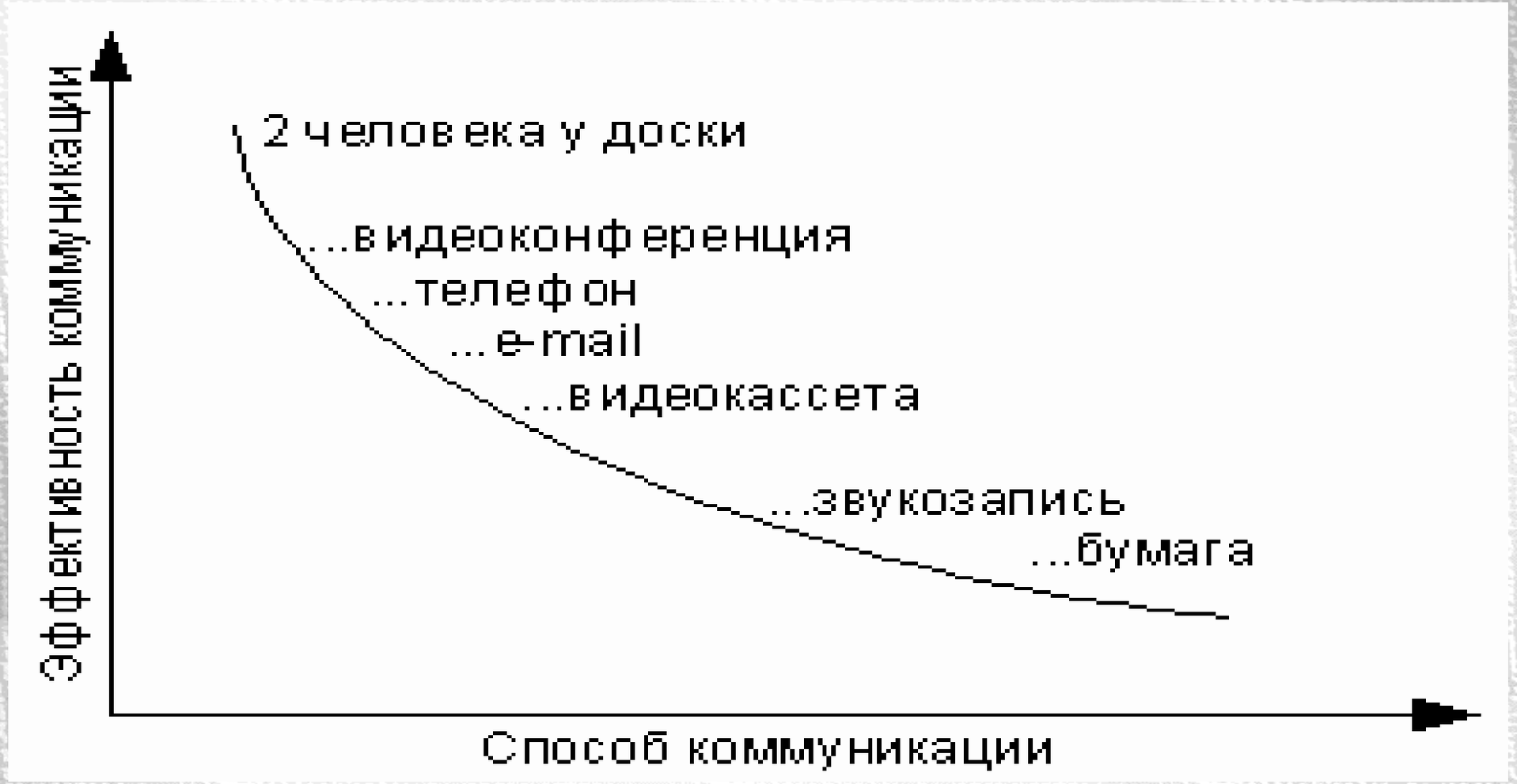
Алистэр Коуберн

«Люди как нелинейные и наиболее важные компоненты в создании программного обеспечения»

Что меня удивило во всех этих проектах? То, что они ясно свидетельствуют о следующем:

- Практически любую методологию можно с успехом применять в каком-нибудь проекте.
- Любая методология может привести к провалу проекта.
- Тяжеловесные методологии тоже могут успешно применяться в работе.
- Облегченные методологии чаще приводят к успеху, и, что более важно, разработчики сами говорят, что успех проекта был обеспечен именно методологией.

Люди как главный фактор в разработке ПО



Люди как главный фактор в разработке ПО

Я прихожу к заключению, что большинство проектов вполне можно вести, руководствуясь (верными) не очень точными описаниями: не очень точную документацию по проекту легче читать, приводить в порядок и обсуждать. Архитектуру системы, изображенную с невысокой степенью точности, легче запомнить; в таблицах с не очень точно описанными требованиями легче расставлять приоритеты и легче оценивать масштабы сделанной работы на ранних стадиях проекта. Выполненная не очень точно проектная документация лучше передает "идею" проекта, после чего читатель может начать "ориентироваться в ситуации".

Создание артефактов с низкой степенью точности позволяет снизить стоимость работ за счет сильных качеств человеческой натуры.

Технологии — мелочь. Главное верить в людей, считать, что, в принципе, люди хорошие и умные, и понимать, что если дать им нужные инструменты, то эти люди станут просто творить чудеса.

Стив Джобс

- Что такое программная инженерия
- Свод знаний по программной инженерии (SWEBOK)

Классификация программного обеспечения или что программировать



Классификация ПО

Программное обеспечение принято подразделять:

- по назначению на *системное, прикладное и инструментальное*, *служебное*.
- по способу распространения и использования на *несвободное/закрытое, открытое и свободное*.

Свободное программное обеспечение может распространяться, устанавливаться и использоваться на любых компьютерах дома, в офисах, школах, вузах, а также коммерческих и государственных учреждениях без ограничений.

Системное ПО

Это часть системы, которая помогает следить за аппаратной стороной ПК и управлять ею. Сюда входят программы, контролирующие работу оперативной памяти, центрального процессора, видеокарты, устройств ввода и вывода информации, сетевые подпрограммы. Таким ПО считается:

- Драйверы – утилиты небольшого размера, функционирование которых заключается в обеспечении корректной работы остальных элементов оборудования;
- Операционные системы;
- Дополнения – языковые пакеты или настройки расширения экрана.

Основное отличие системной разновидности считается то, что она не рассчитана на выполнение конкретной поставленной задачи. Она необходима, чтобы обеспечивать бесперебойную работу остальных частей компьютера. Ее можно назвать посредником между оборудованием – «железом» и программным кодом.

Прикладное ПО

Прикладная программа (программное приложение, или приложение) представляет собой компьютерную программу, предназначенную для выполнения конкретной задачи, отличной от той, которая относится к работе самого компьютера, обычно используется конечными пользователями. Собирательное существительное «прикладное программное обеспечение» относится ко всем приложениям в совокупности.

К прикладному программному обеспечению относят:

- Архиваторы, антивирусы, браузеры, САПР, плееры
- Проигрыватели, веб-обозреватели, видеоигры, редакторы графики
- Средства для резервирования данных, работы с реестром, текстовые процессоры
- Менеджеры автозапуска, проигрыватели, менеджеры загрузок, текстовые редакторы

Инструментальное ПО

Специфическое обеспечение любой компьютерной техники. Его можно было бы отнести к прикладному, но из-за специфики применения его выделили в отдельный вид. Основная функция – отладка, настройка, переписывание программного кода.

Сюда входят *компиляторы, отладчики, переводчики высокого уровня, редакторы, интерпретаторы* и другие средства. Они необходимы, потому что техника не понимает человеческих слов. Чтобы ей «объяснить», что надо сделать, требуется специальный «машинный язык».

Постоянно пользоваться этим кодом базовым пользователям довольно сложно, поэтому были разработаны системы, которые позволяют переводить обычную речь в двоичную, привычную для ПК.

Разница между часто используемыми компиляторами и интерпретаторами заключается в том, что первый генерирует готовый файл, который можно запускать. А второй создает архив, который функционирует только с помощью самого сервиса.

Служебное ПО (утилиты)

Служебное программное обеспечение - это программное обеспечение, предназначенное для анализа, настройки, оптимизации или обслуживания компьютера. Используется для поддержки компьютерной инфраструктуры — в отличие от прикладного программного обеспечения, которое направлено на непосредственное выполнение задач, выгодных обычным пользователям. Однако **утилиты** часто являются частью прикладных систем. Например, пакетное задание может выполнять написанный пользователем код для обновления базы данных, а затем может включать шаг, который запускает служебную программу для резервного копирования базы данных, или задание может запускать служебную программу для сжатия диска перед копированием файлов.

Хотя базовый набор служебных программ обычно распространяется с операционной системой (ОС), и это основное программное обеспечение часто считается частью операционной системы, пользователи часто устанавливают замены или дополнительные утилиты. Эти утилиты могут предоставлять дополнительные возможности для выполнения задач, выходящих за рамки возможностей операционной системы.

Типы программных систем

1. **Системы работающие только с людьми** – например: информационные системы организаций – используются людьми для ввода, хранения поиска и отображения информации;
2. **Системы работающие с людьми и внешними техническими устройствами**: – например: информационные системы соревнований.
3. **Системы работающие с внешними техническими устройствами** – встроенные системы реального времени – встраиваются в технические устройства для выполнения управления ими.

Коробочные и заказные продукты

В зависимости от того, для кого разрабатываются программные продукты (конкретного заказчика или рынка), программные продукты бывают двух типов:

- **коробочные продукты** (generic products – общие продукты или shrink-wrapped software – упакованное ПО)
- **заказные продукты** (bespoke – сделанный на заказ или customized products – настроенный продукт).

Важная разница между ними заключается в том, кто ставит задачу (определяет, или специфицирует требования).

1. В первом случае это делают сами разработчики на основе анализа рынка (маркетинга) – и при этом рискуют сами.
2. Во втором – заказчик и при этом рискует, что разработчик не сможет реально выполнить все требования в срок и при выделенном бюджете.

Модели C2C, B2B, B2C

- **C2C** (Consumer-to-Consumer) — взаимоотношения между потребителями
- **B2B** (Business-to-Business) — взаимоотношения между организациями
- **B2C** (Business-to-Consumer) — взаимоотношения между организацией и потребителем
- **G2B** (Government-to-Business) — взаимоотношения между государством и бизнесом
- **B2G** (Business-to-Government) — взаимоотношения между бизнесом и государством

- Что такое программная инженерия
- Свод знаний по программной инженерии (SWEBOK)

ПРОГРАММНАЯ ИНЖЕНЕРИЯ



Область действия программной инженерии

В западной литературе часто используются термины: **computer science, software engineering**, **system engineering** и **computer engineering**. В чем разница?

- **Computer science** имеет дело с теорией и основами разработки ПО.
- **Computer engineering (компьютерная инженерия)**
- **System engineering (системная инженерия)** связана с вопросами разработки систем с участием компьютеров (архитектура, дизайн, интеграция, ПО...).
- **Software engineering (программная инженерия)** – часть System engineering, имеющая дело с ПО.

Computer science
Основные дисциплины

Программная инженерия (*Software Engineering*)

- **Теория построения компьютерных систем (КС)**
  - программ для оборудования, оснастки
  - инфраструктура КС
- **Теория, принципы, концепции программных систем (ПС)**
  - фундаментальные,
  - прикладные
- **Методы применения, развертывания, конфигурирования КС и ПС**
- **Програмные методы и технологии**
  - ОС, коммуникационные программы
  - организация вычислений
- **Прикладные информационные технологи (ИТ)**
  - веб-браузеры, БД, машины поиска
  - теории для нужд организаций
- **Организационные вопросы информационных систем**
  - теория и принципы организации ИС

Компьютерная инженерия (*Computer Engineering*)

- **Теория построения Hardware Frameworks**
  - компьютеров, микрокомпьютеров, кластеров
  - ПК, суперкомпьютеры (Скит2)
- **Принципы построения Software для разных типов машин**
  - ОС, управления заданиями
  - СУБД, трансляторов, интерпретаторов, метатрансляторов
- **Компьютерные архитектуры**
  - микросхемы, микропроцессоры
  - процессоры (Pentium, Intel…)
  - операции последовательные, параллельные, рекурсивные
  - форматы данных, преобразователи
  - интеграция устройств, блоков, карт, кабелей...
  - интеграция процессоров в кластеры)
- **Фундаментальные теории компьютерной инженерии**
  - теория автоматов
  - машины Тьюринга
  - неймановские машины...

Системная инженерия (*System Engineering*)

- **Построение сложных компьютерных систем**
  - АС, ИС, ИПС, СОД, АСК, ЗДАС...
  - систем реального времени
  - моделирование Computer Application
  - систем програмування
- **Информационные системы**
  - искусственный интеллект
  - управление информацией
  - поисковые системы Интернет
  - БД. БЗ, информационные ресурсы
  - веб-сервисы, веб-семантика
  - електронні бібліотеки
- **Информационные технологии**
  - интерфейс пользователя
  - технологии взаимодействия
  - процессы обработки, потоки работ
  - накопичення, витяг знань
  - инфраструктура (техника, средства, инструменты, ПЗ, среды)
  - коммуникации и маршрутизации
  - делопроизводство, документооборот
- **Компьютерные системы**
  - компьютерная графика
  - мультимедиа среды
  - системы защиты информации
  - системы электронного обучения
  - электронная Computer Science
  - прикладные компьютерные системы (экология, медицина, мат. физика вычислительная математика,…)

Область действия программной инженерии

Итак, **computer science** предоставляет теоретический базис. На практике его недостаточно. Открытые проблемы:

- Поиск финансирования.
- Работа с заказчиком.
- Подбор персонала.
- Этические вопросы. Микроклимат в коллективе. Команда.
- Обеспечение качества программного продукта.
- ...

Всем этим занимается **программная инженерия**.

Программная инженерия как инженерная дисциплина

- **Программная инженерия** определяется как системный подход к анализу, проектированию, оценке, реализации, тестированию, обслуживанию и модернизации программного обеспечения, то есть **применение инженерного подхода к разработке программного обеспечения**.
- Термин "разработка программного обеспечения" является более общим и часто используемым, по сравнению с термином "программная инженерия" и не обязательно включает в себя парадигмы инженерии.

Дисциплины ПИ

Для решения важных проблем индустрии, отображения состояния научно-технических достижений в ПИ и обучение студентов в состав ПИ предложены новые дисциплины программной инженерии (ПИ).

Данные дисциплины отражают процесс управления проектом с привлечением современных научных основ (управление, инженерия, экономика и др.) сопутствующих фундаментальных наук, которые используются при разработке линий (Product Lines) для конструирования качественных программных продуктов.

Классификация дисциплин ПИ



- **научная дисциплина (теория программирования)** включает классические науки (теория алгоритмов, множеств, доказательства, математическая логика и т.п.) и соответствующие общие языковые средства проектирования с привлечением абстрактных моделей и архитектур целевых программных объектов, моделей проектирования, а также теории сборки и взаимодействия компонентов между собой и средами.
- **инженерная дисциплина** — совокупность технологических средств и методов проектирования ПП на основе фундаментальных и стандартных моделей ЖЦ, техники анализа домена, инженерии требований, разработки, сопровождения, изменения ПС и адаптации ПП к другим платформам и средам.
- **управленческая дисциплина** — общая теория управления, адаптированная к коллективной разработке ПП, включающая в себя графики работ, наблюдения за их выполнением, управления рисками, версиями (конфигурационный файл) ПП и сопровождением.

- **экономическая дисциплина** — совокупность методов экспертного, количественного и качественного оценивания промежуточных артефактов, конечного результата выполнения процессов ЖЦ и методов расчета времени, объема, трудоемкости и стоимости изготовления ПП.
- **производственная дисциплина** — сборочный конвейер с подготовленными линиями производства определенных видов ПС и СПС, по которым изготавливаются члены семейства с помощью готовых программных ресурсов (КПИ, сервисы, аспекты, агенты и т.п.), расположенных в библиотеках или репозиториях путем конфигурирования их в выходную структуру, а также тестирования и оценивания качества полученного продукта.

Развитие ПИ

Международное компьютерное сообщество специалистов для развития ПИ сделало довольно много, в частности:

- построено ядро знаний SWEBOK (Software Engineering body of Knowledge,www.swebok.com, 2001 г.);
- функционируют международные научные журналы по программной инженерии IEEE Trans. on SE, ACM Trans. on SE and Metodology, зарубежные монографии и учебники с ПИ и т.п.;
- сформирована международная программа обучения ПИ — Curricula-2004 (www.computing.org/education/cc2004, intuit.ru);

- создан институт SEI при Carnegie Mellon University (1984 г.), кафедры ПИ во многих университетах США и Европы для обучения и развития ПИ;
- распространены автоматизированные мощные инструментальные среды (Microsoft Visual Studio, MSF, Rational Rose, COM, JAVA, CORBA и т.п.) поддержки разработки и производства разных целевых объектов.
- проводятся международные научные конференции по разным направлениям ПИ (например, reusability, Software Engineering Factories и др.) и публикуются соответствующие материалы;
- используются новые парадигмы программирования (объектно-ориентированное, компонентное, сервисное, агентное, генерирующее и т.п.);

- создана технология создания технологических и продуктовых линии (Product Lines) для разработки целевых объектов: программных систем - ПС, семейств систем — СПС, доменов и т.п., которые базируются на готовых ресурсах (ГОР) и компонентах повторного использования (КПИ) типа reuses, assets, servises и т.п.;
- используются методы реинженерии, реверсной инженерии и рефакторинга для обновления устаревших программ, накопленных в предыдущих поколениях вычислительных машин, с целью их повторного использования на платформах новых компьютеров;

Цели программных инженеров...

Цели программных инженеров:

- **Создать качественный продукт.**
- **Уложиться в бюджет.**
- **Уложиться в сроки.**

Разберем эти вопросы подробнее.

Цели программных инженеров. Качественный программный продукт

ПО должно быть качественным.

- **Удобство в сопровождении**:
  - ПО должно допускать развитие в связи с изменением потребностей пользователей.
- **Надежность**:
  - Возможные неполадки в работе не должны нанести существенный, тем более невосполнимый ущерб.
- **Эффективность**:
  - ПО должно эффективно использовать имеющиеся ресурсы.
- **Удобство в использовании:**
  - ПО должно приниматься пользователями «на ура», работа должна быть удобной и естественной.

Цели программных инженеров. Бюджет

Создание ПО должно укладываться в бюджет

- Типовое распределение средств:
  - 60% – разработка.
  - 40% – тестирование.
  - Развитие – зачастую больше, чем создание.
- Детали зависят от специфики предметной области, требований к ПО, используемых подходов к организации разработки.

Цели программных инженеров. Сроки

Создание ПО должно укладываться в сроки.

- Необходимо грамотное планирование.
- Анализ возможных рисков и способы реагирования.
- Борьба за четкие границы проекта.
- Мотивирование сотрудников.

СВОД ЗНАНИЙ ПО ПРОГРАММНОЙ ИНЖЕНЕРИИ (SWEBOK)

Любая профессия базируется на некой совокупности теоретических знаний и практического опыта. Если эту основу удается формализовать и соответствующим образом документировать, подобный свод знаний становится отправной точкой для разработки учебных программ.

Немного истории…

Термин **software** (программное обеспечение, ПО) ввел в 1958 году всемирно известный статистик Джон Тьюкей (John Tukey). Термин **software engineering** (программная инженерия) впервые появился в названии конференции НАТО, состоявшейся в Германии в 1968 году и посвященной так называемому кризису программного обеспечения. С 1990-го по 1995 год велась работа над международным стандартом, который должен был дать **единое** представление о процессах разработки программного обеспечения. В результате был выпущен **стандарт ISO/IEC 12207**. В 2004 году в отрасли был создан основополагающий труд «**Руководство к своду знаний по программной инженерии» (SWEBOK)**, в котором были собраны основные теоретические и практические знания, накопленные в этой отрасли.

SWEBOK
(Software Engineering Body Knowledge)

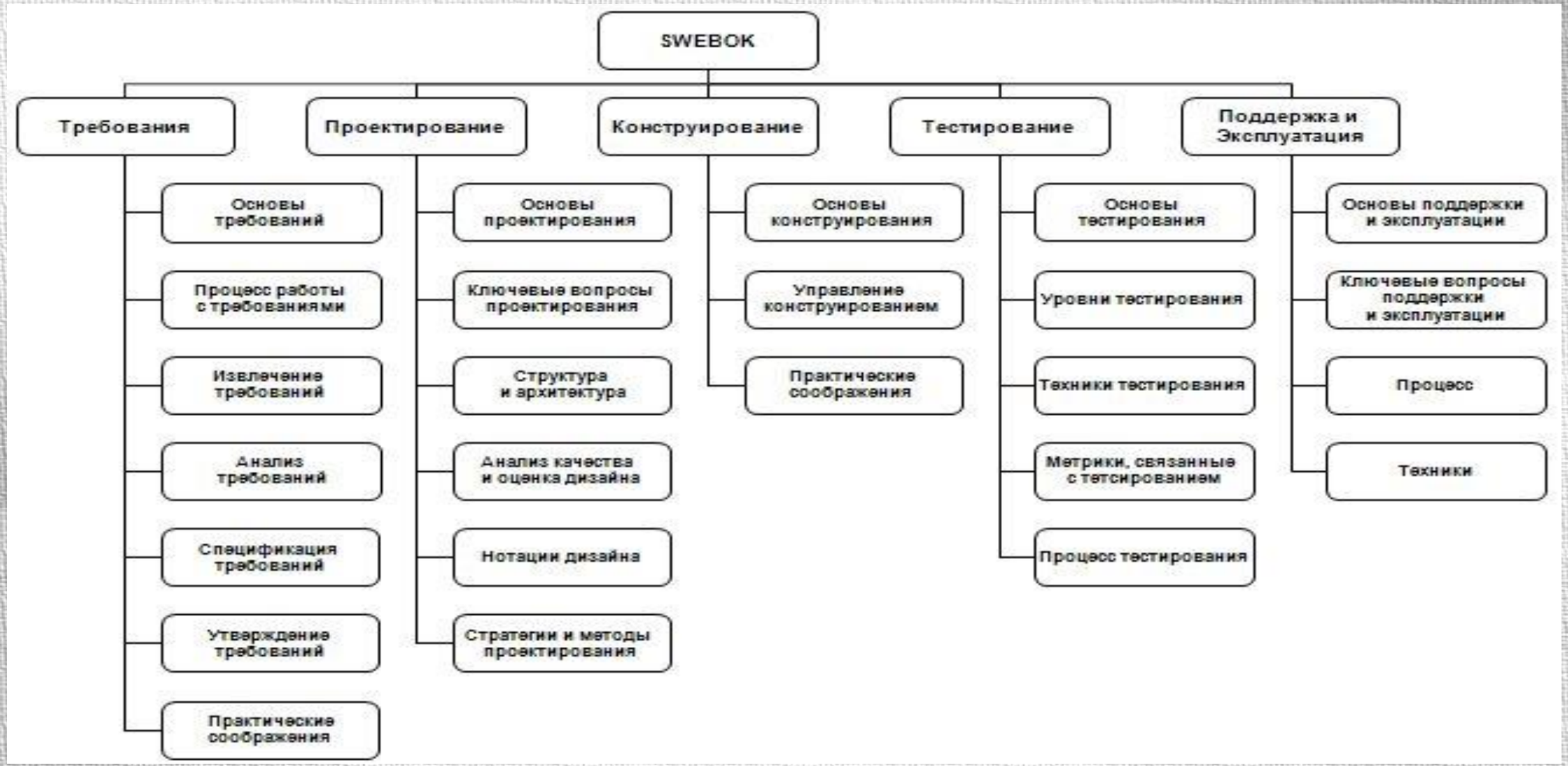
- Знания разработчиков ПО отличаются большим разнообразием, являются не согласованными и разнородными, ориентированными на разные предметные области, поэтому мировая компьютерная общественность пришла к необходимости систематизировать знания в области программной инженерии, создав ядро знаний SWEBOK (Software Engineering Body Knowledge).
- Согласно SWEBOK 2004, программная инженерия включает в себя 10 основных и 7 дополнительных областей знаний, на которых базируются процессы разработки ПО.

Основные области знаний

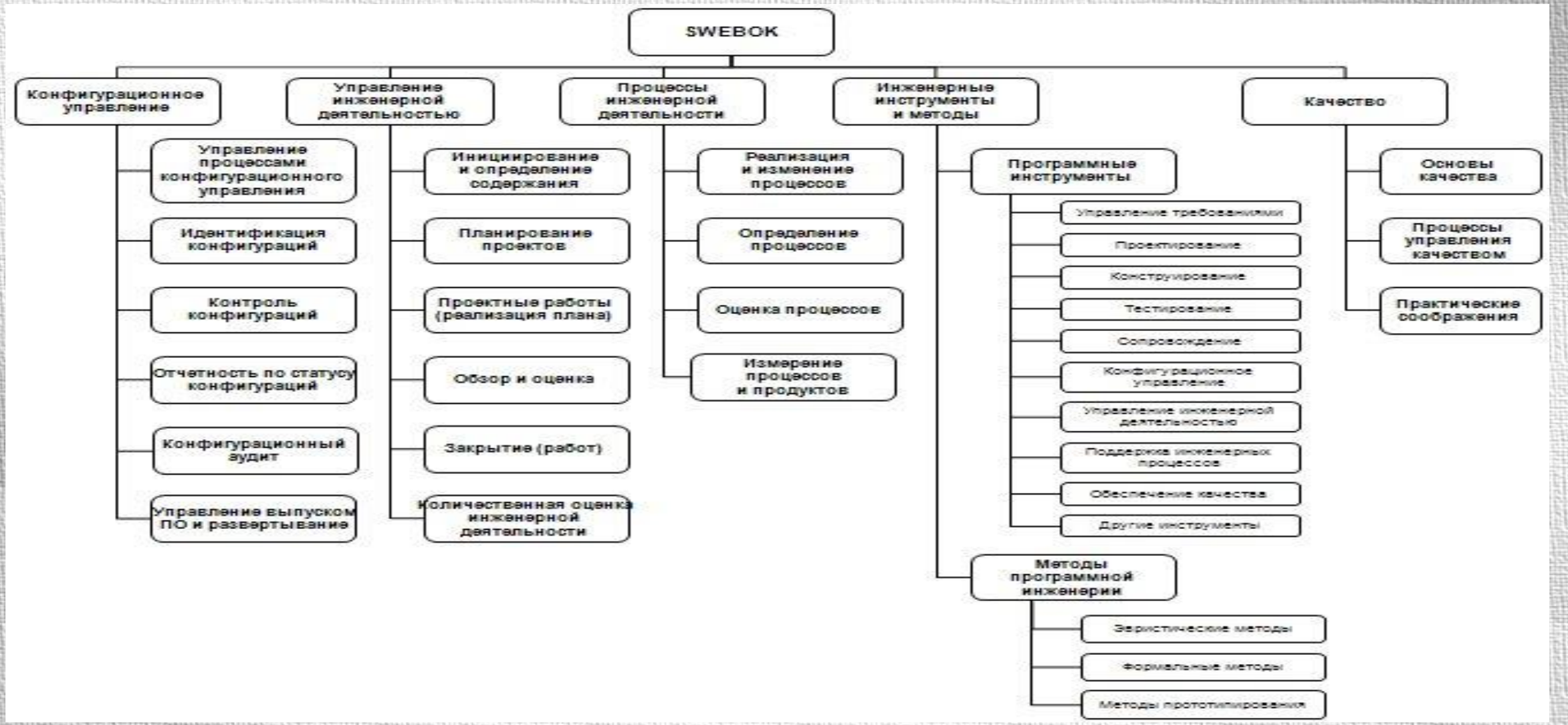
1. **Software requirements** – программные требования.
2. **Software design** – дизайн (архитектура).
3. **Software construction** – конструирование программного обеспечения.
4. **Software testing** – тестирование.
5. **Software maintenance** – эксплуатация (поддержка) программного обеспечения.

6. **Software configuration management** – конфигурационное управление.
7. **Software engineering management** – управление в программной инженерии.
8. **Software engineering process** – процессы программной инженерии.
9. **Software engineering tools and methods** – инструменты и методы.
10. **Software quality** – качество программного обеспечения.

Основные области знаний SWEBOK



Организационные области знаний SWEBOK



Дополнительные области знаний

1. **Computer engineering** – разработка компьютеров.
2. **Computer science** – информатика.
3. **Management** – общий менеджмент.
4. **Mathematics** – математика.
5. **Project management** – управление проектами.
6. **Quality management** – управление качеством.
7. **Systems engineering** – системное проектирование.

Новая версия SWEBOK

В настоящий момент сообществом разрабатывается новая, дополненная версия, включающая 15 областей:

1. **Software Requirements** — требования к ПО.
2. **Software Design** — проектирование ПО.
3. **Software Construction** — конструирование ПО.
4. **Software Testing** — тестирование ПО.
5. **Software Maintenance** — сопровождение ПО.
6. **Software Configuration Management** — управление конфигурацией.
7. **Software Engineering Management** — управление IT проектом.

8. **Software Engineering Process** — процесс программной инженерии.
9. **Software Engineering Models and Methods** — модели и методы разработки.
10. **Software Engineering Professional Practice** — описание критериев профессионализма и компетентности.
11. **Software Quality** — качество ПО.
12. **Software Engineering Economics** — экономические аспекты разработки ПО.
13. **Computing Foundations** — основы вычислительных технологий, применимых в разработке ПО.
14. **Mathematical Foundations** — базовые математические концепции и понятия, применимые в разработке ПО.
15. **Engineering Foundations** — основы инженерной деятельности.

ОБЩИЕ ПРИНЦИПЫ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

Программное обеспечение различается по назначению, выполняемым функциям, формам реализации. В этом смысле всякое ПО — сложная, достаточно уникальная программная система. Однако существуют некоторые общие принципы, которые следует использовать при разработке ПО.

- **Частотный принцип**. Основан на выделении в алгоритмах и в обрабатываемых структурах групп действий и данных по частоте использования. Для действий, которые чаще встречаются при работе ПО, обеспечиваются условия их наиболее быстрого выполнения. К данным, к которым происходит частое обращение, обеспечивается наиболее быстрый доступ. «Частые» операции стараются делать более короткими.

- **Принцип модульности.** Под модулем в общем случае понимают функциональный элемент рассматриваемой системы, имеющий оформление, законченное и выполненное в пределах требований системы, и средства сопряжения с подобными элементами или элементами более высокого уровня данной или другой системы. Способы обособления составных частей ПО в отдельные модули могут быть существенно различными. Чаще всего разделение происходит по функциональному признаку. В значительной степени разделение системы на модули определяется используемым методом проектирования ПО.

- **Принцип функциональной избирательности**. Этот принцип является логическим продолжением частотного и модульного принципов и используется при проектировании ПО, объем которого существенно превосходит имеющийся объем оперативной памяти. В ПО выделяется некоторая часть важных модулей, которые постоянно должны быть в состоянии готовности для эффективной организации вычислительного процесса. Эту часть в ПО называют ядром или монитором. В состав монитора, помимо чисто управляющих модулей, должны войти наиболее часто используемые модули. Программы, входящие в состав монитора, постоянно хранятся в оперативной памяти. Остальные части ПО размещаются на внешних запоминающих устройствах и загружаются в оперативную память только по вызову, перекрывая друг друга при необходимости.

- **Принцип генерируемости**. Основное положение этого принципа определяет такой способ исходного представления ПО, который бы позволял осуществлять настройку на конкретную конфигурацию технических средств, круг решаемых проблем, условия работы пользователя.
- **Принцип функциональной избыточности**. Этот принцип учитывает возможность выполнения одной и той же работы (функции) различными средствами. Особенно важен учет этого принципа при разработке пользовательского интерфейса для выдачи данных из-за психологических различий в восприятии информации.
- **Принцип умолчания**. Применяется для облегчения организации связей с системой как на стадии генерации, так и при работе с уже готовым ПО. Принцип основан на хранении в системе некоторых базовых описаний структур, модулей, конфигураций оборудования и данных, заранее определяющих условия работы с ПО. Эту информацию ПО использует в качестве заданной, если пользователь забудет или сознательно не конкретизирует ее.

Общесистемные принципы

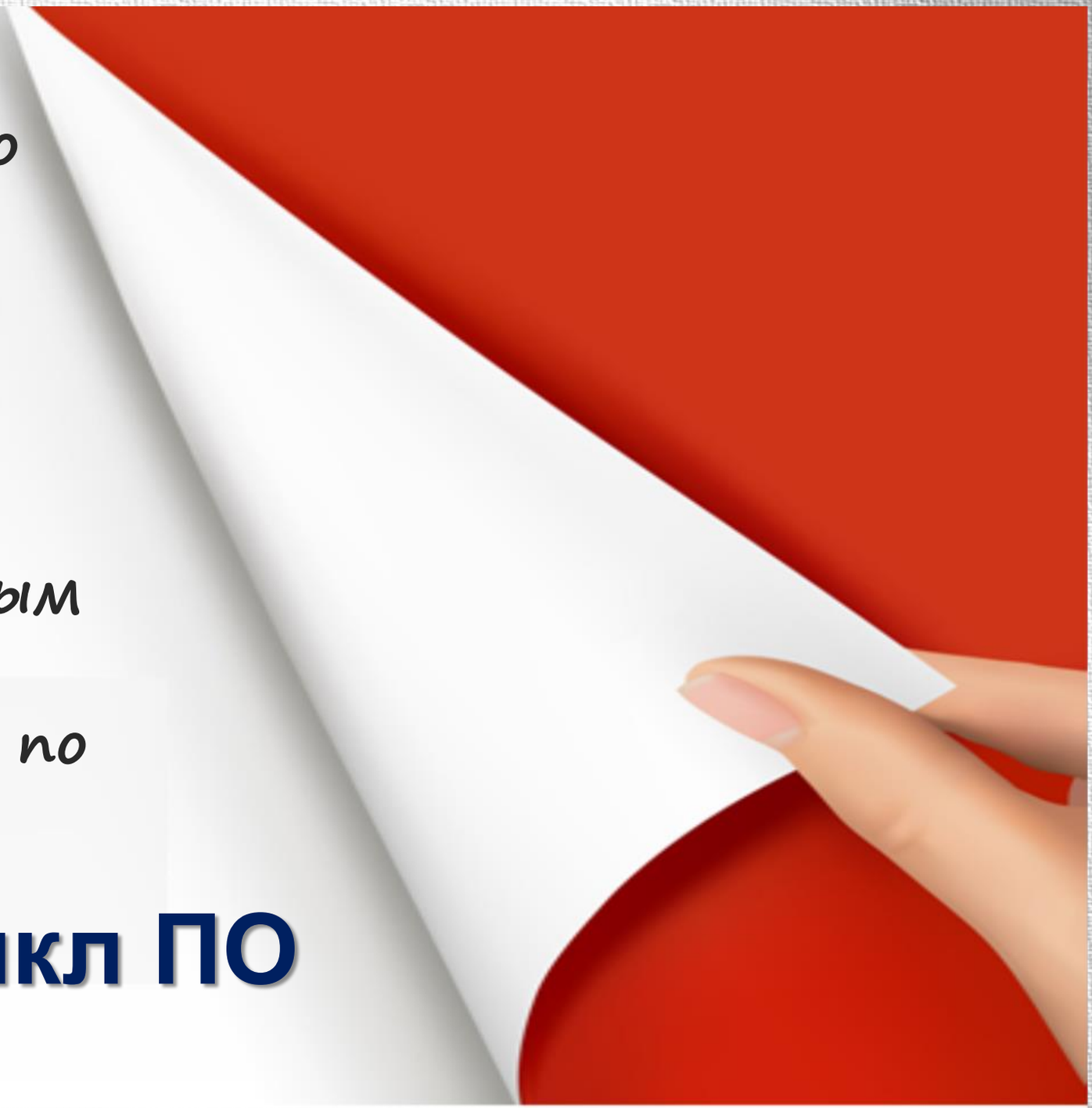
При создании и развитии ПО рекомендуется применять следующие общесистемные принципы:

- **принцип включения**, который предусматривает, что требования к созданию, функционированию и развитию ПО определяются со стороны более сложной, включающей его в свой состав системы;
- **принцип системного единства**, который состоит в том, что на всех стадиях создания, функционирования и развития ПО его целостность будет обеспечиваться связями между подсистемами, а также функционированием подсистемы управления;
- **принцип развития**, который предусматривает в ПО возможность его наращивания и совершенствования компонентов и связей между ними;

- **принцип комплексности**, который заключается в том, что ПО обеспечивает связность обработки информации как отдельных элементов, так и для всего объема данных в целом на всех стадиях обработки;
- **принцип информационного единства**, т. е. во всех подсистемах, средствах обеспечения и компонентах ПО используются единые термины, символы, условные обозначения и способы представления;
- **принцип совместимости**, который состоит в том, что язык, символы, коды и средства обеспечения ПО согласованы, обеспечивают совместное функционирование всех его подсистем и сохраняют открытой структуру системы в целом;
- **принцип инвариантности**, который предопределяет, что подсистемы и компоненты ПО инвариантны к обрабатываемой информации, т.е. являются универсальными или типовыми.

Жизненный цикл ПО

- Процессы жизненного цикла ПО
- Стандарт ISO/IBC 12207
- Стадии и этапы жизненного цикла
- Управление жизненным циклом приложений
- Стадии разработки по ГОСТ 19.102-77



Концепция CDIO, международные стандарты CDIO Standards

Любая продукция или услуга проходит свой жизненный цикл, который включает в себя период от возникновения потребности в создании продукции (услуги) до её ликвидации вследствие исчерпания потребительских свойств. Международный стандарт ГОСТ Р ИСО/МЭК 15288–2005 "Системная инженерия. Процессы жизненного цикла систем" определяет жизненный цикл как эволюцию системы, продукции, услуги, проекта или иного рукотворного объекта от замысла до прекращения использования.

Одним из примеров системного и комплексного подхода к проектированию, реализации и оценке качества современных программ базового инженерного образования является концепция **CDIO** (***Conceive, Design, Implement, Operate – Планировать, Проектировать, Производить, Применять***). Концепция разработана в рамках международного проекта CDIO Initiative и в настоящее время реализуется в более чем ста университетах мира. Международный проект CDIO, инициированный мировым лидером инженерного образования – MIT (США) и тремя техническими университетами Швеции, направлен на установление консенсуса между теорией и практикой в инженерном образовании.

Цепочка "**Задумка – Проектирование – Реализация – Управление**" **(CDIO)** выступает своеобразной "молекулой ДНК", в которой содержится вся "генетическая" информация об эволюции рукотворных объектов (систем, продукции, изделий, услуг, проектов) от замысла до прекращения использования. Это и дает нам основание использовать философию CDIO не только для совершенствования практики инженерного образования.

Жизненный цикл программного обеспечения

В основе деятельности по созданию и использованию программного обеспечения лежит понятие жизненного цикла (ЖЦ). Жизненный цикл определяется как период времени, который начинается с момента принятия решения о необходимости создания программного обеспечения и заканчивается в момент его полного изъятия из эксплуатации.

Стандарты жизненного цикла ПО

- ГОСТ 34.601-90 — предусматривает стадии и этапы создания автоматизированной системы
- ISO/IEC 12207:2008 «System and software engineering — Software life cycle processes» (российский аналог — ГОСТ Р ИСО/МЭК 12207-2010 Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств)

Фазы жизненного цикла

Согласно стандарту жизненный цикл программы, программной системы, программного продукта включает в себя фазы жизненного цикла

- ***Разработка***
- ***Поддержка***
- ***Развертывание***
- ***Сопровождение***

Поддержка заключается в разработке новой функциональности, переработке уже существующей функциональности, в связи с изменением требований, и улучшением продукта, а также устранение некритических замечаний к ПО, выявленных при его эксплуатации .

Развертывание. Если программный продукт не коробочный, а достаточно сложный, то его развертывание у клиентов, как правило, реализуется отдельными самостоятельными проектами внедрения.

Сопровождение включает в себя устранение критических неисправностей в системе и реализуется часто не как проект а, как процессная деятельность.

Фазы ЖЦ


Разработка и поддержка
Проект разработки версии 1.0
Проект разработки версии 2.0
Развертывание
Проект внедрения версии 1.0
Проект внедрения версии 2.0
Сопровождение
Время

ПРОЦЕССЫ ЖИЗНЕННОГО ЦИКЛА

(МЕТАМОДЕЛИ ПРОЦЕССА РАЗРАБОТКИ)

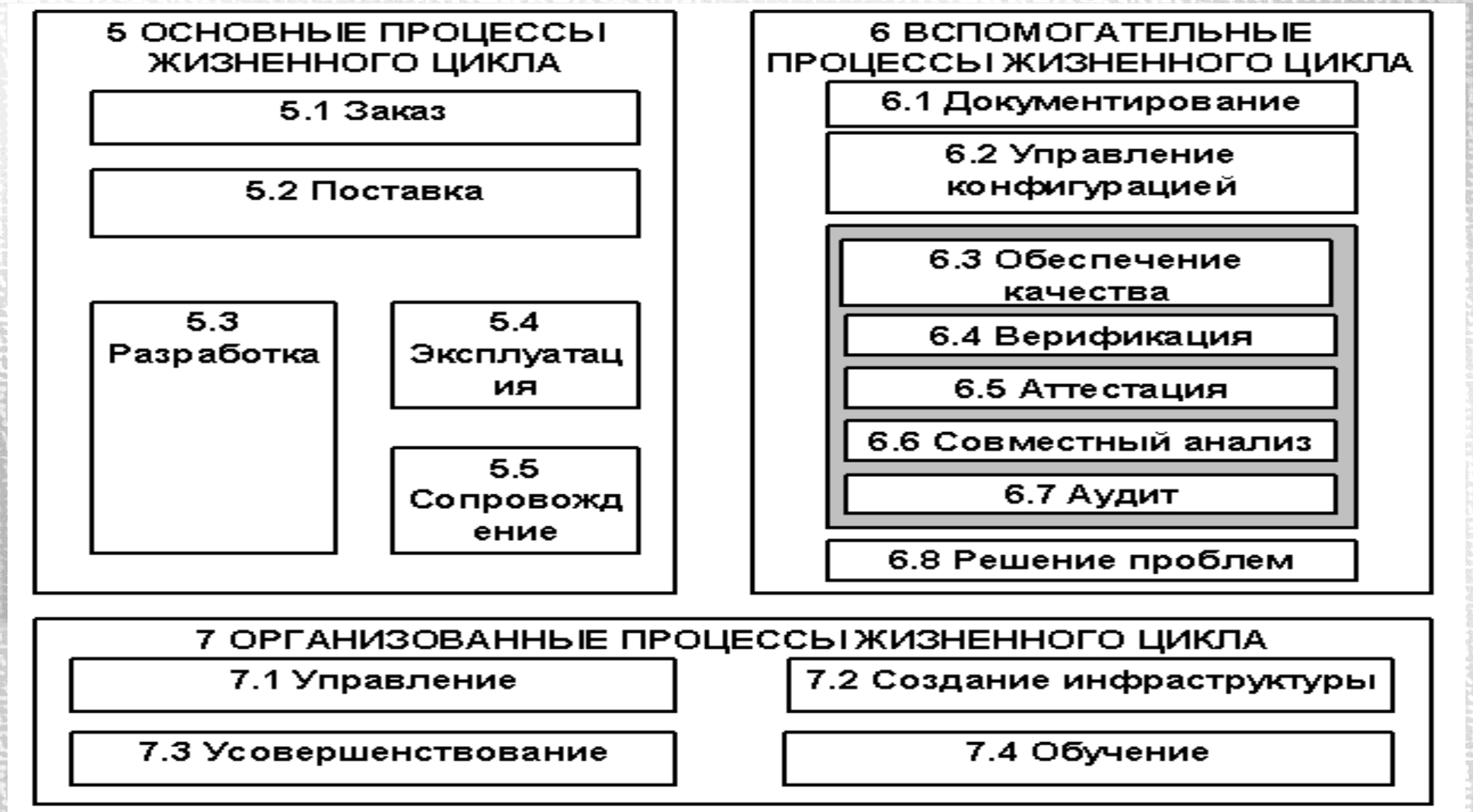
Процессы жизненного цикла

Основным нормативным документом, регламентирующим состав процессов ЖЦ, является **международный стандарт ISO/IBC 12207**. В соответствии со стандартом все процессы ЖЦ разделены на три группы:

- **Основные процессы**
- **Вспомогательные процессы**
- **Организационные процессы**

Стандарт описывает структуру процессов ЖЦ, но не конкретизирует в деталях как реализовать или выполнить действия и задачи, включенные в эти процессы.

Процессы жизненного цикла ПС по ГОСТ Р ИСО/МЭК 12207



Процессы, действия и задачи

Стандарт определяет структуру жизненного цикла, содержащую процессы, действия и задачи , которые должны быть выполнены во время создания ПО.

Каждый процесс разделен на набор действий, каждое действие — на набор задач. Каждый процесс, действие или задача инициируется и выполняется другим процессом по мере необходимости, причем не существует заранее определенных последовательностей выполнения. Связи по входным данным при этом сохраняются.

Пример

Процесс

Приобретение (действия и задачи заказчика, приобретающего ПО)

Действия

1. Инициирование приобретения
2. **Подготовка заявочных предложений**
3. Подготовка и корректировка договора
4. Надзор за деятельностью поставщика
5. Приемка и завершение работ

Задачи

1. Формирование требований к системе
2. Формирование списка программных продуктов
3. Установление условий и соглашений
4. Описание технических ограничений (среда функционирования системы и т. д.)

Основные процессы

- приобретение (заказ)
- поставка
- разработка
- эксплуатация
- сопровождение

Вспомогательные процессы

- документирование
- управление конфигурацией
- обеспечение качества
- верификация
- аттестация
- совместная оценка
- аудит
- решение проблем

Организационные процессы

- управление проектом
- создание инфраструктуры проекта
- усовершенствование: определение, оценка и улучшение самого ЖЦ
- обучение

СТАДИИ И ЭТАПЫ ЖИЗНЕННОГО ЦИКЛА

Стадии жизненного цикла ПО, взаимосвязь между процессами и стадиями

Модель жизненного цикла ПО — структура, определяющая последовательность выполнения и взаимосвязи **процессов, действий и задач** на протяжении жизненного цикла. Модель жизненного цикла зависит от специфики, масштаба и сложности проекта и специфики условий, в которых система создается и функционирует.

Стандарт ГОСТ Р ИСО/МЭК 12207-2010 не предлагает конкретную модель жизненного цикла. Его положения являются общими для любых моделей жизненного цикла, методов и технологий создания ИС. Он описывает структуру процессов жизненного цикла, не конкретизируя, как реализовать или выполнить действия и задачи, включенные в эти процессы.

Модель ЖЦ ПО включает в себя:

- Стадии;
- Результаты выполнения работ на каждой стадии;
- Ключевые события — точки завершения работ и принятия решений.

Стадия — часть процесса создания ПО, ограниченная определенными временными рамками и заканчивающаяся выпуском конкретного продукта (моделей, программных компонентов, документации), определяемого заданными для данной стадии требованиями.

На каждой стадии могут выполняться несколько процессов, определенных в стандарте ГОСТ Р ИСО/МЭК 12207-2010, и наоборот, один и тот же процесс может выполняться на различных стадиях. Соотношение между процессами и стадиями также определяется используемой моделью жизненного цикла ПО.

1. **Процесс планирования**, который определяет и координирует действия процессов разработки и интегральных процессов для данного проекта.

2. **Процессы разработки**, в ходе выполнения которых создается программное средство. Этими процессами являются:
   — процесс определения требований к ПО;
   — процесс проектирования ПО;
   — процесс кодирования ПО;
   — процесс интеграции.

3. **Интегральные процессы**, которые обеспечивают корректную реализацию и качество выполнения процессов разработки и их выходных данных:
   — процесс верификации ПО;
   — процесс управления конфигурацией ПО;
   — процесс обеспечения качества ПО;
   — процесс сертификационного сопровождения.

Этапы жизненного цикла

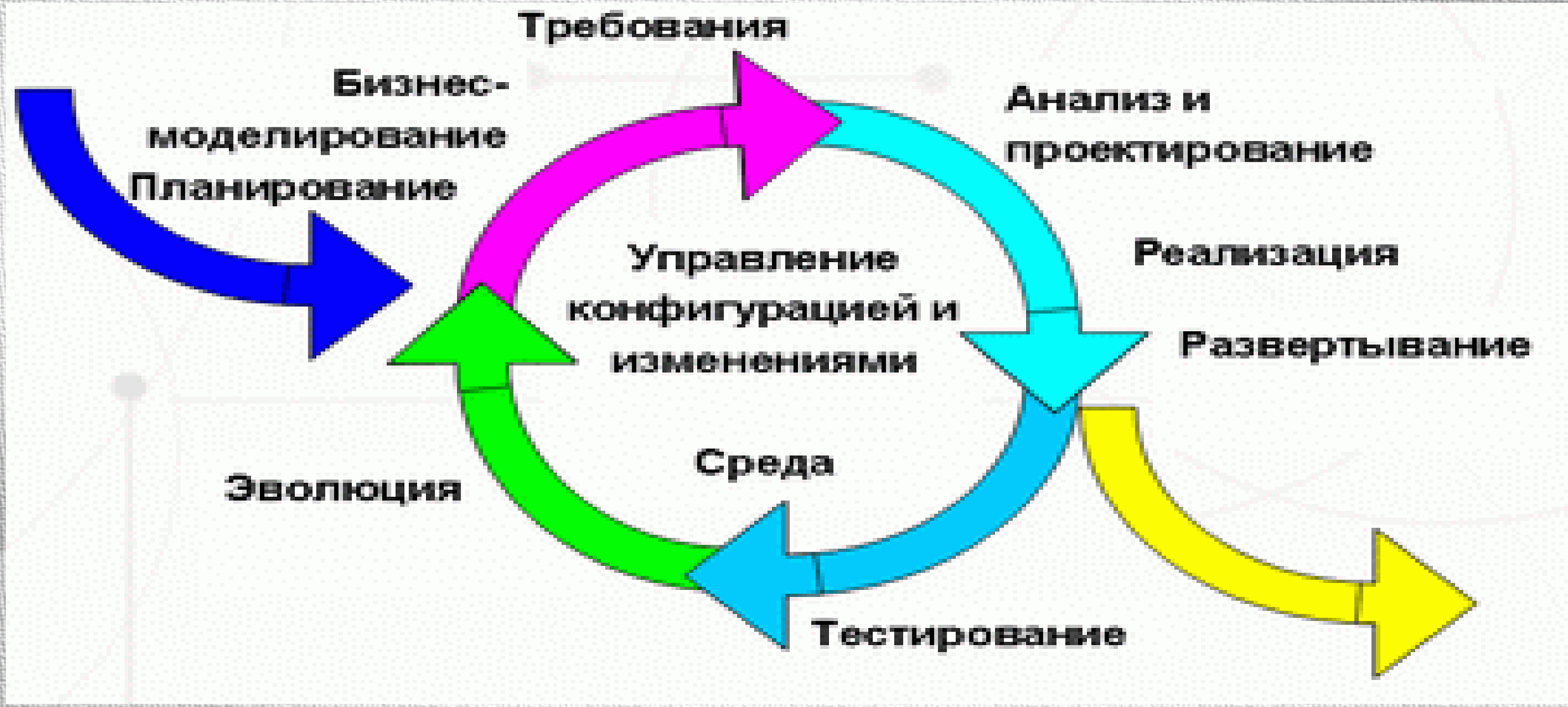
Традиционно, во всех стандартных моделях, выделяют следующие основные этапы жизненного цикла:

1. **планирование, бизнес-моделирование;**
2. **анализ (определение требований);**
3. **проектирование (предварительное и детальное);**
4. **реализация (или кодирование, программирование);**
5. **тестирование и отладка;**
6. **развертывание, внедрение;**
7. **эксплуатация и сопровождение.**

Каждому этапу соответствуют определенный результат и набор документации, являющейся исходными данными для следующего этапа

Этапы жизненного цикла

1. Requirement (определение требований);
2. Analysis&Design (анализ и проектирование);
3. Implementation (реализация, кодирование, программирование);
4. Testing (тестирование и отладка);
5. Deployment (развертывание);


Требования
Бизнес-
моделирование
Планирование
Анализ и
проектирование
Управление
конфигурацией и
изменениями
Реализация
Развертывание
Среда
Эволюция
Тестирование

Стратегическое планирование (системный анализ)

Стратегического планирование предполагает обследование системы. Основная задача обследования — это оценка реального объема проекта, его целей и задач, а также получение определений сущностей и функций на высоком уровне.

Если архитектура – это скелет проекта, то концепция – это его дух. Нет концепции, и от проекта остаётся бездыханное тело, от которого в скором времени начинает смердеть. Но когда она есть, проект развивается, и чем качественнее выполнена концепция, тем более здоровым будет проект, и тем более уверенными и успешными будут участники проектной команды.

Начальная подготовка

- Область исследования
- Предмет исследования
- Проблема
- Потребности
- Цель
- Основные задачи

Бизнес-моделирование

Формирование компактного упрощённого представления о бизнесе, предназначенного для целостного представления и анализа деятельности всей системы взаимосвязанных бизнес-процессов

Анализ. Требования к системе

Разработка требований к системе подразумевает определение ее функциональных возможностей, пользовательских требований, требований к надежности и безопасности, требований к внешним интерфейсам и т.д.

Отвечаем на вопрос «Что делать?»

Проектирование
(архитектурное и детальное)

На **этапе проектирования** разрабатывается архитектура будущей ПС, а также программные компоненты и связи между ними

Отвечаем на вопрос «Как делать?»

Реализация

Реализация подразумевает выбор языка программирования и составление текста программы (кодирование), а также, возможно, выполнение тестирования и отладки отдельных фрагментов.

Тестирование и отладка

Этап тестирования и отладки включает выполнение комплексного тестирования всей программной системы специальной группой и исправление ошибок.

Интеграция и внедрение

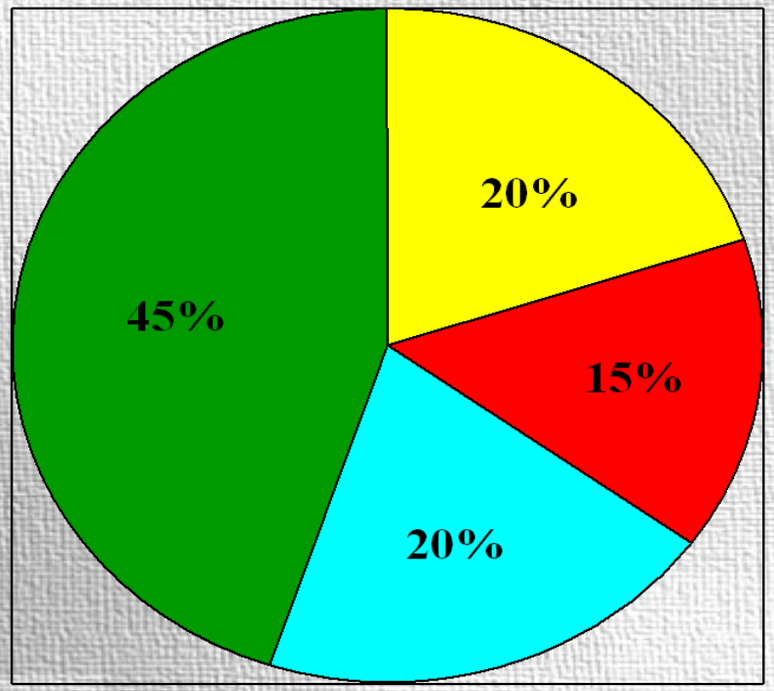
- Интеграция отдельных компонентов системы в единый программный комплекс.
- Общая отладка всей программной системы (системное тестирование)
- Внедрение. Создание инсталляционного пакета, технической документации.

Сопровождение и эксплуатация

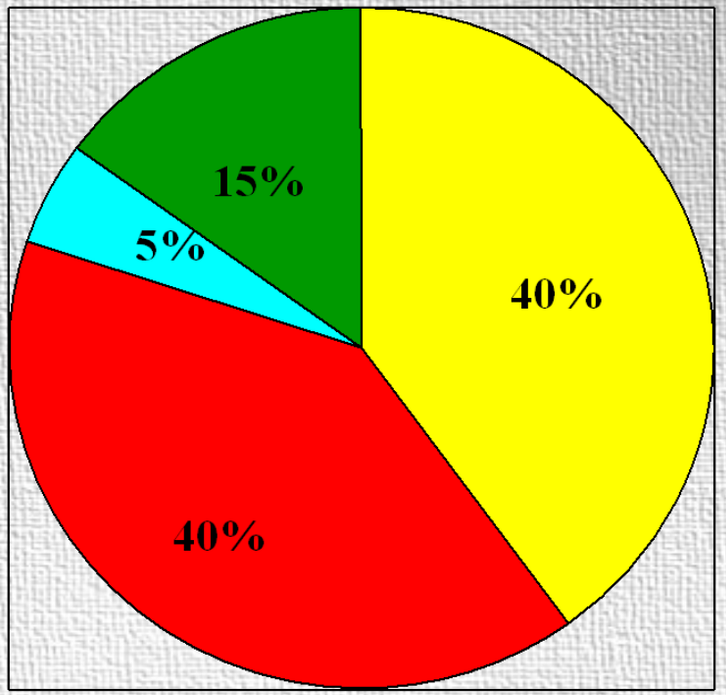
На **этапе сопровождения и эксплуатации** программная система сдается в эксплуатацию, происходит обслуживание пользователей, возможно устранение незначительных ошибок (сейчас это делается повсеместно с помощью распространения так называемых patch - файлов).

Оценка трудозатрат по этапам ЖЦ

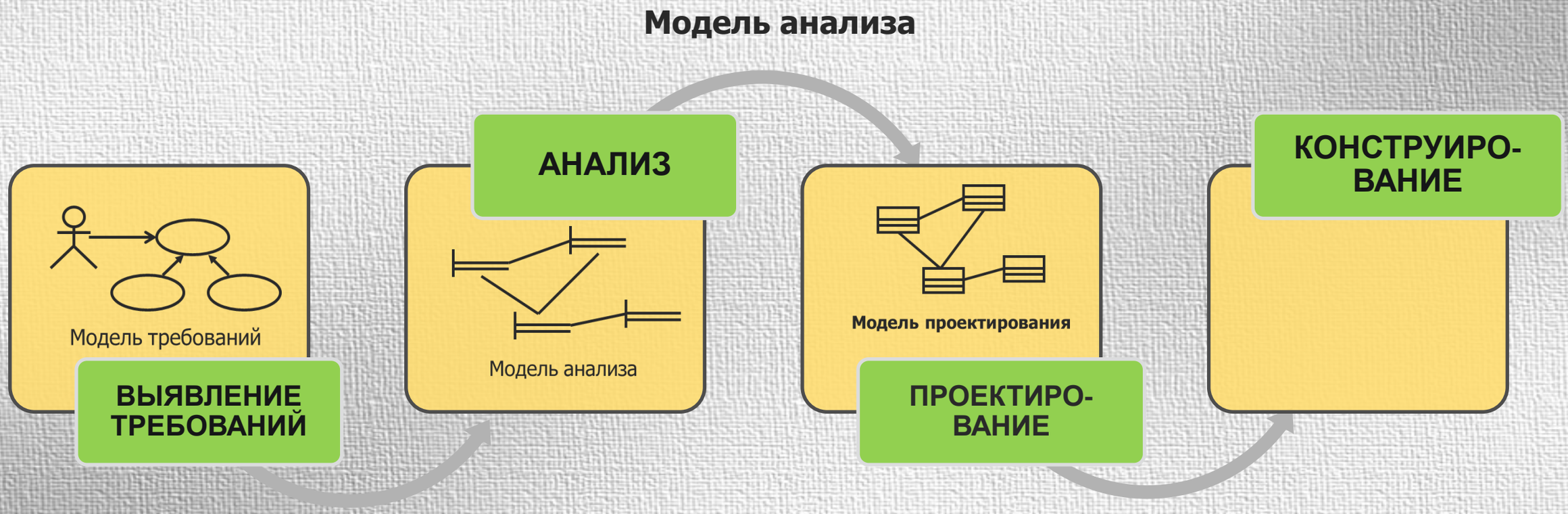
Каноническое проектирование



Автоматизированное проектирование



МОДЕЛИ ЭТАПОВ РАЗРАБОТКИ



Модель требований

1. Бизнес-модель
2. Спецификация требований
3. Словарь

Модель проектирования

1. Модель данных
2. Документ Архитектура

ТЕХНОЛОГИЧЕСКИЕ ПРОЦЕССЫ И ФАЗЫ В МЕТОДОЛОГИИ RUP

Учитывая итерационный характер разработки в методологии Rational Unified Process (RUP) взамен этапам вводят понятия технологических процессов и стадий (фаз).

См в слайдах UML **ТРЕБОВАНИЯ И ВАРИАНТЫ ИСПОЛЬЗОВАНИЯ В RUP**

Технологические процессы

В RUP определено **9** технологических процессов, для каждого из которых предложена методика выполнения. Технологические процессы делятся на две категории – **основные процессы** и **процессы поддержки**.

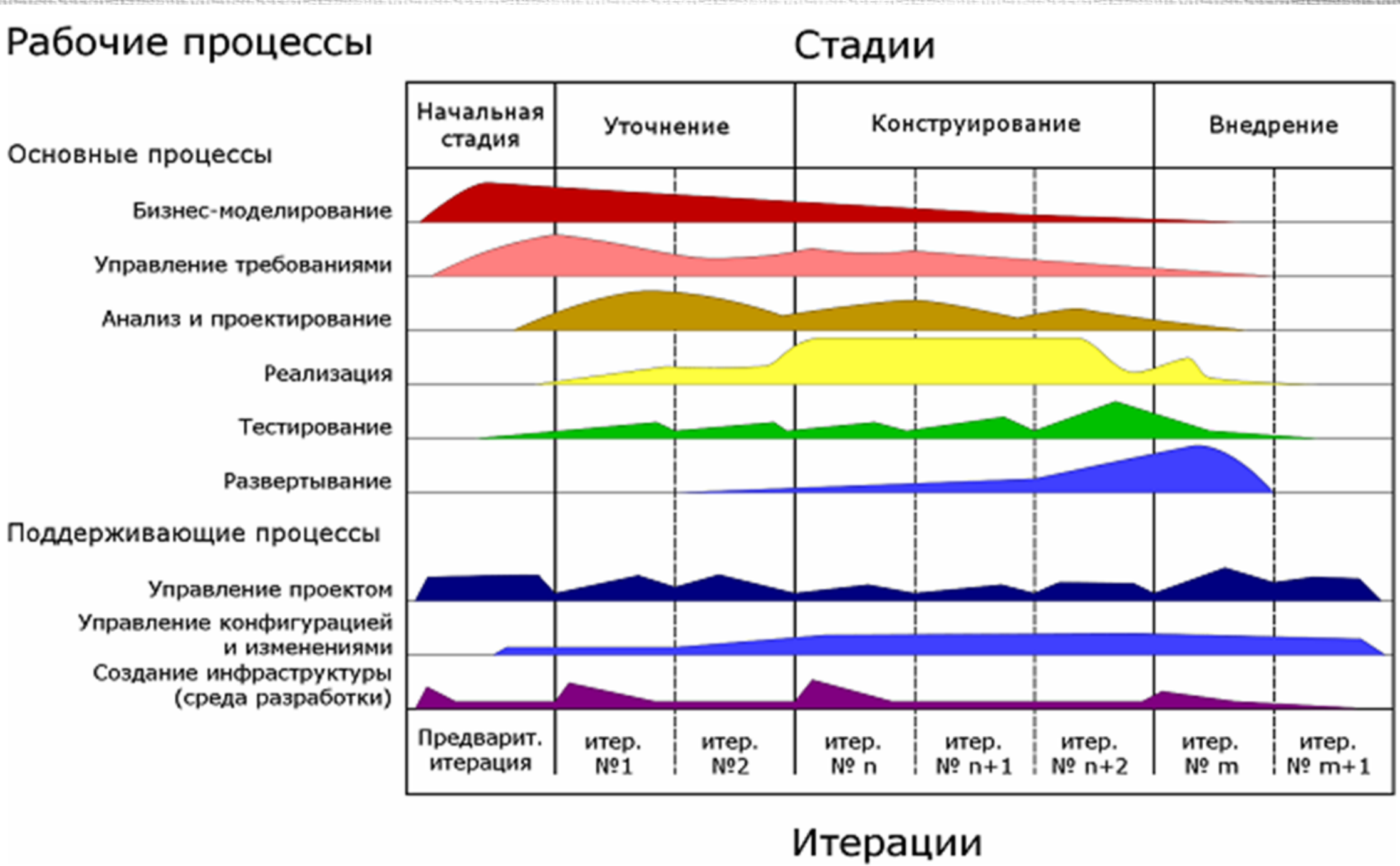
К основным относятся:

1. **Бизнес-анализ**
2. **Управление требованиями**
3. **Анализ и проектирование**
4. **Реализация**
5. **Тестирование**
6. **Развертывание**

Вспомогательные процессы включают:

7. **Управление проектом,**
8. **Управление конфигурацией,**
9. **Управление средой.**

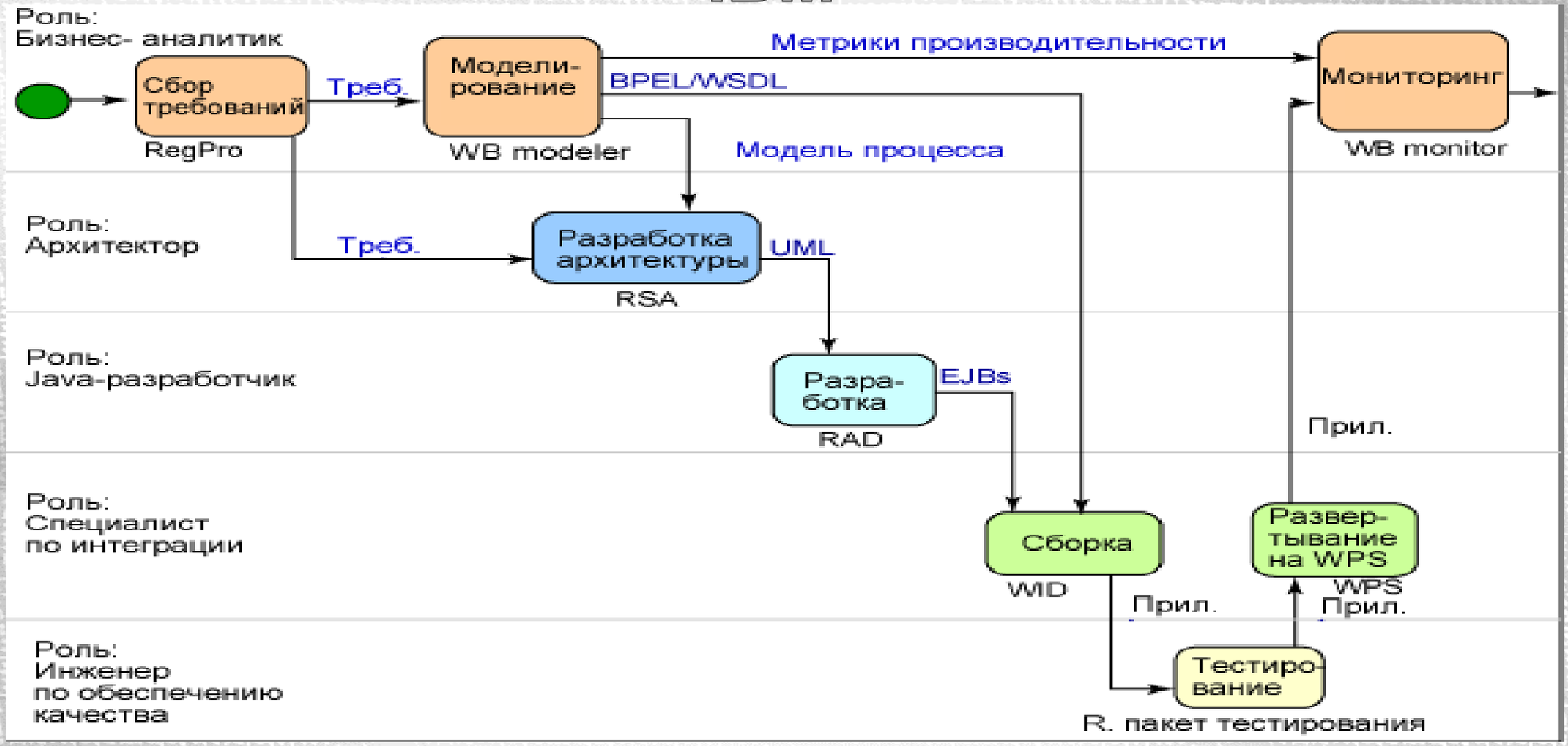
Для каждого технологического процесса предусмотрены роли, определяющие поведение и обязанности отдельных лиц и групп, работающих в одной команде (например, системный аналитик, тестировщик), виды деятельности, определяющие работы, выполняемые исполнителями (например, проектирование класса, проектирование ВИ) и артефакты – документы, используемые, порождаемые или модифицируемые процессом. Основные артефакты в RUP – модель, элемент модели, документ, исходный код, исполняемая программа.


Рабочие процессы
Стадии
Начальная стадия
Уточнение
Конструирование
Внедрение
Основные процессы
Бизнес-моделирование
Управление требованиями
Анализ и проектирование
Реализация
Тестирование
Развертывание
Поддерживающие процессы
Управление проектом
Управление конфигурацией и изменениями
Создание инфраструктуры (среда разработки)
Предварит. итерация
итер. №1
итер. №2
итер. № n
итер. № n+1
итер. № n+2
итер. № m
итер. № m+1
Итерации

Для каждого технологического процесса предусмотрены:

- **роли**, определяющие поведение и обязанности отдельных лиц и групп, работающих в одной команде (например, системный аналитик, тестировщик),
- **виды деятельности**, определяющие работы, выполняемые исполнителями (например, проектирование класса, проектирование ВИ) и
- **артефакты** – документы, используемые, порождаемые или модифицируемые процессом. Основные артефакты в RUP – модель, элемент модели, документ, исходный код, исполняемая программа.

Процесс разработки, с основными ролями, которые поддерживаются ПО компании IBM



УПРАВЛЕНИЕ ЖИЗНЕННЫМ ЦИКЛОМ ПРИЛОЖЕНИЙ

УПРАВЛЕНИЕ ЖИЗНЕННЫМ ЦИКЛОМ ПРИЛОЖЕНИЙ

Концепция управления программным проектом на всех этапах его жизни

ALM

Улучшению процессов создания программного обеспечения служит **методология *управления жизненным циклом приложений*** (*ALM* - **a**pplication **l**ife cycle **m**anagement), которая представляет собой концепцию управления программным проектом на всех этапах его жизни. *ALM* определяет непрерывный *процесс управления* жизненным циклом приложения *по* его управлению, развитию и обслуживанию. Принципы *ALM* реализуются ИТ-решениями различных вендоров.

Инструментарий управления жизненным циклом приложений

- Решение HP ALM on SaaS компании Hewlett-Packard
- Решение Open ALM компании Borland
- Решение Rational ClearQuest компании IBM
- Visual Studio и Team Foundation Server компании Microsoft

СТАДИИ РАЗРАБОТКИ

В разработке программного обеспечения, стадии разработки программного обеспечения используются для

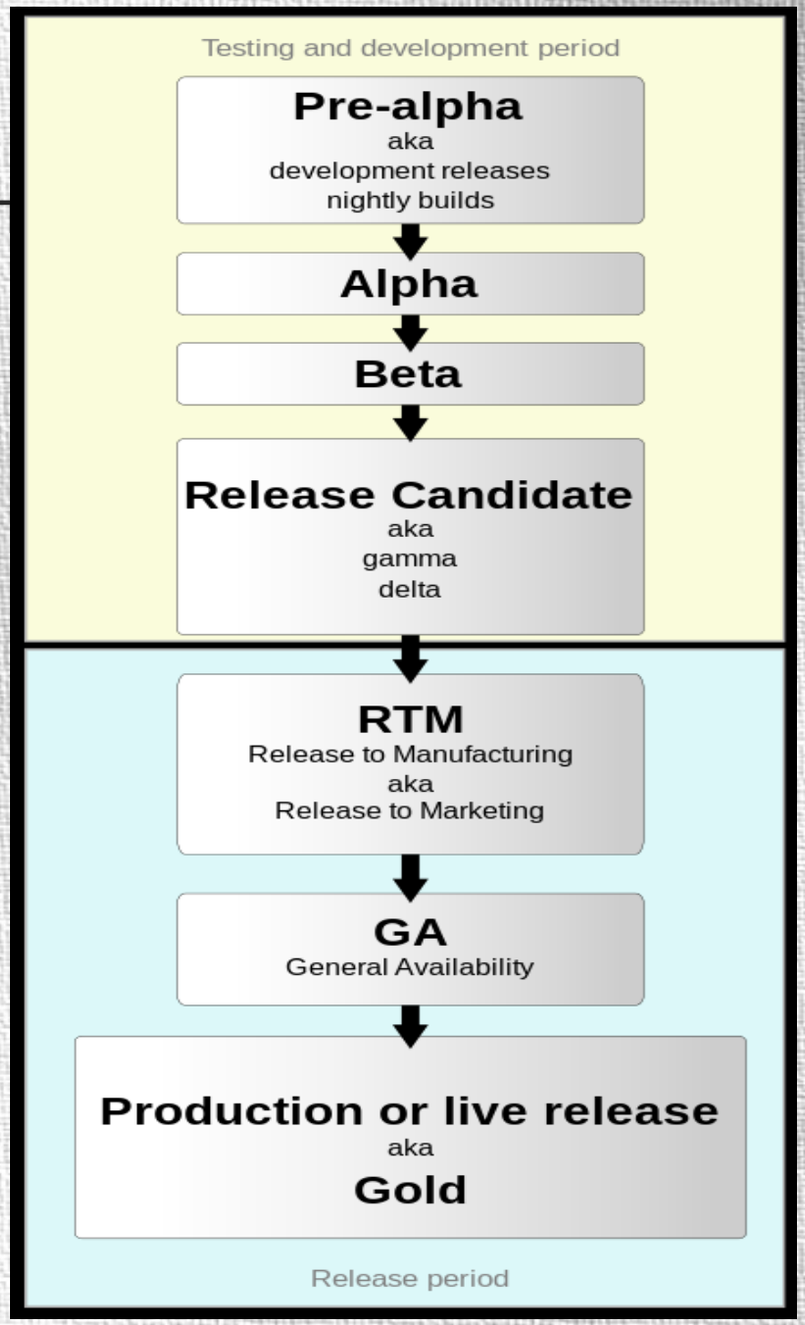
- описания степени готовности программного продукта.
- отражения количество реализованных функций, запланированных для определённой версии программы.

Стадии либо могут быть официально объявлены и регламентируются разработчиками, либо иногда этот термин используется неофициально для описания состояния продукта.

Каждой стадии разработки ПО присваивается определенный порядковый номер. Также каждый этап имеет свое собственное название, которое характеризует готовность продукта на этой стадии.

Жизненный цикл разработки ПО:

- **Пре-альфа**
- **Альфа**
- **Бета**
- **Релиз-кандидат**
- **Релиз или RTM**
- **Пост-релиз**
- **Общая доступность (general availability, GA)**



Пре-альфа - начальная стадия разработки

Период времени со старта разработки до выхода стадии Альфа (или до любой другой, если стадии Альфа нет). Также так называются программы, не вышедшие еще в стадию альфа или бета, но прошедшие стадию разработки, для первичной оценки функциональных возможностей в действии.

В отличие от альфа и бета версий, пре-альфа может включать в себя не весь спектр функциональных возможностей программы. В этом случае, подразумеваются все действия выполняемые во время проектирования и разработки программы вплоть до тестирования. К таким действиям относятся — разработка дизайна, анализ требований, собственно разработка приложения, а также отладка отдельных модулей.

Альфа - внутреннее тестирование

Стадия начала тестирования программы в целом специалистами-тестерами, обычно не разработчиками программного продукта, но, как правило, внутри организации или сообществе разрабатывающих продукт. Также это может быть стадия добавления новых функциональных возможностей. Программы на данной стадии могут применяться только для ознакомления с будущими возможностями.

Бета - публичное тестирование

Стадия активного бета-тестирования и отладки программы, прошедшей альфа-тестирование (если таковое было). Программы этого уровня могут быть использованы другими разработчиками программного обеспечения для испытания совместимости. Тем не менее, программы этого этапа могут содержать достаточно большое количество ошибок.

Поскольку бета-продукт не является финальной версией, и публичное тестирование производится на страх и риск пользователя, производитель не несёт никакой ответственности за ущерб, причинённый в результате использования бета-версии. Таким образом, многие производители уходят от ответственности, предоставляя пользователям только бета-версии продукта.

Релиз-кандидат

Релиз-кандидат или **RC** (англ. release candidate), Пре-релиз или Pre, иногда «**гамма-версия**» — стадия-кандидат на то, чтобы стать стабильной. Программы этой стадии прошли комплексное тестирование, благодаря чему были исправлены все найденные критические ошибки. Но в то же время существует вероятность выявления ещё некоторого числа ошибок, не замеченных при тестировании.

Релиз

Релиз или RTM (*release to manufacturing* промышленное издание) — издание продукта, готового к тиражированию. Это стабильная версия программы, прошедшая все предыдущие стадии, в которых исправлены основные ошибки, но существует вероятность появления новых, ранее не замеченных, ошибок. RTM предшествует общей доступности (GA), когда продукт выпущен для общественности.

Пост-релиз

Пост-релиз или Post-RTM - издание продукта, у которого есть несколько отличий от RTM и помечается как самая первая стадия разработки следующего продукта. Такие релизы не выпускаются на продажу, а раздаются бета-тестировщикам. Это издание может быть либо стабильным (если не замечено ошибок), либо с ошибками.

Стадии разработки. ГОСТ 19.102-77

Настоящий стандарт устанавливает стадии разработки программ и программной документации для вычислительных машин, комплексов и систем независимо от их назначения и области применения.

1. Техническое задание
2. Эскизный проект
3. Технический проект
4. Рабочий проект

Обследование и разработка технического задания



Для успешного создания программы, разработчику необходимо провести обследование с целью выявления всех требований заказчика к создаваемому программному продукту. На данном этапе происходит уяснение поставленной задачи. Специалисты выезжают к заказчику, уточняют требования, анализируют бизнес процессы, общаются со специалистами.
Со стороны заказчика должны быть выделены сотрудники для взаимодействия.

Аналитики компании разработчика могут запросить у заказчика образцы входящей и исходящей документации, участвующей в автоматизируемых бизнес-процессах.

ГОСТ 19.102-77
(техническое задание)

Стадии разработки	Этапы работ	Содержание работ
1. Техническое задание	Обоснование необходимости разработки программы	Постановка задачи Сбор исходных материалов Выбор и обоснование критериев эффективности и качества разрабатываемой программы. Обоснование необходимости проведения научно-исследовательских работ.
	Научно-исследовательские работы	Определение структуры входных и выходных данных. Предварительный выбор методов решения задач. Обоснование целесообразности применения ранее разработанных программ. Определение требований к техническим средствам. Обоснование принципиальной возможности решения поставленной задачи Определение требований к программе. Разработка технико-экономического обоснования разработки программы.
	Разработка и утверждение технического задания	Определение стадий, этапов и сроков разработки программы и документации на неё. Выбор языков программирования. Определение необходимости проведения научно-исследовательских работ на последующих стадиях. Согласование и утверждение технического задания.

ГОСТ 19.102-77
(эскизный проект)

2. Эскизный проект	Разработка эскизного проекта	Предварительная разработка структуры входных и выходных данных. Уточнение методов решения задачи. Разработка общего описания алгоритма решения задачи Разработка технико-экономического обоснования. Разработка пояснительной записки.
	Утверждение эскизного проекта	Согласование и утверждение эскизного проекта

ГОСТ 19.102-77
(технический проект)

3. Технический проект	Разработка технического проекта	Уточнение структуры входных и выходных данных. Разработка алгоритма решения задачи. Определение формы представления входных и выходных данных. Определение семантики и синтаксиса языка. Разработка структуры программы. Окончательное определение конфигурации технических средств. Разработка плана мероприятий по разработке и внедрению программ. Разработка пояснительной записки.
	Утверждение технического проекта	Согласование и утверждение технического проекта.

ГОСТ 19.102-77
(рабочий проект)

4. Рабочий проект	Разработка программы	Программирование и отладка программы.
	Разработка программной документации	Изготовление программы-оригинала Разработка программных документов в соответствии с требованиями ГОСТ 19.101-77.
	Испытания программы	Разработка, согласование и утверждение порядка и методики испытаний. Проведение предварительных, государственных, межгосударственных, межведомственных, приёмосдаточных и других видов испытаний. Корректировка программы и программной документации по результатам испытаний.

Стандарт ГОСТ 34.601-90

Стандарт ГОСТ 34.601-90 предусматривает следующие стадии и этапы создания автоматизированной системы (АС):

1. **Формирование требований к АС**
   - Обследование объекта и обоснование необходимости создания АС
   - Формирование требований пользователя к АС
   - Оформление отчета о выполнении работ и заявки на разработку АС
2. **Разработка концепции АС**
   - Изучение объекта
   - Проведение необходимых научно-исследовательских работ
   - Разработка вариантов концепции АС и выбор варианта концепции АС, удовлетворяющего требованиям пользователей
   - Оформление отчета о проделанной работе
3. **Техническое задание**
   - Разработка и утверждение технического задания на создание АС

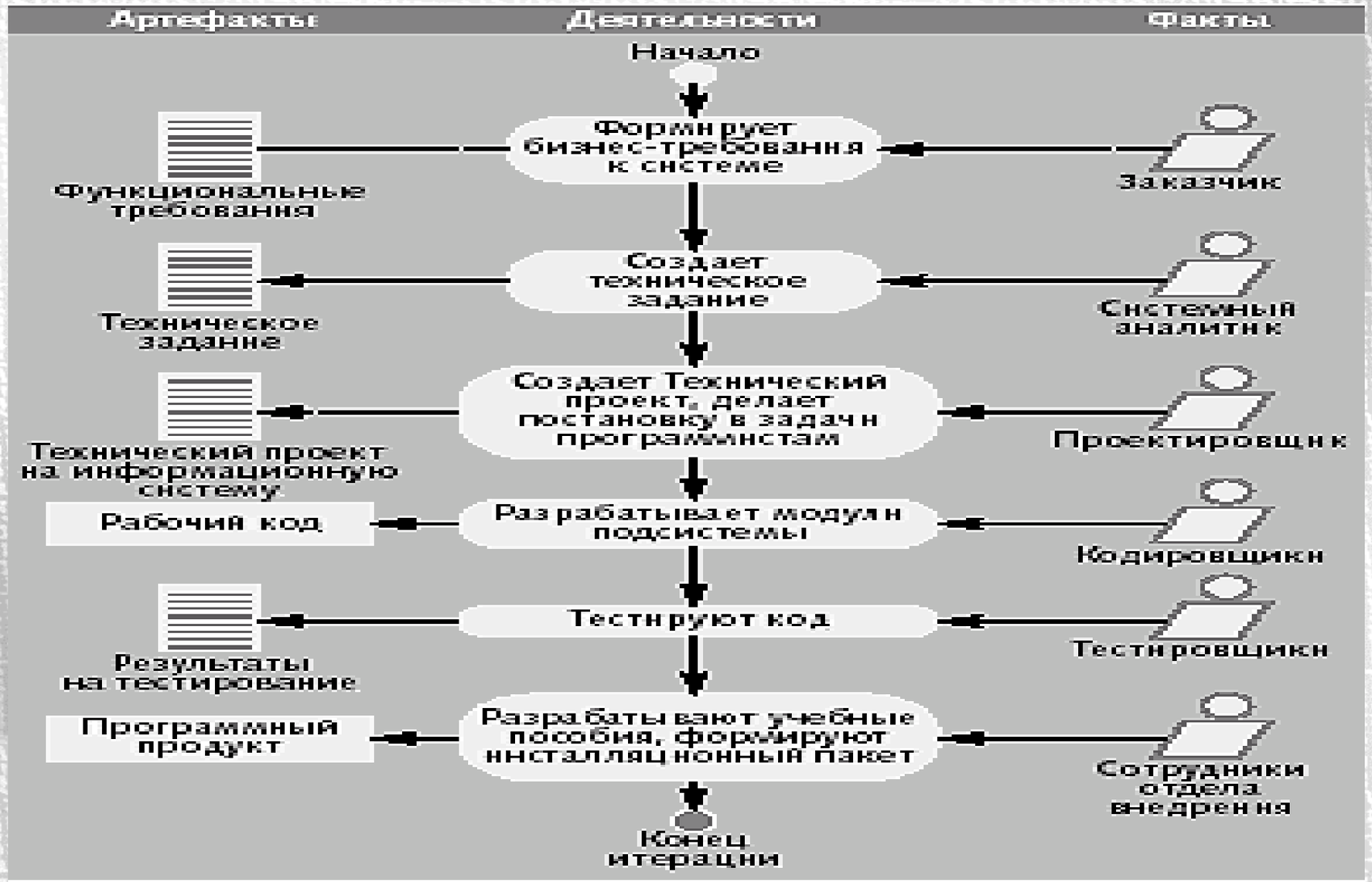
4. **Эскизный проект**
   - Разработка предварительных проектных решений по системе и её частям
   - Разработка документации на АС и её части
5. **Технический проект**
   - Разработка проектных решений по системе и её частям
   - Разработка документации на АС и её части
   - Разработка и оформление документации на поставку комплектующих изделий
   - Разработка заданий на проектирование в смежных частях проекта
6. **Рабочая документация**
   - Разработка рабочей документации на АС и её части
   - Разработка и адаптация программ

7. **Ввод в действие**
   - Подготовка объекта автоматизации
   - Подготовка персонала
   - Комплектация АС поставляемыми изделиями (программными и техническими средствами, программно-техническими комплексами, информационными изделиями)
   - Строительно-монтажные работы
   - Пусконаладочные работы
   - Проведение предварительных испытаний
   - Проведение опытной эксплуатации
   - Проведение приемочных испытаний
8. **Сопровождение АС**
   - Выполнение работ в соответствии с гарантийными обязательствами
   - Послегарантийное обслуживание

Данный стандарт не вполне подходит для проведения разработок в настоящее время: многие процессы отражены недостаточно, а некоторые положения устарели.

Эскизный, технический проекты и рабочая документация — это последовательное построение все более точных проектных решений. Допускается исключать стадию «Эскизный проект» и отдельные этапы работ на всех стадиях, объединять стадии «Технический проект» и «Рабочая документация» в «Технорабочий проект», параллельно выполнять различные этапы и работы, включать дополнительные.

Технология цикла разработки



Стандартные модели жизненного цикла ПО

- Стратегии конструирования ПО
- Каскадная модель (модель водопада)
- Итерационная модель
- Спиральная модель



КРАТКИЙ ИСТОРИЧЕСКИЙ ОБЗОР

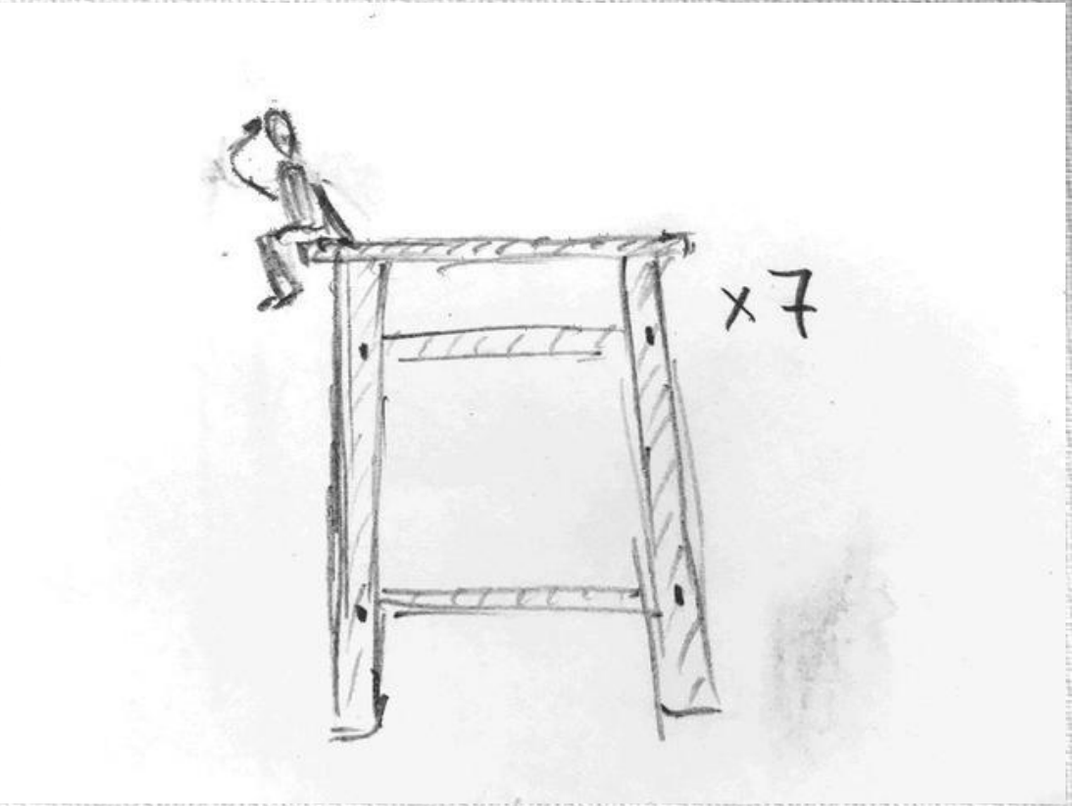
Три способа

- **Нисходящий способ**. Это когда сначала глобально думают «что делать», потом «как делать», а потом уже делают, если время осталось.
- **Восходящий способ.** Это когда сначала делают, понемножку, мелкими частями. Думают «что, да как» только над этой частью, не заморачиваясь на глобальное и непреходящее.
- **Спиральный способ**. Начинаем думать глобально, но по звонку будильника просыпаемся и выкатываем прототип. И так несколько раз, накапливая опыт, пока не появится решение согласно требованиям.

Нисходящая разработка

Создание способа приходится на 1960-е годы, органично дополняя требования структурного программирования. Глобальная задача делилась на более мелкие подзадачи, те, в свою очередь, на еще более детальные и так далее до уровня структурного блока, который можно непосредственно реализовать на языке программирования.

В просторечии все нисходящие методы обзывают «**водопадом**».



Русская пословица «Семь раз отмерь — один раз отрежь» как раз про нисходящий метод.

Восходящий способ

На самом деле, восходящий способ еще древнее нисходящего. В 1950-х уже существовали и Фортран, и Кобол, но не было ясных представлений, как строить софт. Поэтому поступали самым естественным образом: сегодня сделаем одно, завтра другое, потом когда-нибудь склеим в третье. Если нужно реализовать несколько задач, то выбираем наиболее важные.



Иногда способ также называют **инкрементальным**, видимо, по аналогии с i++. Добавили функцию -- инкремент.

Методика была и остается наиболее характерной для домашних (in-house) разработок. Свои программисты должны делать то, что нужно для бизнеса еще вчера. Чтобы думать о более глобальном, в 1960-х появились "софтверхаузы" -- крупные проектные конторы по разработке заказных систем, включая таких монстров, как IBM.

Продолжалось все это восходящее софтостроительство без существенных изменений до 1990-х.

В 1990-х годах пошел мощный тренд замены "домашних" программистов на внешних консультантов. Одна из причин -- усложнение технологий и специализация, труднодостижимая внутри фирмы для которой софтостроение -- непрофильная деятельность. Домашний софт теперь разрабатывается временными командами шабашников контрактников. Изменились отношения, а заказчик -- не спец во всех этих методологиях, анализах, архитектурах. Заказчик в лучшем случае знает, в чем отдельные проблемы и может расставить над ними приоритеты. Команда контрактников в свою очередь, не знает о бизнесе заказчика достаточно, чтобы спланировать всю реализацию.

В такой ситуации простой метод "функция за функцией" снова кажется подходящим. Но при работе с подрядчиком на подневной ставке требуется больше контроля, чем за штатными работниками "на окладе". Подход нужно адаптировать, избегая бюрократических процедур. Действительно, чтобы "выдать техническое задание" требуется общее планирование и соответствующие документы, то есть, воленс-ноленс, пришлось бы переходить на формальные нисходящие или спиральные методы. А зачем это нужно транспортной компании или производителю авто, например? Люди просто хотят решить небольшую внутреннюю проблему, зарплату там посчитать, или свести расписание отпусков.

Спрос рождает предложение, к концу 1990-х появились так называемые проворные **agile** методы, худо-бедно позволявшие заказчику держать руку на пульсе, а подрядчику постепенно понять, что, собственно, они реализуют.

Что улучшилось:

- планирование по-прежнему короткое, но охватывает не одну функцию, а их группу при этом срок этапа жестко ограничен;
- ведется протокол сделанного;
- теоретически, должно выделяться время на упрощение и чистку кода (рефакторинг);
- теоретически, риски регрессии должны парироваться всеобъемлющими тестами.

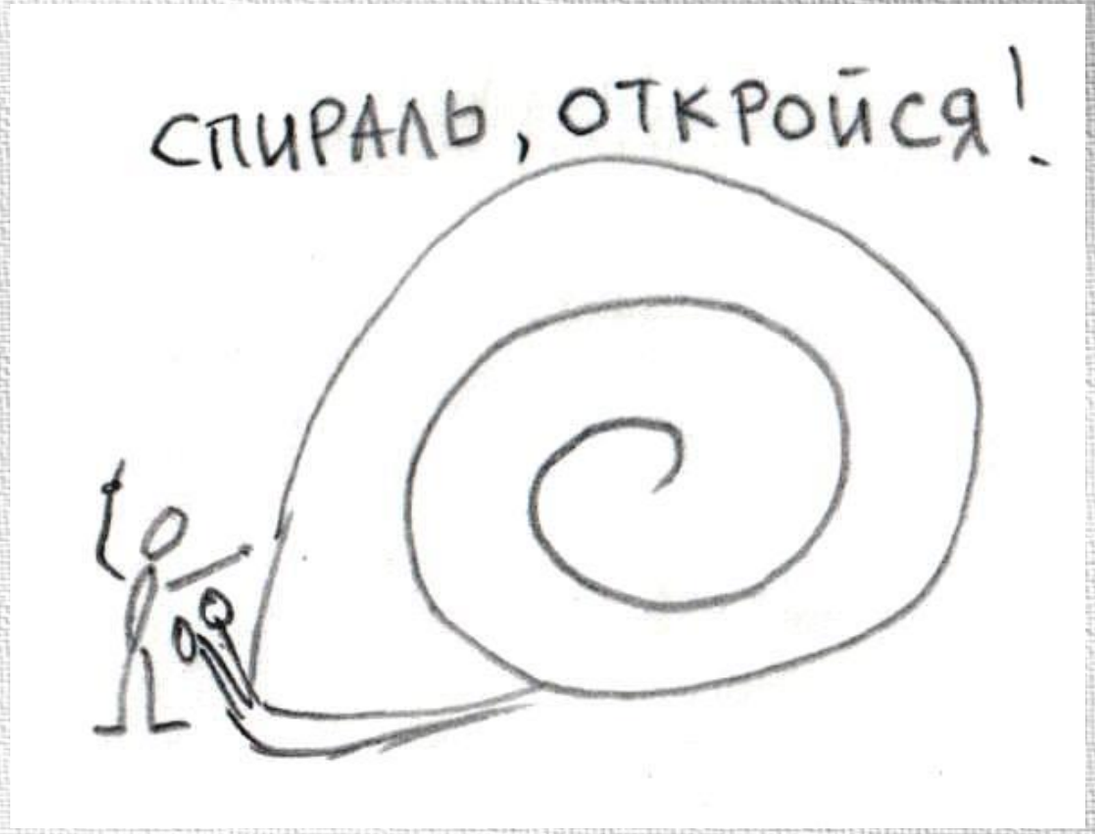
Подведем итоги, что в восходящей разработке хорошего. Прежде всего, резко снижается порог входа. Чтобы запуститься "с нуля" не нужны дорогущие спецы по анализу и проектированию с опытом. Хотя вся стройка может длиться бесконечно, первые бараки возводятся скоро, и в них можно начинать обживаться. Управлять процессом проще, контрольные точки и локальные цели прозрачны.

Проблемы, как и в случае с "водопадами", принципиальные. Проще управлять процессом, но практически невозможно -- конечным результатом, так как он нигде четко не фиксируется. Риски перекладываются на заказчика: пускай владелец продукта думает, у него башка большая.

Если с технической архитектурой при наличии опыта, тестов и рефакторинга команда справляется (до определенного порога сложности), то с архитектурой функциональной дело плохо. Упрощение постановки задачи, тот самый "рефакторинг", теперь уже модели предметной области, помог бы избавиться от навороченного и дорогого в сопровождении кода, но сделать его некому. Да и модели-то нет, **если честно, вся семантика в головах и в коде**.

Спиральная разработка

Упомянутые «водопадные» ограничения стали ясны еще в 1970-х, после массовых внедрений соответствующих методологий типа SADT/IDEF. Параллельно работали и по восходящим методикам на «домашних» проектах с другими заморочками, что создавало ощущение некоторого тупика. Поэтому озадаченные проблемой исследователи выдали к середине 1980-х годов обновленное видение процесса софтостроения в виде спирали.



Логика рассуждений "спиральщика" примерно такова. Да, с ростом сложности задач мы тратим все больше времени на анализ и проектирование, риск ошибок на этой стадии растет. Но реализуя отдельные функции без общего плана мы рискуем ошибиться не меньше, получив в итоге неэффективно работающие или просто несовместимые части. Поэтому оставим в покое "святую корову" -- проектирование, но

а) ограничим ему сроки и

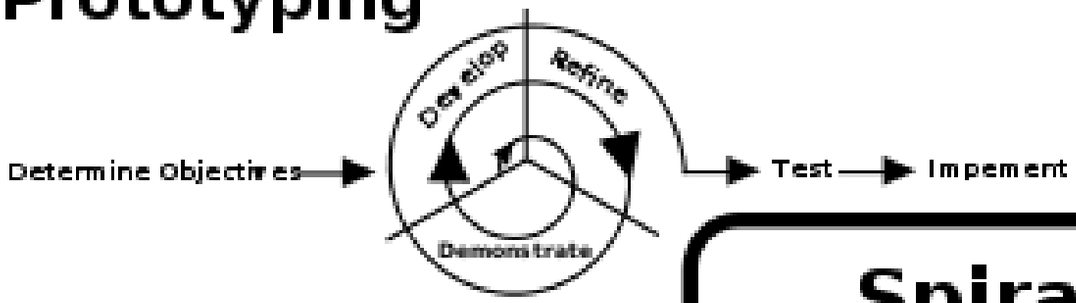
б) будем жестко проверять принятые решения, выкатывая полный прототип.

Пункт "б" очень важен. При восходящей разработке вы тоже постоянно что-то выдаете, багфиксы там, новые фичи. Но это сработает, если система уже в эксплуатации и на сопровождении. А если нет? Представьте, заказчик хочет систему управления движением поездов, а вы ему через пару-тройку недель показываете экраны ввода информации о подвижном составе и симулятор расписания. Несерьезно.

Прототип должен включать в себя большинство функций, пусть не до конца реализованных или даже с "заглушками". С полного прототипа, соответственно, получаем и полный фидбэк: здесь ошиблись, не поняли сути требований, тут не учли среду эксплуатации, там выходной формат не стыкуется с входным и т.д. С этим драгоценным фидбэком начинаем второй виток спирали, обоснованно полагая высоким шанс довести следующий прототип до уровня готового продукта. Системам среднего масштаба (до миллиона строк кода примерно) два-три витка хватает.

Из сказанного понятно, что относить спираль к инкрементным способам так же нелепо, как китов к рыбам, хотя у тех и других есть хвост.

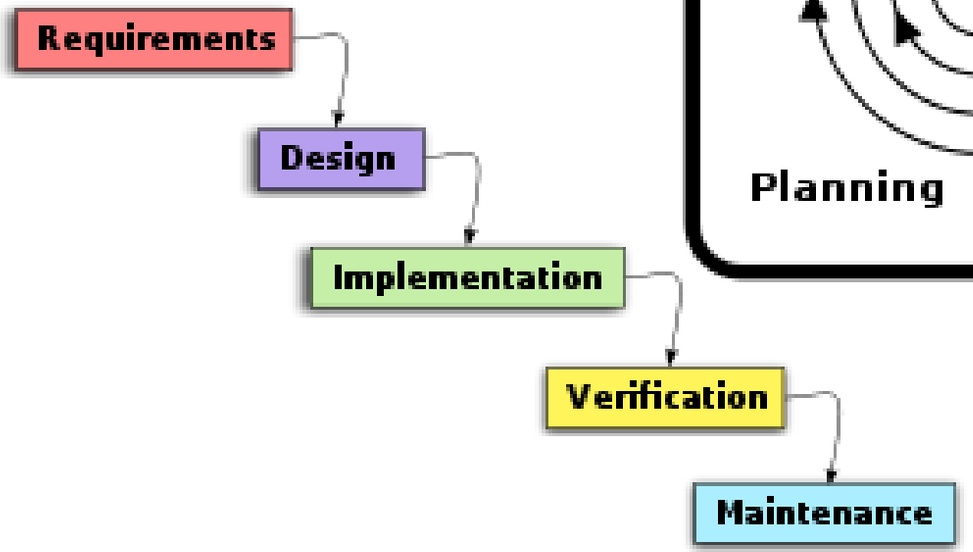
Prototyping



Spiral



Waterfall



СТРАТЕГИИ РАЗРАБОТКИ ПО. ОСНОВНЫЕ ВИДЫ ДЕЯТЕЛЬНОСТИ

Процесс разработки программного обеспечения

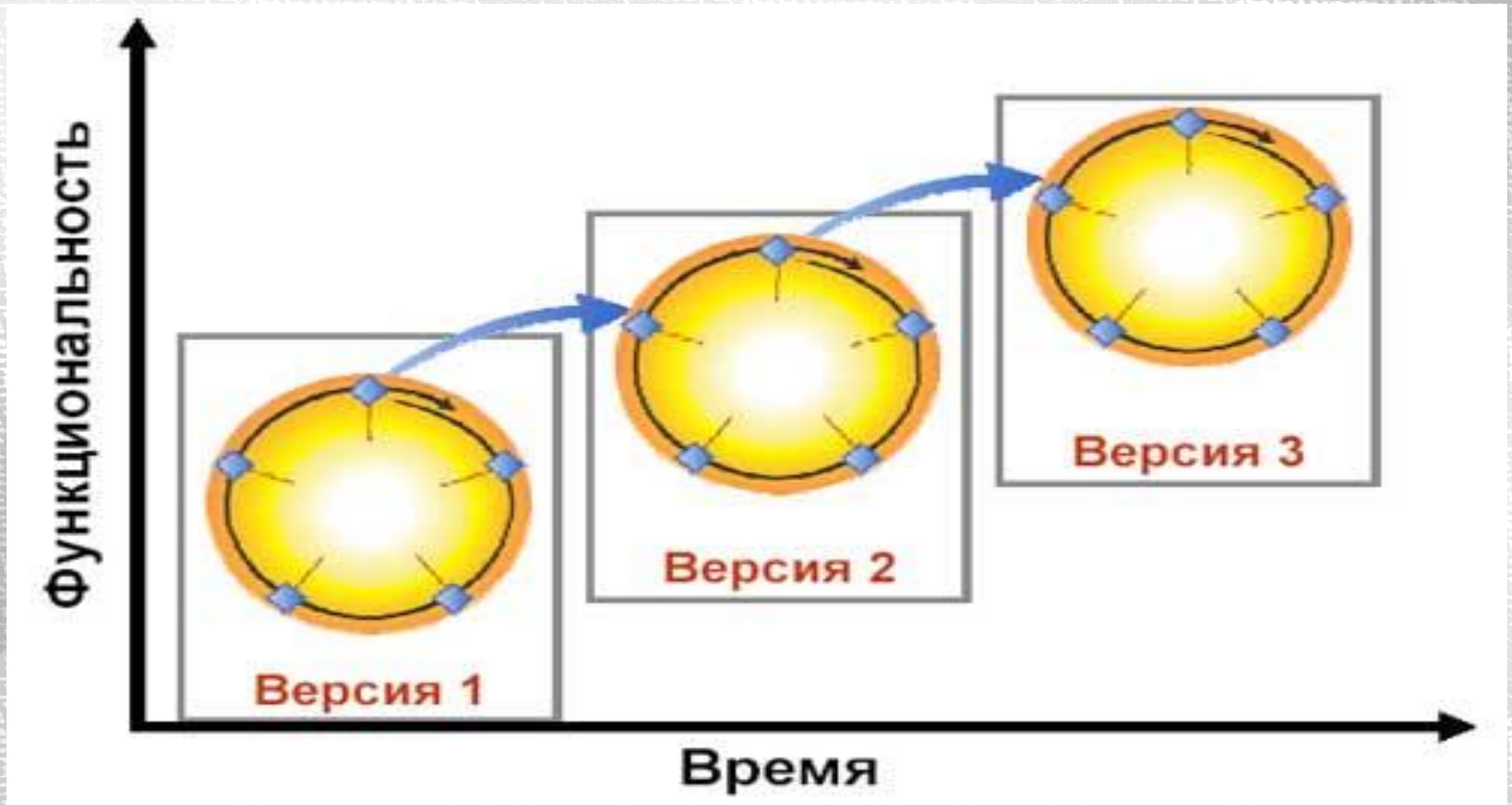
В программной инженерии , процесс разработки программного обеспечения представляет собой процесс разделения разработки программного обеспечения работы на отдельные фазы чтобы улучшить дизайн , управление продуктом и управление проектами . Он также известен как **жизненный цикл разработки программного обеспечения** (SDLC). Методология может включать предварительное определение конкретных результатов и артефактов, которые создаются и завершаются проектной группой для разработки или поддержки приложения.

Стратегии разработки ПО

1. *Однократный проход* (водопадная стратегия)
2. *Инкрементная стратегия* (запланированное улучшение продукта). В начале процесса определяются *все* пользовательские и системные требования. Первая версия реализует часть запланированных возможностей, следующая версия реализует дополнительные возможности и т.д., пока не будет получена полная система.
3. *Эволюционная стратегия*. Система также строится в виде последовательности версий, но вначале процесса определены *не все* требования. Требования уточняются в результате разработки версий.

Стратегии 2 и 3 характеризуются так называемым ***итеративным*** подходом

Итеративный подход



Модель жизненного цикла

- Под моделью ЖЦ понимается структура, определяющая последовательность выполнения и взаимосвязи процессов, действий и задач, выполняемых на протяжении ЖЦ.
- Модель жизненного цикла включает в себя набор этапов и связей между ними. Дается детальное описание действий, моделей и результатов каждого этапа.
- Может быть предложено несколько моделей жизненного цикла, каждая из которых определяет различную методологию создания систем.
- Стандарт ISO/IBC 12207 не предлагает конкретную модель ЖЦ и методы разработки ПО.

ПЯТЬ ВИДОВ ОСНОВНОЙ ДЕЯТЕЛЬНОСТИ

- **ПОДГОТОВКА (сбор требований)**
- **ПЛАНИРОВАНИЕ (оценка, расписание)**
- **МОДЕЛИРОВАНИЕ (анализ и проектирование)**
- **КОНСТРУИРОВАНИЕ (кодирование и тестирование)**
- **РАЗВЕРТЫВАНИЕ (поставка и сопровождение)**

Для многих программных проектов эти виды деятельности выполняются итеративно.

Базис процессов разработки ПО

Обобщённый базис процессов для программной инженерии включает пять видов основной деятельности:

1. **Подготовка**. К любой технической работе нужно правильно подготовиться. Подготовка заключается в тесном сотрудничестве с заказчиком и другими заинтересованными лицами. **Главное - понять цели заинтересованных лиц** в отношении продукта и проекта, собрать их требования к характеристикам и функциям ПО.
2. **Планирование**. По понятным причинам опытный путешественник всегда берет в дорогу карту. Программный проект подобен сложному путешествию, и деятельность планирования создает карту, упрощающую путешествие команды.

Карта называется планом программного проекта. План определяет порядок инженерной работы. Он описывает технические задачи, которые надо выполнять, наиболее вероятные факторы риска, подстерегающие команду, требуемые ресурсы, рабочие продукты (модели, документы, данные, отчеты, формы и т. д.), которые будут созданы, и расписание работы.

3. **Моделирование**. В любой человеческой деятельности каждый день работают с моделями. Почему? Ответ в том, что модель - упрощённое представление реальности. Модели «с высоты птичьего полета» наглядно демонстрируют желаемую структуру и поведение системы (обычно визуально). Модели позволяют лучше понять общую картину эскиз будущего решения. При необходимости эскиз дополняется деталями. Так формируется окончательный способ решения проблемы.

Обычно моделирование включает в себя два действия: **анализ и проектирование.** *Модель анализа* улучшает понимание требований к ПО, а *модель проектирования* показывает эскиз структуры и поведения ПО, выполняющего эти требования.

4. **Конструирование**. Эта деятельность объединяет два действия: **генерацию программного кода** ПО (ручную или автоматическую) и **тестирование**, которое требуется для обнаружения ошибок в коде.
5. **Развертывание**. ПО (окончательный вариант или частично завершенная версия) поставляется заказчику, который оценивает полученный продукт и обеспечивает обратную связь, дающую возможность улучшения продукта. Последний этап нацелен на два действия: **поставку** разработанного продукта заказчику и **сопровождение** процесса эксплуатации этого продукта.

Сегодня развертывание (деплой) принято производить максимально автоматизированным способом -- т.е. писать скрипты (или использовать готовые инструменты), которые автоматизируют:

- перенос кода проекта,
- его адаптацию (скажем, разворот данных БД)
- и запуск на новой машине.

Перенос системы с одной машины на другую должен быть "приятным процессом", а не собиранием её (системы) руками по кускам (файлам), сопровождающимся танцами с бубном

Эти пять видов основной деятельности могут использоваться как в ходе разработки малых, простых программ, так и при создании больших и сложных компьютерных систем. В каждом случае детали процесса (действия, задачи, порядок их выполнения и взаимодействия) будут меняться, но виды деятельности останутся одинаковыми.

Для многих программных проектов перечисленные типы основной деятельности применяются итеративно, по мере развития проекта. То есть подготовка, планирование, моделирование, конструирование и развертывание применятся повторно, в ходе ряда итераций проект. Каждая итерация проекта производит для заинтересованных лиц версию ПО с частичной реализацией характеристики функций. По мере производства очередной версии возможности программного продукта возрастают.

Макетирование

Достаточно часто заказчик не может сформулировать подробные требования по вводу, обработке или выводу данных для будущего программного продукта. С другой стороны, разработчик может сомневаться в приспосабливаемости продукта под операционную систему, форме диалога с пользователем или эффективности реализуемого алгоритма. В этих случаях целесообразно использовать макетирование. Основная цель макетирования - снять неопределенности в требованиях заказчика. Макетирование или прототипирование - это процесс создании модели требуемого программного продукта.

Защитная деятельность

Виды основной деятельности по разработке ПО дополняются набором видов защитной деятельности. Виды защитной деятельности «пронизывают» весь программный проект и помогают команде управлять процессом, контролировать развитие, качество, изменения, риск.

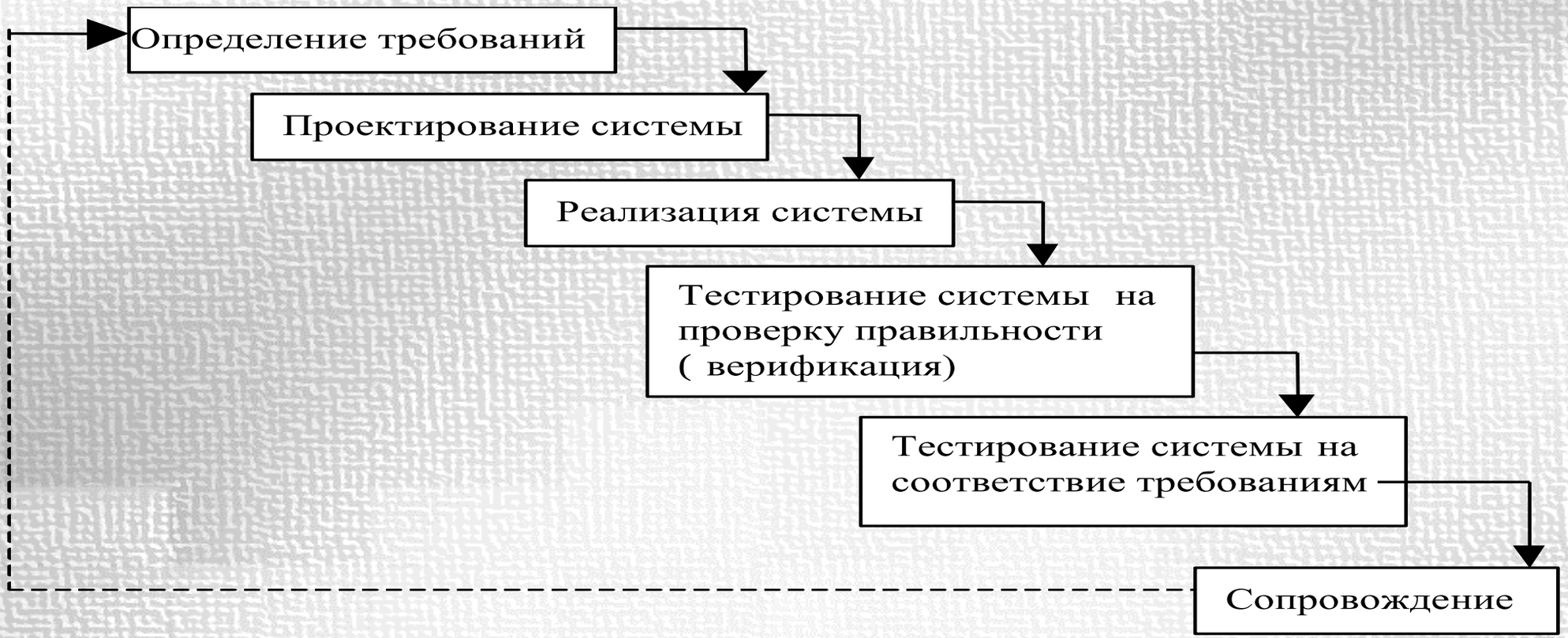
Перечислим типичные виды защитной деятельности.

Виды защитной деятельности

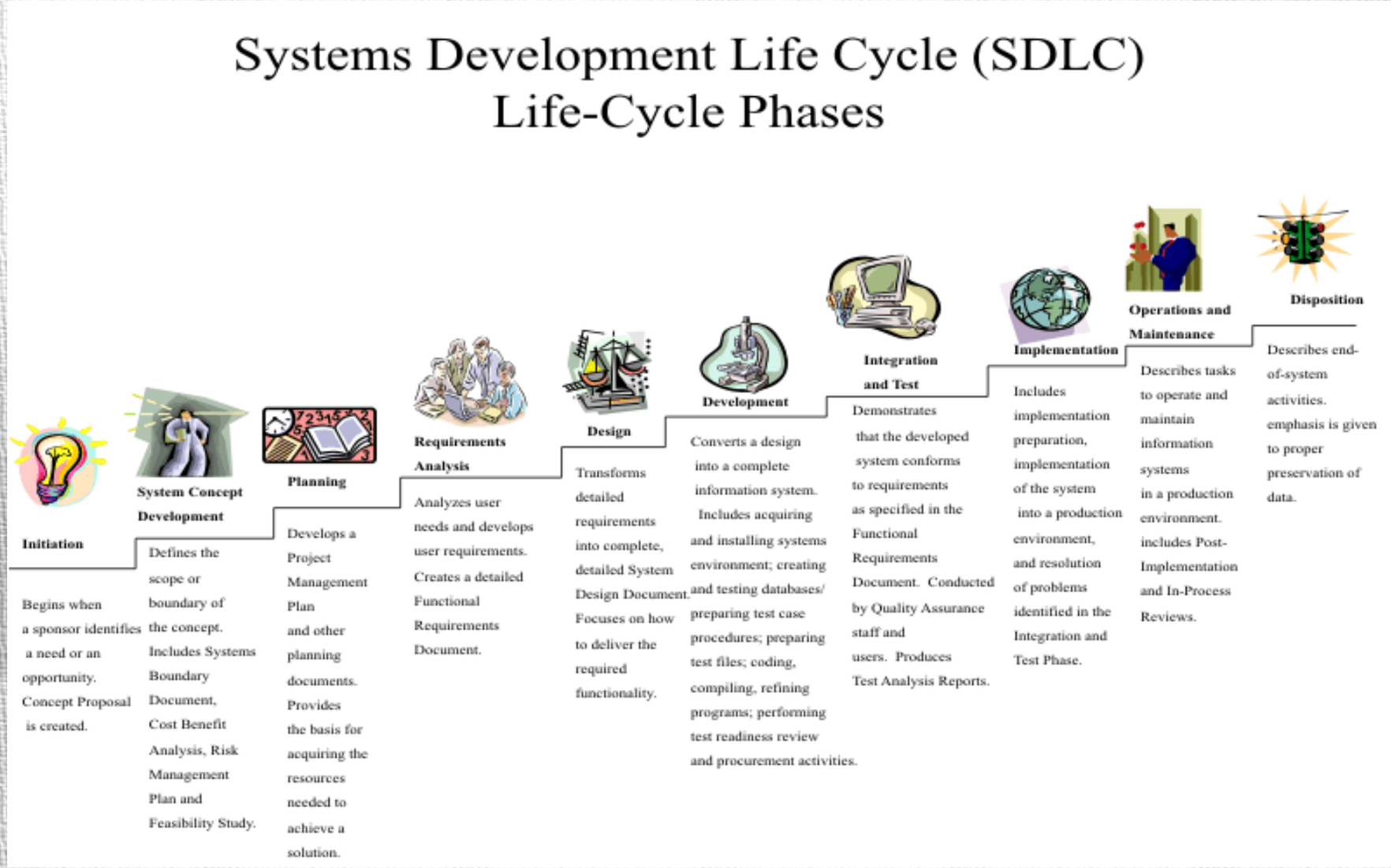
- Отслеживание (трассировка) и контроль программного проекта
- Управление риском
- Обеспечение качества ПО
- Технические проверки
- Измерение
- Управление конфигурацией ПО
- Управление повторной используемостью (reuse)
- Подготовка и производство программного продукта

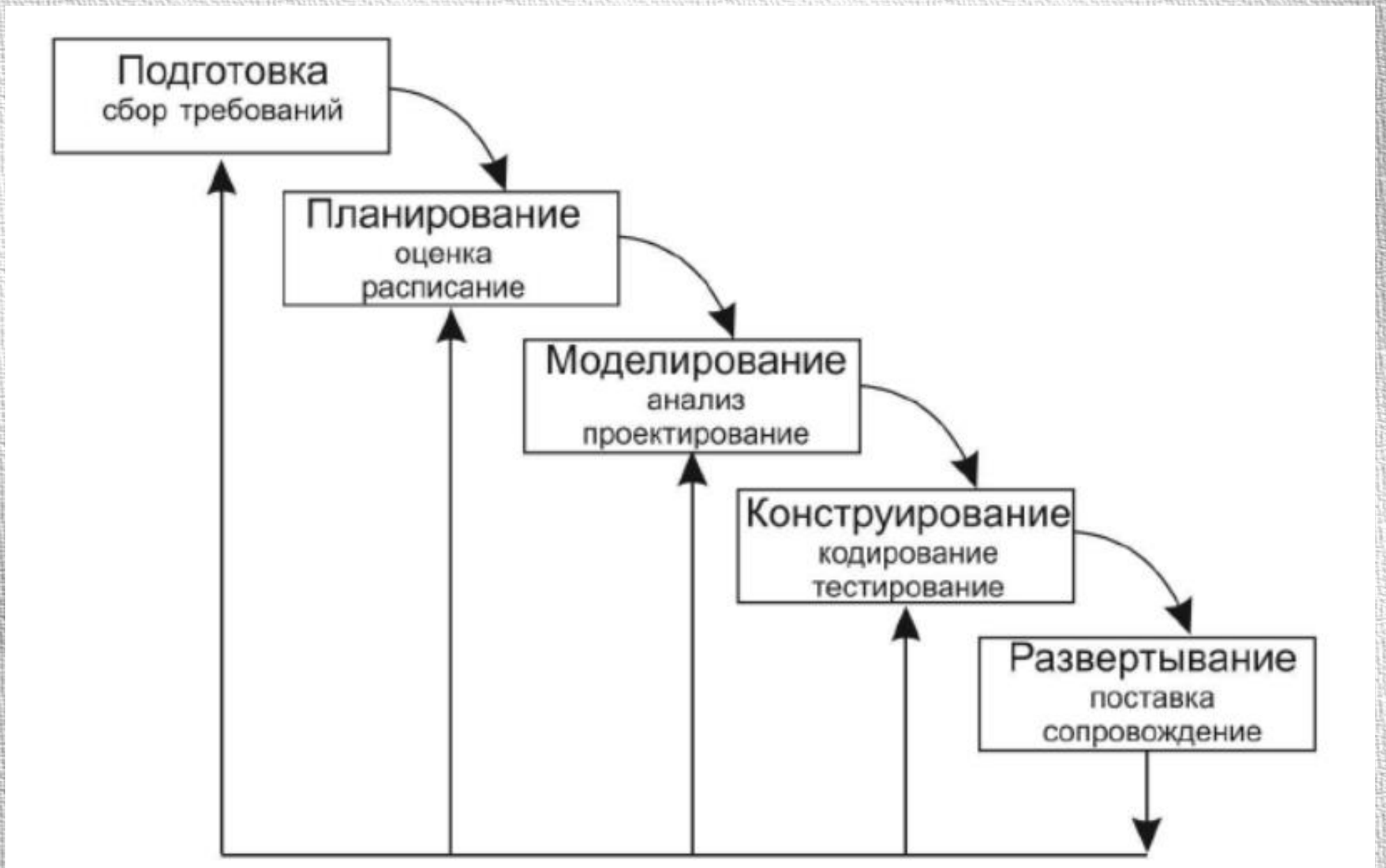
Каскадная модель
(модель водопада)

Первой по времени появления, и самой распространенной являлась каскадная модель или модель водопада



Десятифазная версия жизненного цикла разработки систем




Подготовка
сбор требований
Планирование
оценка
расписание
Моделирование
анализ
проектирование
Конструирование
кодирование
тестирование
Развертывание
поставка
сопровождение

Каскадная модель (модель водопада)

Модель предполагает следующие свойства взаимодействия этапов:

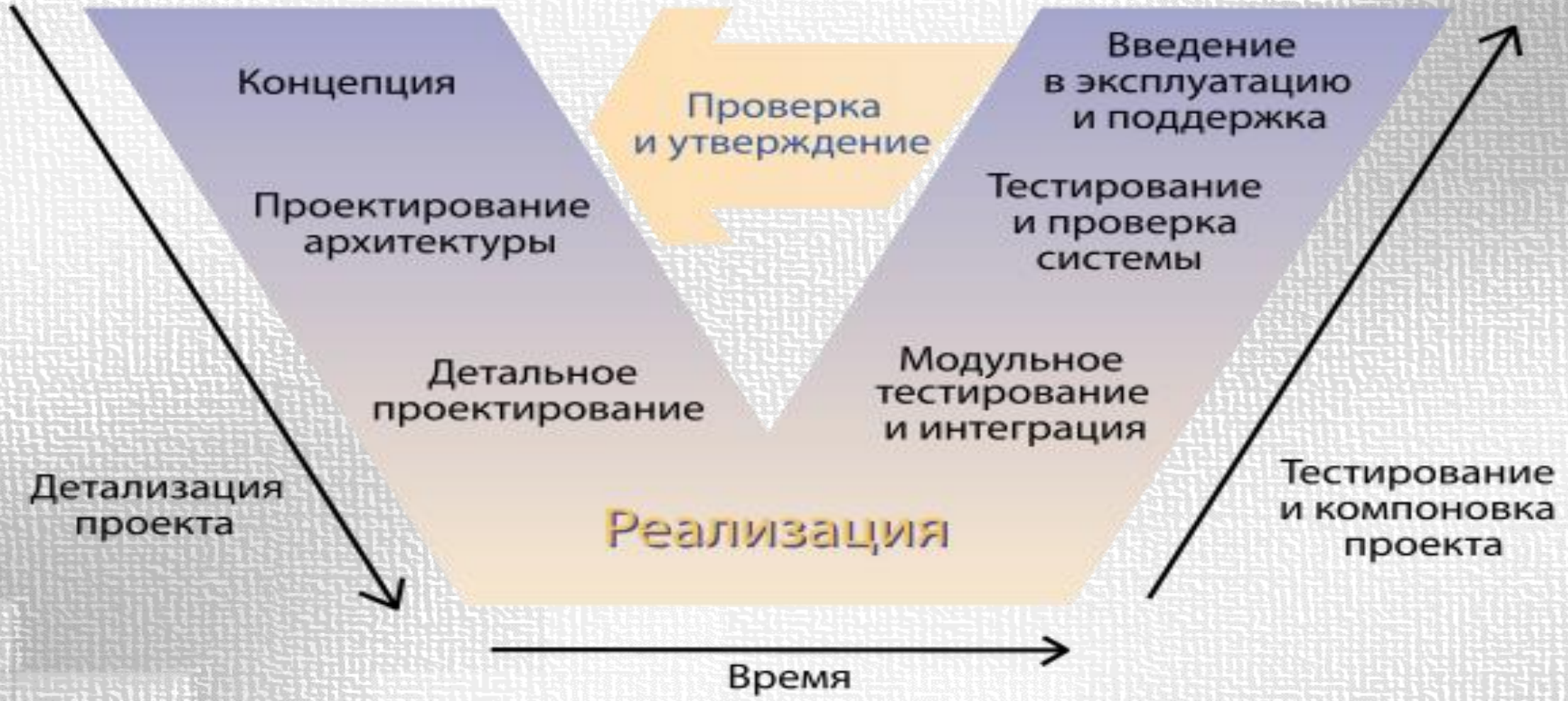
- модель состоит из последовательно расположенных этапов;
- каждый этап полностью заканчивается до того как начнется следующий;
- этапы не перекрываются во времени, т.е. следующий этап не начинается, пока не завершится предыдущий;
- возврат к предыдущим этапам не предусматривается или крайне ограничен;
- результат появляется только в конце разработки.

Главный недостаток такого подхода заключается в том, что любое внесение изменений в требования к ПС обходится достаточно дорого. Это происходит потому, что необходимость внесения изменений обнаруживается при таком подходе обычно к концу разработки поэтому требуется править модель всей системы и вносить изменения в программный код большого объема. Такой подход хорош для маленьких проектов и в тех случаях, когда требования к ПС строго определены и гарантированно не будут изменяться. К сожалению, таких проектов в реальной жизни немного.

Преимущества способа ясны. Чем больше продумано в начале, тем меньше лишней работы придется делать в середине и конце. Исключается многократное переизобретение велосипедов, дублирование функций и ошибок.
Однако, **если функций изначально много, то родить правильную разбивку нелегко**. Нужны хорошие аналитики, проектировщики с опытом работы в данных предметных областях. И даже в этом случае есть риск затянуть планирование реализации на неприличный срок.

V-метод

В дальнейшем способ неоднократно улучшался, например V-метод, где стадиям разработки по левому спуску буквы «V» соответствуют стадии тестов и приемок подъема справа. По горизонтальному уровню сразу видно, какой проектный документ подрядчика служит основанием для приемочного документа заказчика.

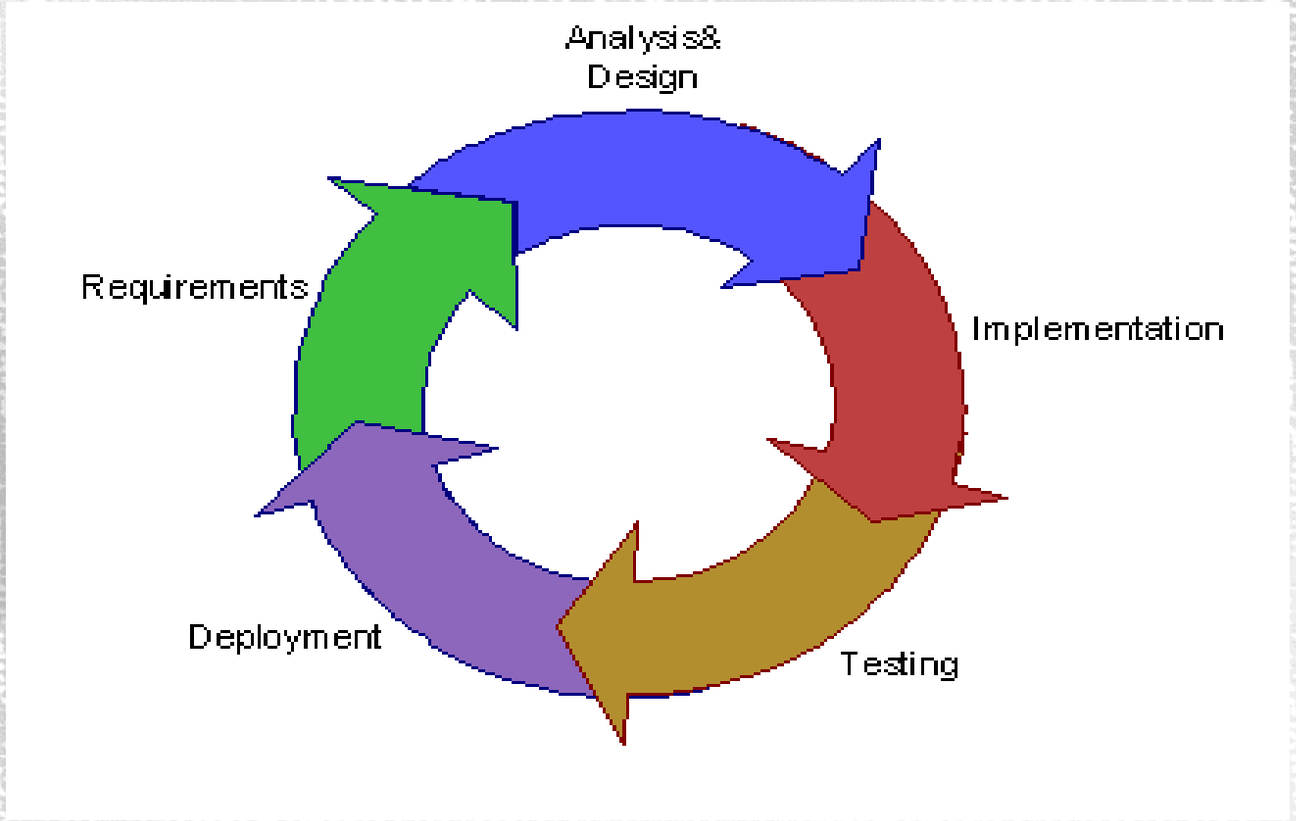

Концепция
Проектирование архитектуры
Детальное проектирование
Проверка и утверждение
Модульное тестирование и интеграция
Тестирование и проверка системы
Введение в эксплуатацию и поддержка
Реализация
Детализация проекта
Время
Тестирование и компоновка проекта

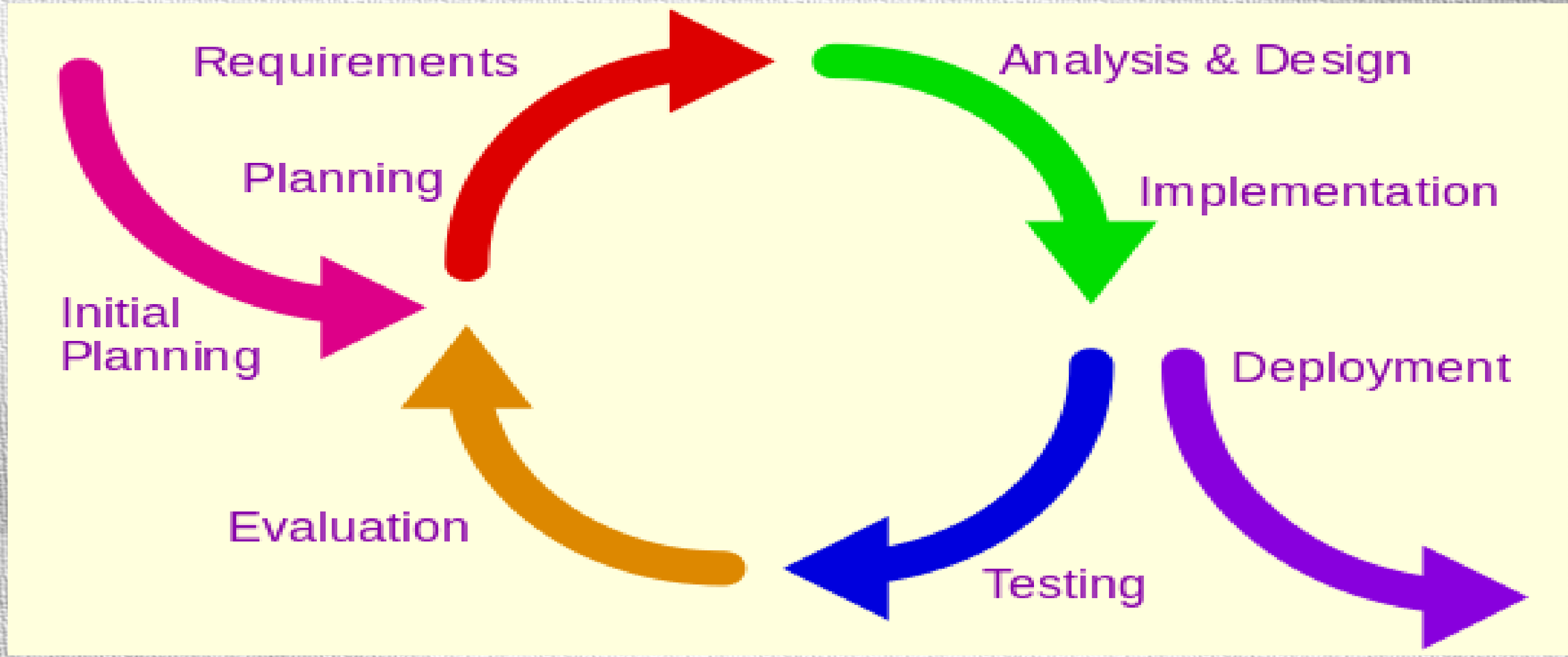
Итерационная модель
(iterative and incremental development, IID)

Модель IID предполагает разбиение жизненного цикла проекта на последовательность итераций, каждая из которых напоминает «мини-проект», включая все процессы разработки в применении к созданию меньших фрагментов функциональности, по сравнению с проектом в целом. Цель каждой *итерации* — получение работающей версии программной системы, включающей функциональность, определённую интегрированным содержанием всех предыдущих и текущей итерации. Результат финальной итерации содержит всю требуемую функциональность продукта. Таким образом, с завершением каждой итерации продукт получает приращение — *инкремент* — к его возможностям, которые, следовательно, развиваются *эволюционно*. Итеративность, инкрементальность и эволюционность в данном случае есть выражение одного и то же смысла разными словами со слегка разных точек зрения

Итеративная разработка

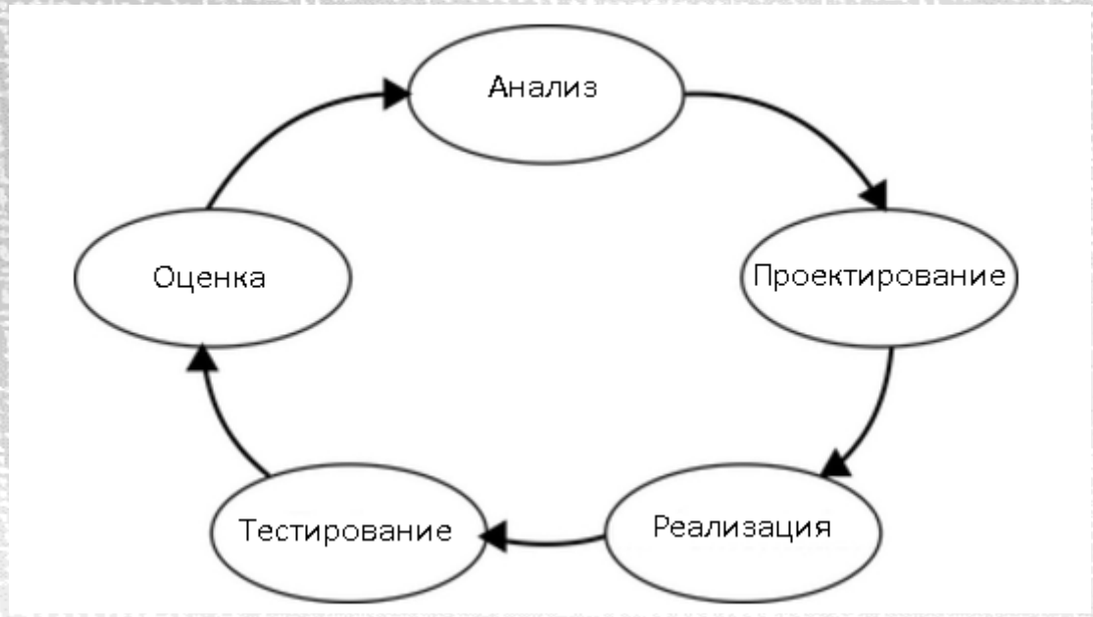
Процесс итеративной (или инкрементальной) разработки стал эволюционным развитием модели водопада. Инкрементная модель является классическим примером инкрементной стратегии разработки




Requirements
Analysis & Design
Planning
Implementation
Initial
Planning
Deployment
Evaluation
Testing

В результате очередной итерации продукт приобретает новую функциональность или улучшения в существующей функциональности. Полный набор требований, зафиксированный границами проекта, оказывается реализованным после завершения финальной итерации.

Итеративная модель используется во многих процессах разработки, включая RUP и гибкие методологии, описанные ниже.



Инкрементая модель



1 

2 

3 

Итеративная модель

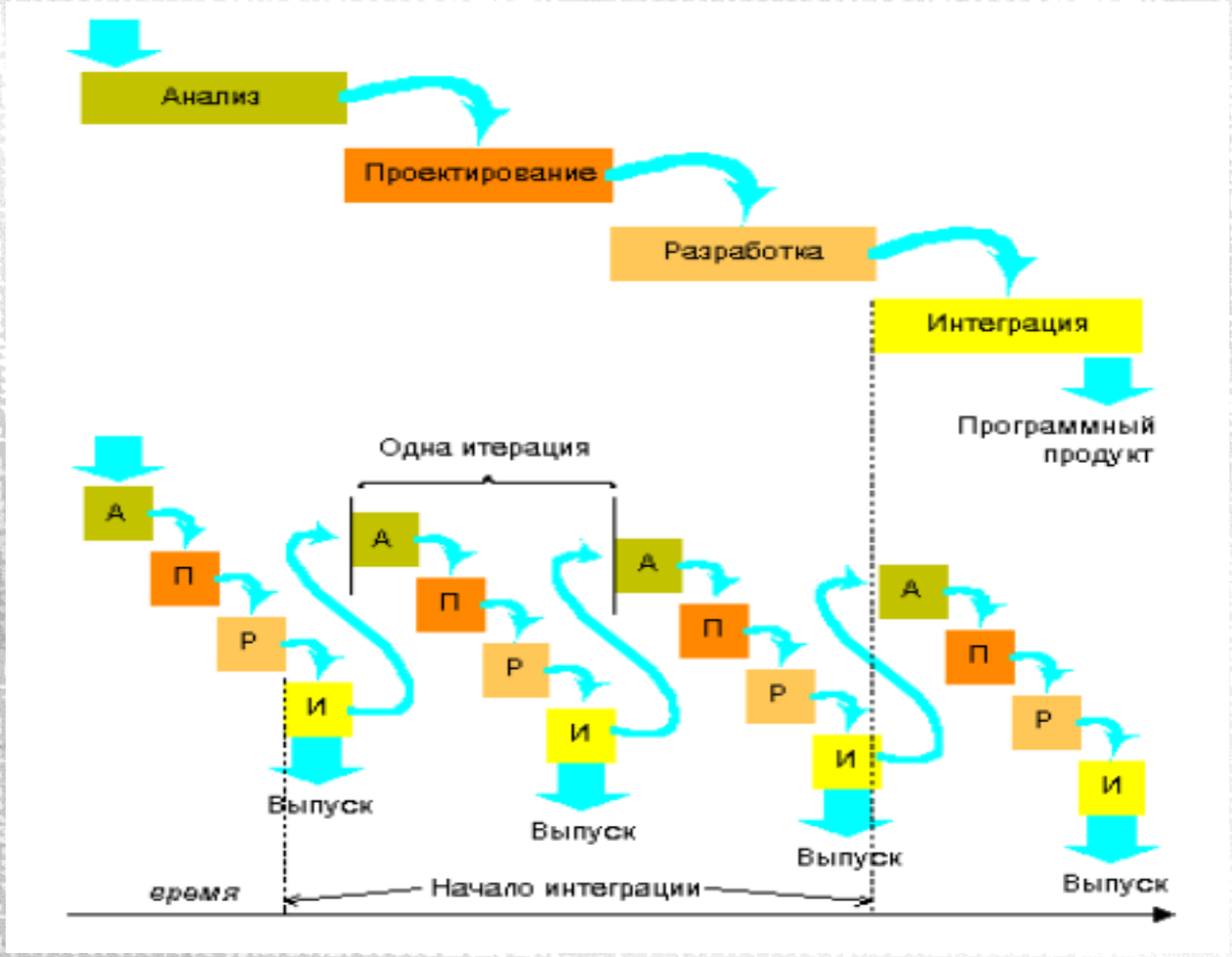


1 

2 

3 

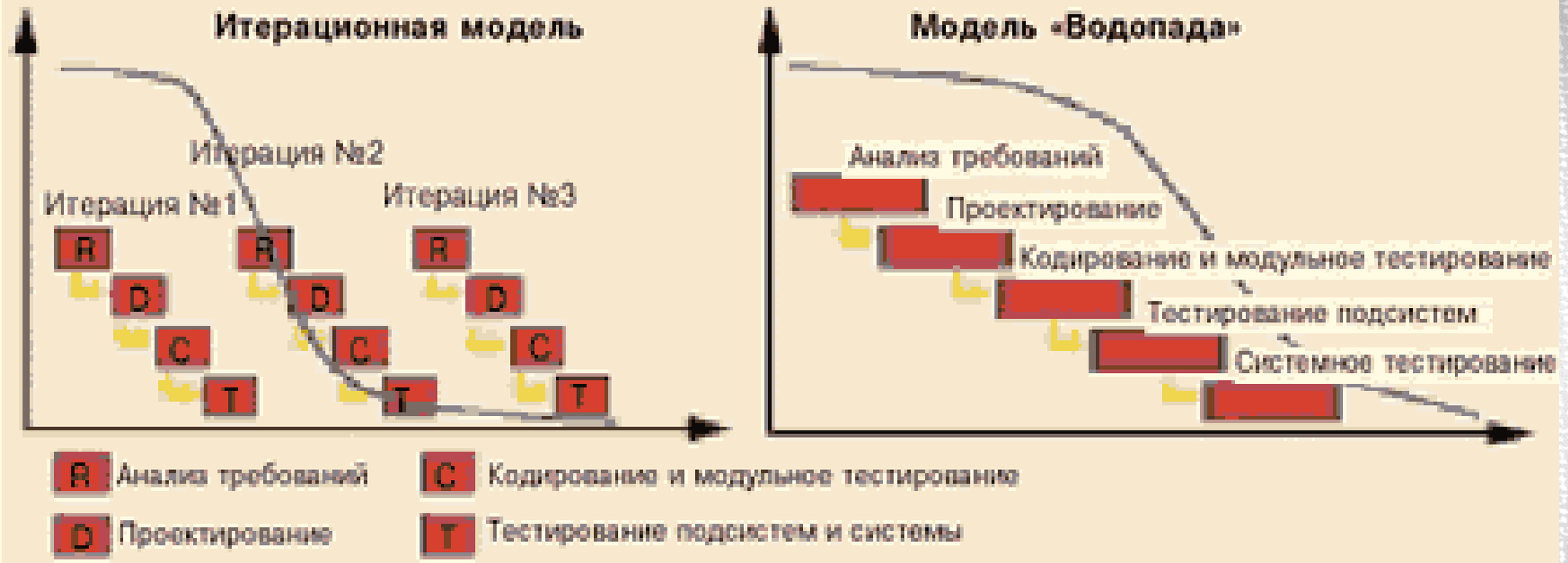
Сравнение стратегии «водопада» с итерационной стратегией





Различные варианты итерационного подхода реализованы в большинстве современных методологий разработки (RUP, MSF, XP).

2 Жизненные циклы разработки



Преимущества

Итеративная разработка обладает рядом преимуществ по сравнению с последовательной моделью.

- Реализация наиболее важных функций может быть завершена в ходе нескольких первых итераций. После их завершения (то есть намного раньше окончания всего проекта) заказчик сможет начать использование системы.
- Уже в начале проекта пользователи получают возможность оценить фунциональность системы и ее соответствие своим потребностям. Необходимые изменения и дополнения могут быть сделаны в течение следующих итераций.

- Основные проектные риски могут (и должны) быть разрешены на первых итерациях. Например, архитектурное решение, приводящее к неприемлемой производительности может быть обнаружено и исправлено уже в первой итерации.

Важно понимать, что все эти преимущества проявляются только при тщательном планировании итераций, в противном случае легко получить ухудшенный вариант модели водопада.

RUP

Один из самых известных процессов, использующих итеративную модель разработки – Rational Unified Process (RUP). Он был создан во второй половине 1990-х годов в компании Rational Software. Основными разработчиками были Филипп Крачтен (Philippe Kruchten), Грейди Буч (Grady Booch), Джеймс Рамбо (James Rumbaugh) и Айвар Якобсон (Ivar Jacobson). Кстати, последние трое являются также создателями нотации UML.

Термин RUP означает как методологию разработки, так и продукт компании IBM (ранее – Rational) для управления процессами разработки. Методология RUP описывает абстрактный общий процесс, на основе которого организация или проектная команда должна создать специализированный процесс, ориентированный на ее потребности.

Спиральная модель

Спиральная модель - классический пример применения эволюционной стратегии разработки.

Спиральная модель, предложенная Барри Боэмом в 1986 году, стала существенным прорывом в понимании природы разработки ПО. При использовании этой модели ПО создается в несколько итераций (витков спирали) методом прототипирования. Делается упор на начальные этапы жизненного цикла: **анализ и проектирование.**

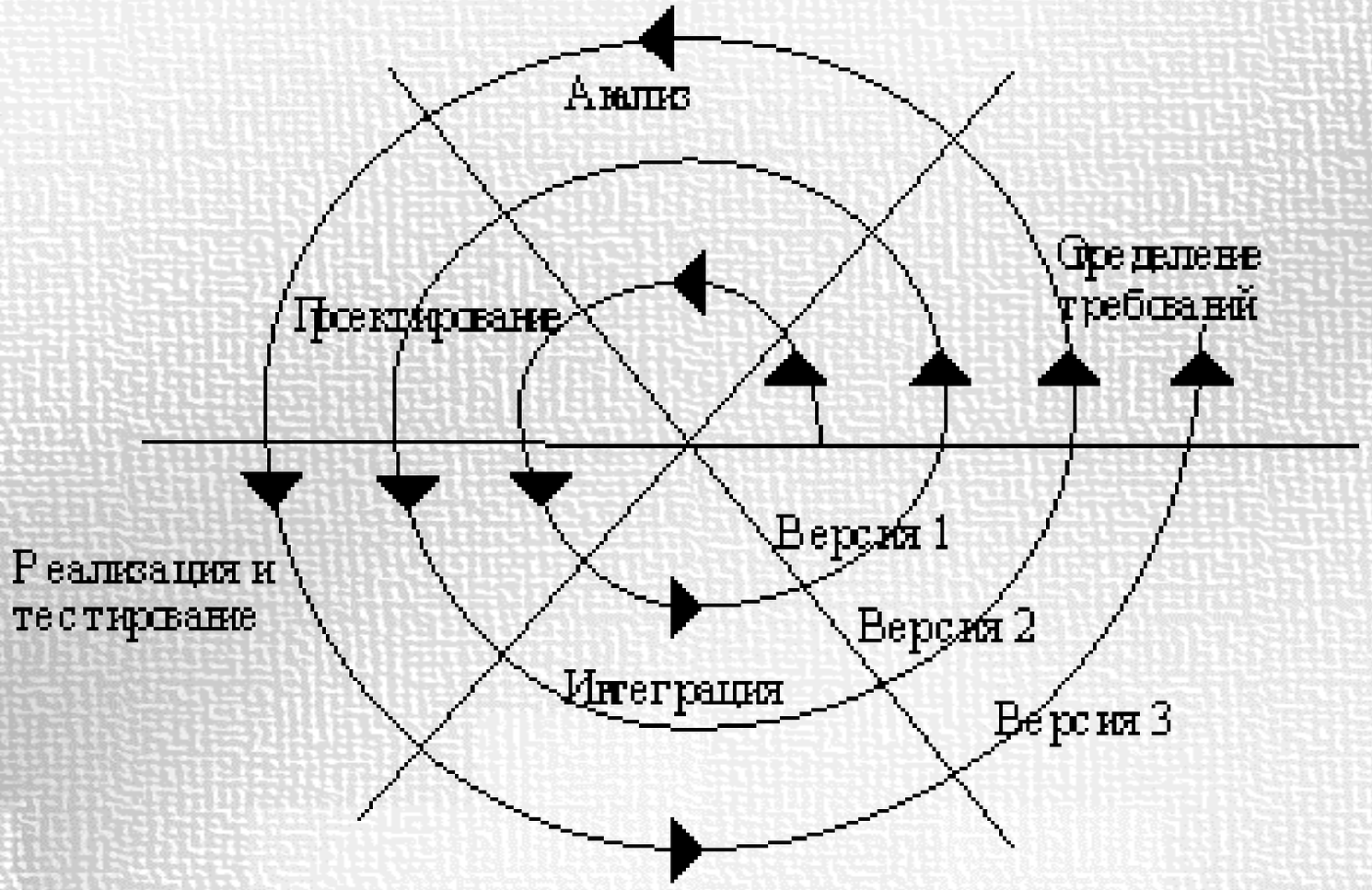
Каждый виток спирали соответствует созданию фрагмента или версии программного обеспечения, на нем уточняются цели и характеристики проекта, определяется его качество и планируются работы следующего витка спирали. Таким образом углубляются и последовательно конкретизируются детали проекта и в результате выбирается обоснованный вариант, который доводится до реализации.

На каждой итерации оцениваются:

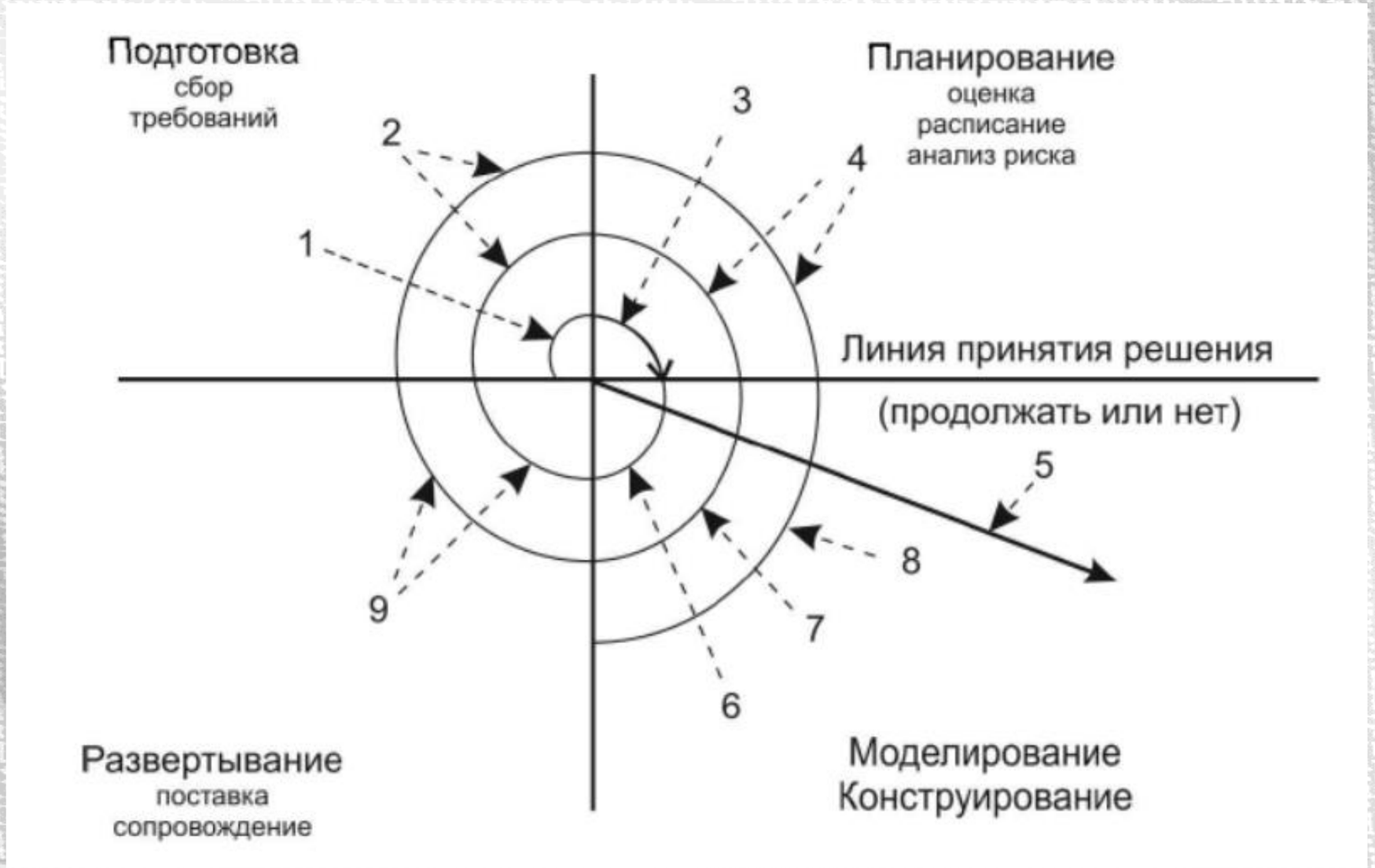
1. *риск превышения сроков и стоимости проекта;*
2. *необходимость выполнения ещё одной итерации;*
3. *степень полноты и точности понимания требований к системе;*
4. *целесообразность прекращения проекта.*

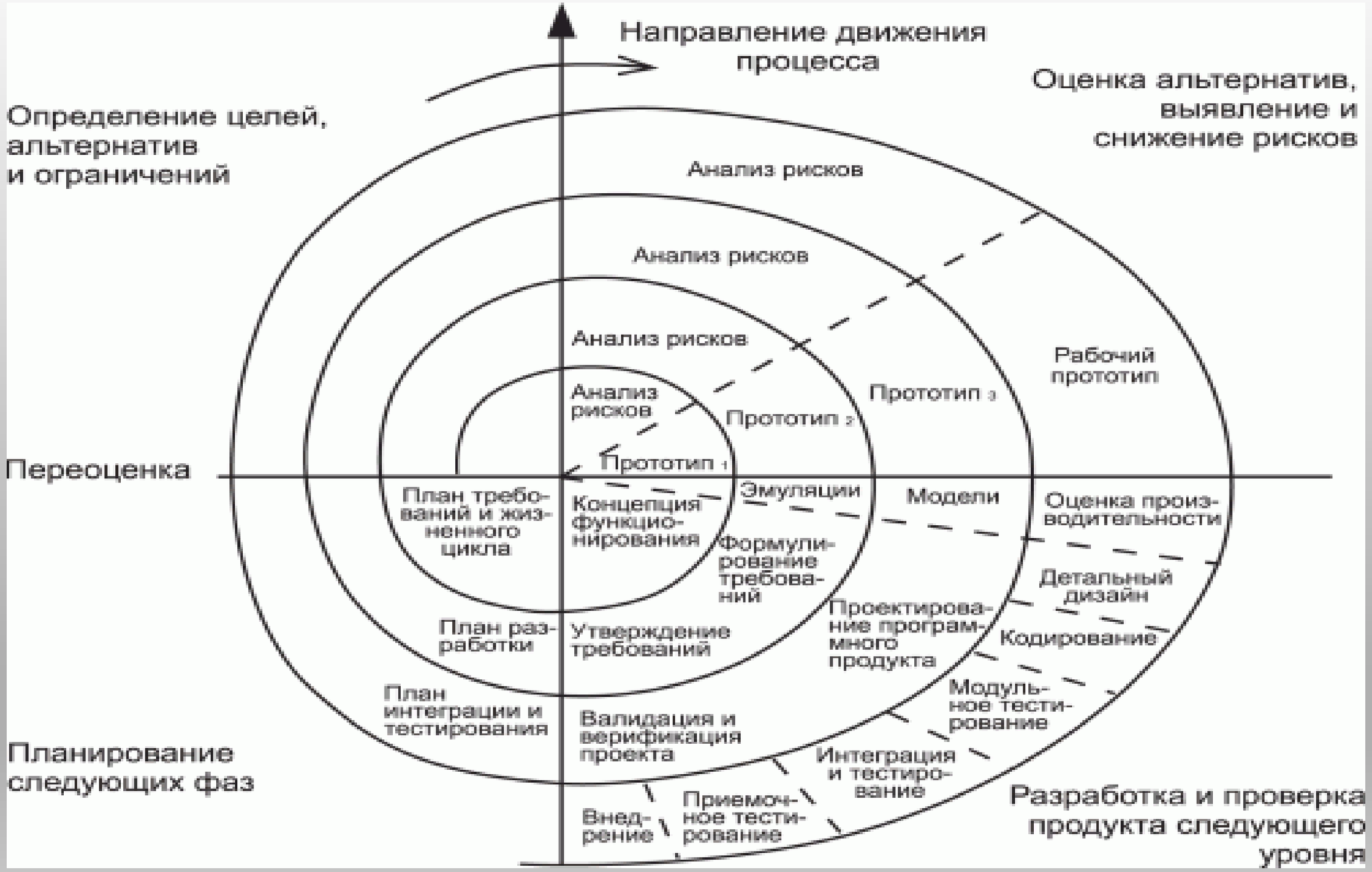
Важно понимать, что спиральная модель является не альтернативой эволюционной модели (модели IID), а специально проработанным вариантом.

Спиральная модель



Спиральная модель




Направление движения процесса
Оценка альтернатив, выявление и снижение рисков
Определение целей, альтернатив и ограничений
Анализ рисков
Анализ рисков
Анализ рисков
Анализ рисков
Рабочий прототип
Прототип 3
Прототип 2
Прототип 1
Переоценка
План требований и жизненного цикла
Концепция функционирования
Эмуляции
Модели
Оценка производительности
Формулирование требований
Детальный дизайн
План разработки
Утверждение требований
Проектирование программного продукта
Кодирование
План интеграции и тестирования
Валидация и верификация проекта
Модульное тестирование
Интеграция и тестирование
Внедрение
Приемочное тестирование
Планирование следующих фаз
Разработка и проверка продукта следующего уровня

Риски

Отличительной особенностью этой модели является специальное внимание рискам, влияющим на организацию жизненного цикла. Боэм формулирует десять наиболее распространённых (по приоритетам) рисков:

1. Дефицит специалистов.
2. Нереалистичные сроки и бюджет.
3. Реализация несоответствующей функциональности.
4. Разработка неправильного пользовательского интерфейса.
5. «Золотая сервировка», перфекционизм, ненужная оптимизация и оттачивание деталей.
6. Непрекращающийся поток изменений.
7. Нехватка информации о внешних компонентах, определяющих окружение системы или вовлечённых в интеграцию.
8. Недостатки в работах, выполняемых внешними (по отношению к проекту) ресурсами.
9. Недостаточная производительность получаемой системы.
10. «Разрыв» в квалификации специалистов разных областей знаний.

Контрольные точки

В сегодняшней спиральной модели определён следующий общий набор контрольных точек:

- **Concept of Operations (COO)** — концепция (использования) системы;
- **Life Cycle Objectives (LCO)** — цели и содержание жизненного цикла;
- **Life Cycle Architecture (LCA)** — архитектура жизненного цикла; здесь же возможно говорить о готовности концептуальной архитектуры целевой программной системы;
- **Initial Operational Capability (IOC)** — первая версия создаваемого продукта, пригодная для опытной эксплуатации;
- **Final Operational Capability (FOC)** —— готовый продукт, развернутый (установленный и настроенный) для реальной эксплуатации.

На каждом витке спирали могут применяться разные модели процесса разработки ПО. В конечном итоге на выходе получается готовый продукт. Модель сочетает в себе возможности модели прототипирования и водопадной модели. Разработка итерациями отражает объективно существующий спиральный цикл создания системы. Неполное завершение работ на каждом этапе позволяет переходить на следующий этап, не дожидаясь полного завершения работы на текущем. При итеративном способе разработки недостающую работу можно будет выполнить на следующей итерации. Главная задача — как можно быстрее показать пользователям системы работоспособный продукт, тем самым активизируя процесс уточнения и дополнения требований. Основная проблема спирального цикла — определение момента перехода на следующий этап. Для ее решения необходимо ввести временные ограничения на каждый из этапов жизненного цикла. Переход осуществляется в соответствии с планом, даже если не вся запланированная работа закончена. План составляется на основе статистических данных, полученных в предыдущих проектах, и личного опыта разработчиков.

Что в спирали хорошего?

Мы резко снижаем риски выкатить заказчику вместо продукта откровенную пустышку, но при этом не приносим в жертву проектирование. То есть, глобальное видение ситуации сохраняется, конечным результатом можно управлять, рассчитывая бюджет и прибыль. Всем, клиенту и подрядчику, виден свет в конце тоннеля. Заказчик, если он серьезен в намерениях, тоже не будет бухтеть: "Что вы мне то хвост, то хобот подсовываете, покажите мне всего слона!" На каждом витке слон виден целиком.

Однако, по сравнению с восходящим софтостроением, обойтись взводом программистов-техников без аналитиков и проектировщиков не получается, и вам придется объяснять этот щекотливый момент заказчику в обосновании более высокой цены.

После стадии проектирования первого витка спецы не простаивают, а продолжают вгрызаться в задачу, ожидая фидбэк, но на финальном витке они, скорее всего, вообще не понадобятся. Длина витка может составлять до нескольких месяцев, но обычно обходится двумя-тремя.

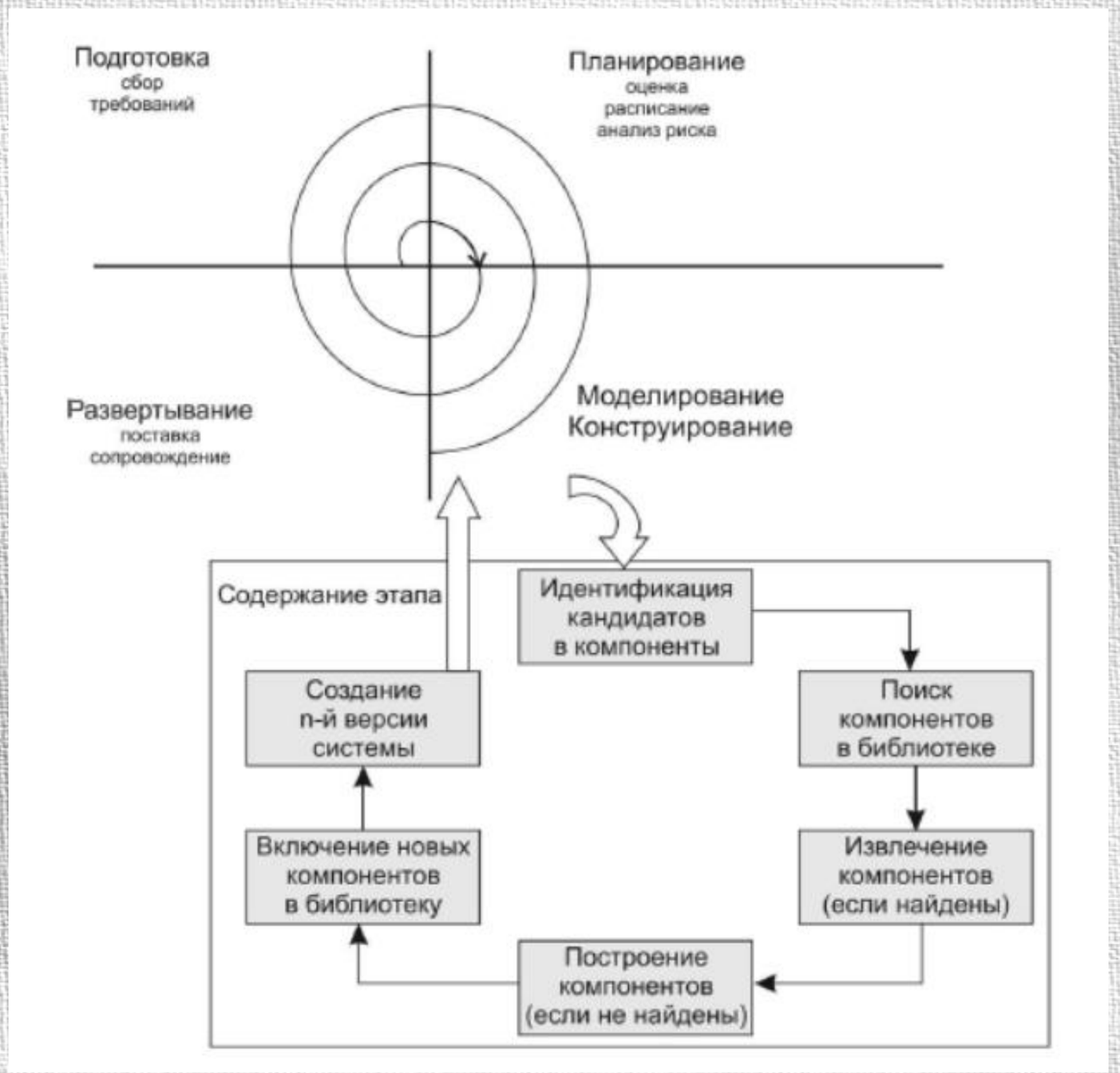
На еженедельный спринтовый экстрим не похоже, но и на годовое ожидание продукта тоже. Нелегким остается и выбор, когда на данном витке "уже хватит проектировать" и пора начинать класть кирпичи.

Наиболее известная и успешная реализация спиральной методики -- MSF (Microsoft Solution Framework). У Микрософта позднее появилась и обстриженная версия, заточенная под восходящие методы, и продаваемая как MSF for Agile для относительно небольших проектов и команд.

Компонентно-ориентированная модель

Компонентно-ориентированная модель является развитием спиральный модели и тоже основывается на эволюционной стратегии разработки. В этой модели модифицируется содержание квадранта моделирования-конструирования - оно отражает тот факт, что в современных условиях новая разработка должна основываться на повторном использовании существующих программных компонентов.

Программный компоненты, созданные в реализованных программных проектах, хранятся в библиотеке. В новом программном проекте, исходя из требований заказчика, появляется кандидаты в компоненты, далее проверяется наличие этих кандидатов в библиотеке. Если они найдены, то компоненты извлекаются из библиотеки и используется повторно. Противном случае создаются новые компоненты, они применяются в проекте и включаются в библиотеку.


Подготовка
сбор
требований
Планирование
оценка
расписание
анализ риска
Развертывание
поставка
сопровождение
Моделирование
Конструирование
Содержание этапа
Идентификация
кандидатов
в компоненты
Поиск
компонентов
в библиотеке
Извлечение
компонентов
(если найдены)
Построение
компонентов
(если не найдены)
Включение новых
компонентов
в библиотеку
Создание
n-й версии
системы

Критика каскадной модели и гибридные методологические решения

Методику «Каскадная модель» довольно часто критикуют за недостаточную гибкость и объявление самоцелью формальное управление проектом в ущерб срокам, стоимости и качеству. Тем не менее, при управлении большими проектами формализация часто являлась очень большой ценностью, так как могла кардинально снизить многие риски проекта и сделать его более прозрачным. Поэтому даже в PMBOK 3-ей версии формально была закреплена только методика «каскадной модели» и не были предложены альтернативные варианты, известные как итеративное ведение проектов.

Начиная с PMBOK 4-й версии удалось достичь компромисса между методологами, приверженными формальному и поступательному управлению проектом, с методологами, делающими ставку на гибкие итеративные методы. Таким образом, начиная с 2009 года, формально Институтом управления проектами (PMI) предлагается как стандарт гибридный вариант методологии управления проектами, сочетающий в себе как плюсы от методики «Водопада», так и достижения итеративных методологов.

Модель хаоса

- Стратегия хаоса — это стратегия разработки программного обеспечения, основанная на модели хаоса. Главное правило — это всегда решать наиболее важную задачу первой.
- Стратегия хаоса похожа на путь, по которому программисты работают в самом конце проекта, когда у них есть список ошибок для исправления и возможность для творчества. Обычно, кто-то расставляет приоритет оставшимся частным задачам и программисты устраняют их по одной. Стратегия хаоса утверждает, что это — единственный корректный путь выполнения работы.
- Стратегия хаоса навеяна стратегией игры Го.

ВЫВОДЫ

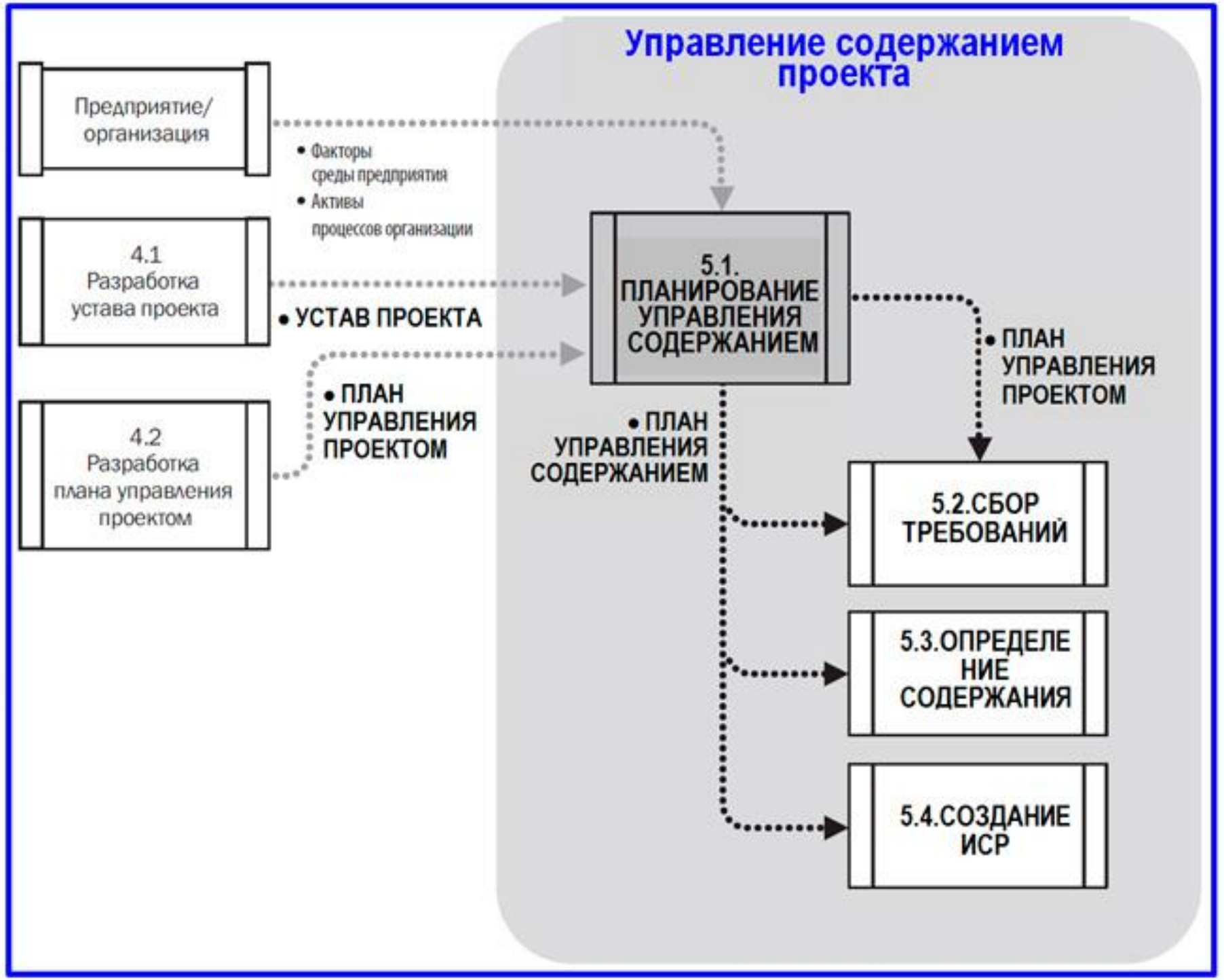
В современной практике модели разработки программного обеспечения многовариантны. Нет единственно верной для всех проектов, стартовых условий и моделей оплаты.

Иерархическая структура работ проекта

- **WBS (Work Breakdown Structure)**
- **Структура разбиения работ (СРР)**

Мы будем пользоваться аббревиатурой ИСР




Управление содержанием проекта
Предприятие/ организация
• Факторы среды предприятия
• Активы процессов организации
4.1 Разработка устава проекта
• УСТАВ ПРОЕКТА
4.2 Разработка плана управления проектом
• ПЛАН УПРАВЛЕНИЯ ПРОЕКТОМ
5.1. ПЛАНИРОВАНИЕ УПРАВЛЕНИЯ СОДЕРЖАНИЕМ
• ПЛАН УПРАВЛЕНИЯ ПРОЕКТОМ
• ПЛАН УПРАВЛЕНИЯ СОДЕРЖАНИЕМ
5.2.СБОР ТРЕБОВАНИЙ
5.3.ОПРЕДЕЛЕНИЕ СОДЕРЖАНИЯ
5.4.СОЗДАНИЕ ИСР

ПРОЕКТ

Проект – выполнение определенных работ для определенного заказчика в рамках определенного бюджета в определенные сроки.

Функциональные специализации (роли)

Уровень проектного решения	А	Б	Англоязычный термин
1	Руководитель проекта, научный руководитель	Руководитель проекта	Project Manager
2	ГИП (главный инженер проекта), главный конструктор (*)	Системный архитектор	System Architect
3	Ведущий инженер (область, специализация)	Архитектор подсистемы или направления (**)	Software architect, database architect, hardware architect
4	Инженер (область, специализация)	Разработчик (специализация)	Software engineer, database engineer, system engineer
5	Техник (область, специализация), оператор ЭВМ	Младший специалист, кодировщик	Software developer, database developer, web developer, support & helpdesk

А - организации времён позднего СССР; **Б –** современные организации

ПРОДУКТ

Продукт – результат выполнения работ по одному или нескольким проектам, который можно продавать как отдельное решение как существующим, так и новым заказчикам.

КОМАНДА

Команда – группа сотрудников компании, которая выполняет определенные виды работ и планирует свою работу итерациями одинаковой длительности.

Иерархическая структура работ проекта

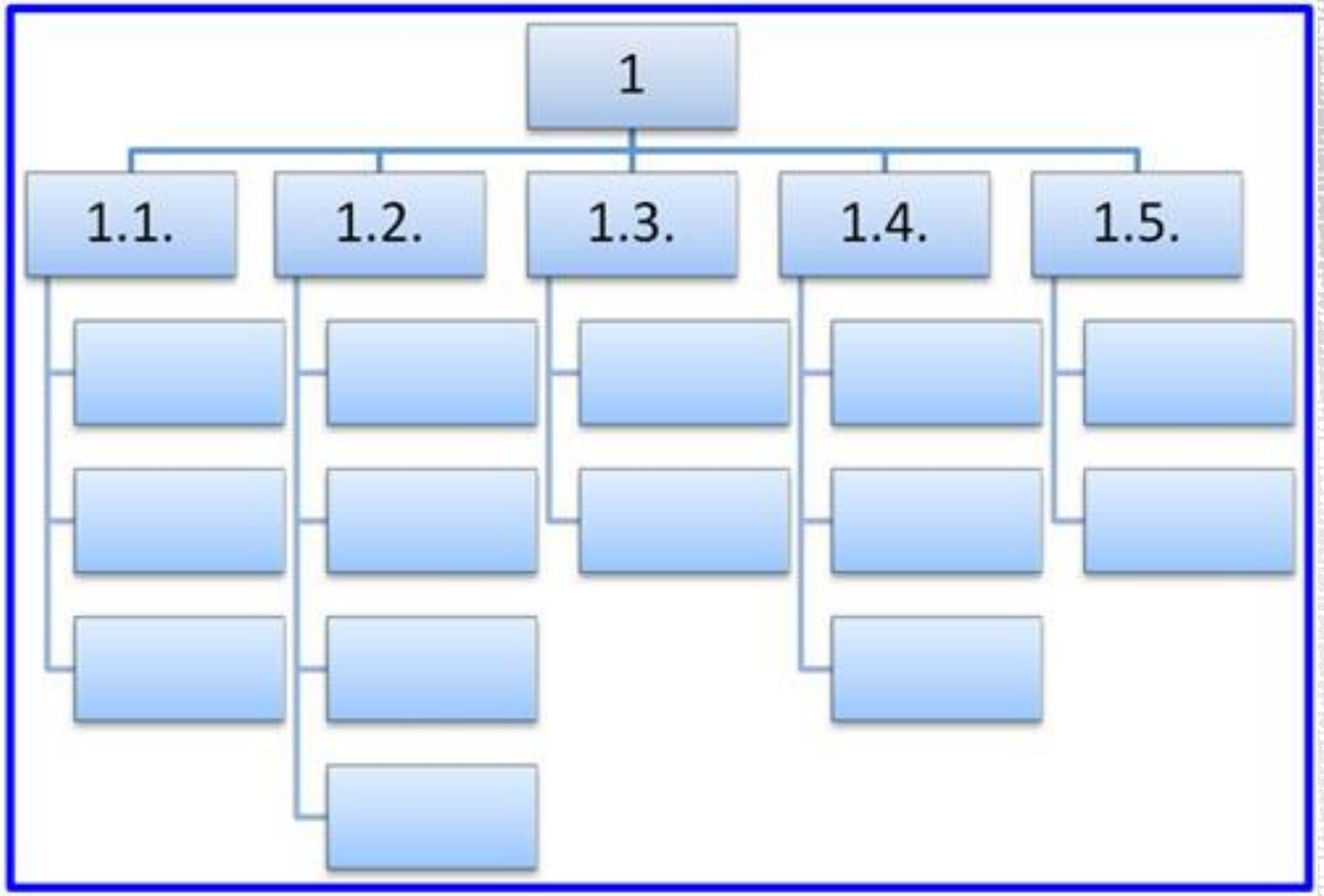
Иерархическая структура работ (ИСР) — это иерархическое разбиение всей работы, которую необходимо выполнить для достижения целей проекта, на более мелкие операции и действия до такого уровня, на котором способы выполнения этих действий вполне ясны и соответствующие работы могут быть оценены и спланированы. Она включает также определение промежуточных результатов всех составляющих эту структуру работ.

ИСР обеспечивает выявление всех работ, необходимых для достижения целей проекта. Многие проекты проваливаются не оттого, что у них нет плана, а оттого, что в этом плане забыты такие важные работы, как тестирование и исправление ошибок, а также продукты проекта, например пользовательская документация. Поэтому если ИСР составлена корректно, то любая работа, которая в нее не вошла, не может считаться работой по проекту. ИСР делит проект на субпроекты, пакеты работ, субпакеты. Каждый следующий уровень декомпозиции обеспечивает последовательную детализацию содержания проекта, что позволяет производить оценку сроков и объемов работ. ИСР должна включать все промежуточные и конечные продукты.

Иерархическая структура работ имеет следующие характеристики:

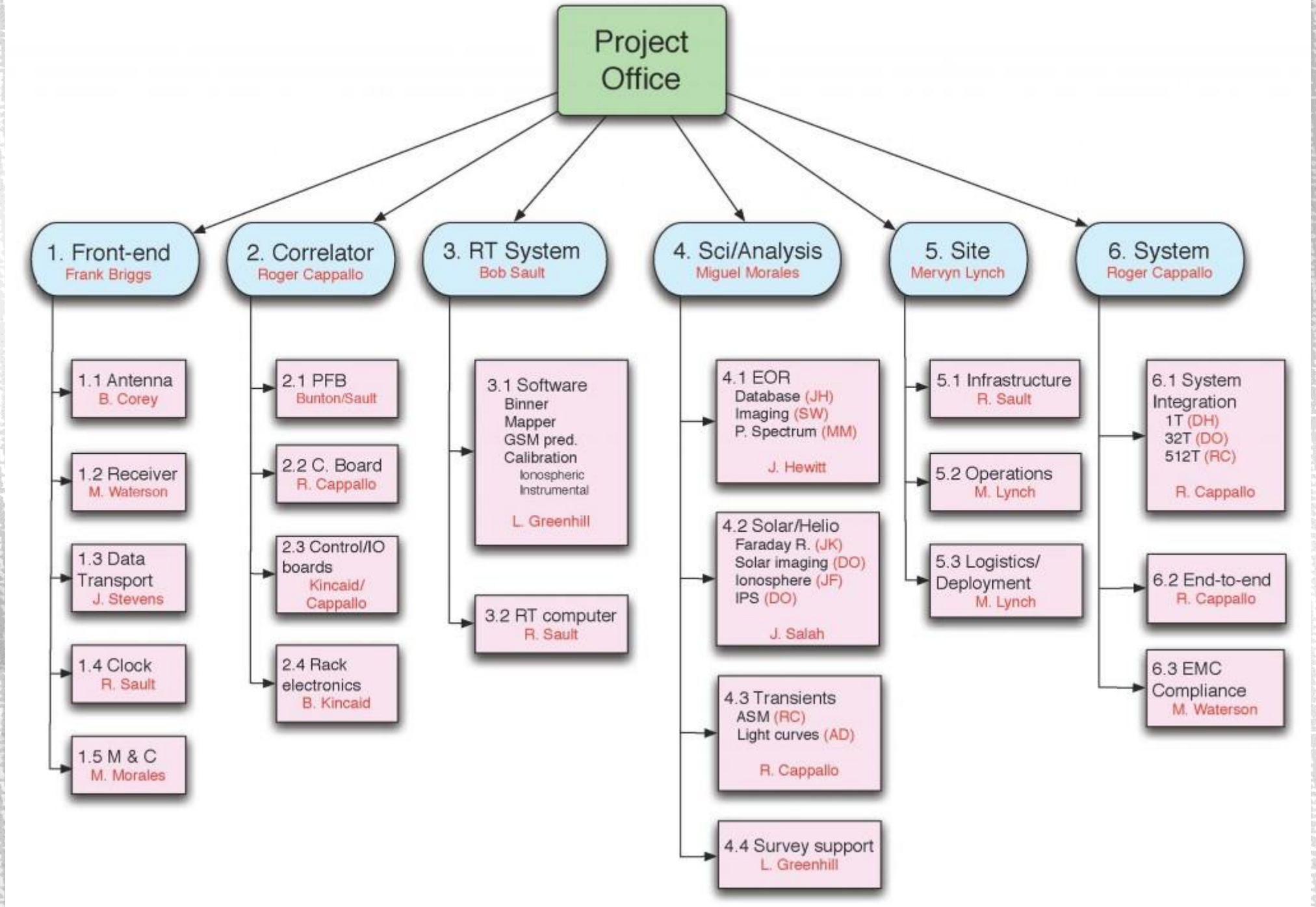
- Описывает с необходимой точностью содержание работ по проекту.
- Определяет весь объем работ по проекту.
- Формируется в виде иерархической структуры (проект декомпозируется на пакеты/субпакеты и т. д. работ).
- Представляет объем работ по пакету как перечень работ, имеющих измеримый или сравнимый результат.
- Имеет объективный или измеримый результат, который рассматривается как результат работы по пакету или совокупность результатов работ.

Визуальная форма иерархической структуры работ проекта



- Все элементы ИСР имеют специальную кодировку, смысл которой — присвоить каждому элементу уникальный номер.
- Самый верхний уровень имеет код 0 (ноль) и его часто именуют просто: «проект». Элементы первого уровня нумеруются последовательно от 1 до количества элементов на уровне (обычно не более 7). Второй и последующие уровни нумеруется таким образом, чтобы элемент сохранил ссылку на вышестоящий, например ″1.2.4″.

Чтобы лучше понять о чем идет речь, посмотрите на пример диаграммы.


Project Office
1. Front-end
Frank Briggs
2. Correlator
Roger Cappallo
3. RT System
Bob Sault
4. Sci/Analysis
Miguel Morales
5. Site
Mervyn Lynch
6. System
Roger Cappallo
1.1 Antenna
B. Corey
1.2 Receiver
M. Waterson
1.3 Data Transport
J. Stevens
1.4 Clock
R. Sault
1.5 M & C
M. Morales
2.1 PFB
Bunton/Sault
2.2 C. Board
R. Cappallo
2.3 Control/IO boards
Kincaid/ Cappallo
2.4 Rack electronics
B. Kincaid
3.1 Software
Binner
Mapper
GSM pred.
Calibration
Ionospheric
Instrumental
L. Greenhill
3.2 RT computer
R. Sault
4.1 EOR
Database (JH)
Imaging (SW)
P. Spectrum (MM)
J. Hewitt
4.2 Solar/Helio
Faraday R. (JK)
Solar imaging (DO)
Ionosphere (JF)
IPS (DO)
J. Salah
4.3 Transients
ASM (RC)
Light curves (AD)
R. Cappallo
4.4 Survey support
L. Greenhill
5.1 Infrastructure
R. Sault
5.2 Operations
M. Lynch
5.3 Logistics/ Deployment
M. Lynch
6.1 System Integration
1T (DH)
32T (DO)
512T (RC)
R. Cappallo
6.2 End-to-end
R. Cappallo
6.3 EMC Compliance
M. Waterson

- Важно понимать, что в ИСР должны входить абсолютно все результаты и работы проекта, даже те, которые будут выполняться сторонними организациями (подрядчиками).
- Также, где это актуально, должны быть включены административные работы (поддержание офиса, закупка канцтоваров, обслуживание техники и пр.).

В некоторых случаях иерархическая структура работ содержит специальное дополнение, т.н. «словарь ИСР«. Это чаще таблица, которая помогает правильно прочитать вашу диаграмму:

1. код элемента
2. название элемента
3. содержание элемента (подробное описание назначения, состава, структуры, подэлементов и т.п. Главное — однозначно определить что(!) это за результат и как его отличить от других)
4. кто отвечает за его получение (фамилия, имя и домашний адрес). Этот пункт сильно активизирует исполнителей!
5. продолжительность, оценка объемов и т.д.

ДВА ОСНОВНЫХ ПРИНЦИПА ПОСТРОЕНИЯ ИСР:

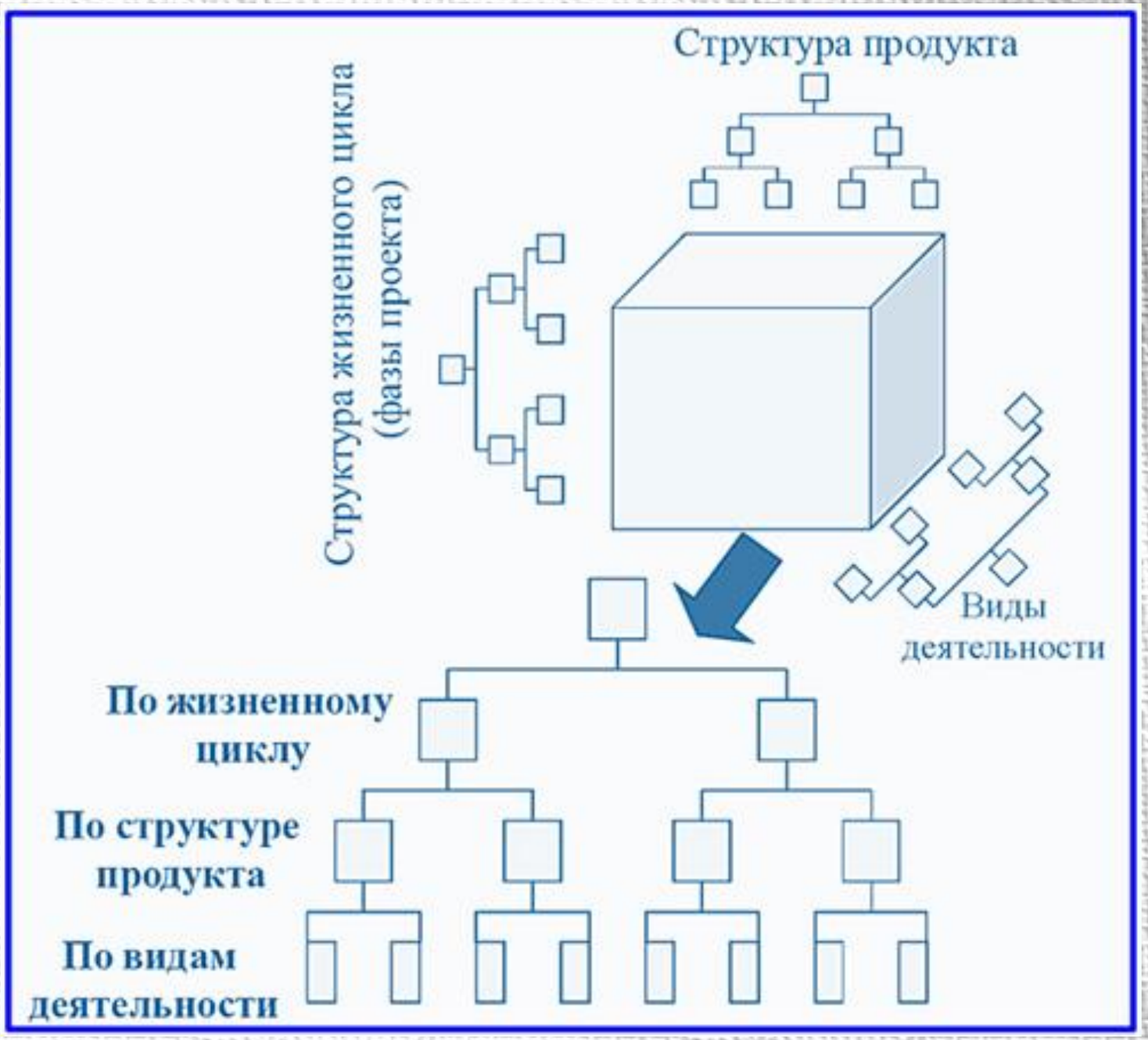
1. **Правило 100%.** Оно говорит о том, что в ИСР должны быть включены все результаты, продукты и работы, которые должны быть выполнены по проекту, вне зависимости от того это внешние, внутренние результаты или переданные для выполнения 3-м лицам. Данное правило также применяется ко всем нижестоящим компонентам.
2. **Взаимоисключаемость элементов.** Данный принцип говорит о том, что в ИСР не может быть 2 элементов с одинаковым содержанием. Т.е. не может быть 2-х одинаковых элементов в разных ветках ИСР. Если это допустить, вас ожидает дублирование работы и непонимание командой проекта своих работ.

ДЕКОМПОЗИЦИЯ

Существует несколько подходов к разбиению результатов проекта на элементы. Основными являются два из них: **продуктовый** и **функциональный**. Данные критерии совмещать на одном уровне нельзя, а на разных уровнях иерархии допустимо и даже целесообразно. Продуктовый подход необходим PM для общения с заказчиком. Заказчику важно видеть, что и когда менеджер будет ему сдавать, все ли продукты проекта будут ему представлены и т.п. Функциональный подход удобен менеджеру для взаимодействия с исполнителями. По нему проще назначить ответственных за отдельные блоки операций.

Иерархическая структура работ, формируемая под руководством PM, на верхних уровнях иерархии может использовать любой критерий, исходя из логики декомпозиции. Часто, если проект выполняется в несколько этапов, может использоваться подход, исходя из кубической модели структур проекта (смотреть схему, размещенную выше). На верхнем уровне мы разбиваем результаты по этапам жизненного цикла проекта, затем структурируем по производимым проектным продуктам, и, наконец, пакеты работ комплектуем по функциональному критерию или по видам деятельности. Ниже вашему вниманию представлен пример продуктовой ИСР.

Кубическая модель структур проекта, преобразуемая в структуру работ



Пример продуктовой иерархической структуры



Пример функциональной иерархической структуры



Принципы декомпозиции

1. По фазам жизненного цикла

Например, ваш проект выполняется по таким фазам: Продажа, Аналитика, Проектирование, Дизайн, Продакшн, Верстка и т.д. Вы можете представить все эти объекты на первом уровне, чтобы дальше разбить их на более измеримые кусочки.

Это удобно, когда у руководителя проекта и заказчика нет разногласий по набору фаз проекта, а также каждая фаза имеет понятый всем участникам набор результатов.

2. По ключевым результатам проекта (deliverables)

- Например, проект по внедрению системы управления транспортом может иметь такой набор результатов: Информационная система, Карты для маршрутизации транспорта, Векторные графы дорог и маршрутов, Механизм поддержки, Обученный персонал, Оборудование.
- Эти результаты удобно отразить на первом уровне, чтобы заказчик ясно и точно видел, что будет сдано ему по завершению проекта. Здесь важно помнить, что если результат не представлен на ИСР, то в проекте он не будет получен.

3. По организационной структуре проекта

Например, в вашем проекте есть 4 структуры, которые вовлечены в реализацию проекта:

- бизнес-заказчик. Он может отвечать за результаты:
  - бизнес требования к системе
  - подготовленные к загрузке в систему данные
  - обученные пользователи
  - бизнес-тестирование и т.д.
- ИТ со стороны заказчика. Они могут отвечать за:
  - серверы
  - закупленные лицензии
  - интерфейс обмена с другими учетными системами и т.д.

- вендор (поставщик программного продукта) отвечает за
  - аудит технической архитектуры
  - компания — внедренец программного обеспечения (ваша структура):
  - техническое задание
  - настройка системы
  - доп.программирование системы и т.п.
- компания — внедренец программного обеспечения (ваша структура):
  - техническое задание
  - настройка системы
  - доп.программирование системы и т.п.

4. По подпроектам



Иногда можно использовать смешанный подход:

1. на первом уровне фазы проекта,
2. на втором организационные структуры,
3. на третьем — результаты.

Главное — не запутайтесь. Важно, чтобы выбранный подход к декомпозиции позволил оценить весь объем проекта.

Основные ориентиры (условия) при создании структуры работ следующие.

1. Для каждого элемента структуры формулируется измеримый результат.
2. Каждый результат вышестоящего элемента носит агрегированный характер, т.е. является итогом результатов «дочерних» элементов декомпозиции.
3. Пакеты и отдельные операции должны быть уникальными.
4. Структура должна быть полной, но не избыточной.

5. Элементы структуры верхних уровней должны быть совместимы с организационной структурой проекта.
6. Размер элементов нижнего уровня должен быть достаточным для эффективного управления, но не избыточным для их контроля.

Этапы разработки иерархической структуры работ

Этап 1. Определение конечных результатов проекта	Рекомендуется рассмотреть и проанализировать документы, описывающие общий объем работ по проекту: концепцию, Устав, бизнес-план (если он есть)
Этап 2. Выбор критериев разбиения на элементы	Среди продуктового, функционального и организационного критериев разбиения нужно выбрать последовательность декомпозиции результатов на элементы
Этап 3. Определение основных пакетов работ	Основой для этого могут служить результаты, необходимые для создания продукта проекта, но не являющиеся его целями
Этап 4. Определение степени детализации работ	Степень детализации должна соответствовать внутренней системе управления и единой системе контроля. Контроль не может стать самоцелью.
Этап 5. Анализ и усовершенствование	Продолжаются до тех пор, пока все участники проекта не будут удовлетворены результатами предварительного планирования

Современный процесс разработки коммерческого ПО должен быть инкрементальным. Это означает, что на верхнем уровне декомпозиции проекта должны находиться продукты проекта, а на следующем уровне — компоненты, из которых эти продукты состоят. Компоненты далее могут быть декомпозированы на «фичи» — функции, которые они должны реализовывать.

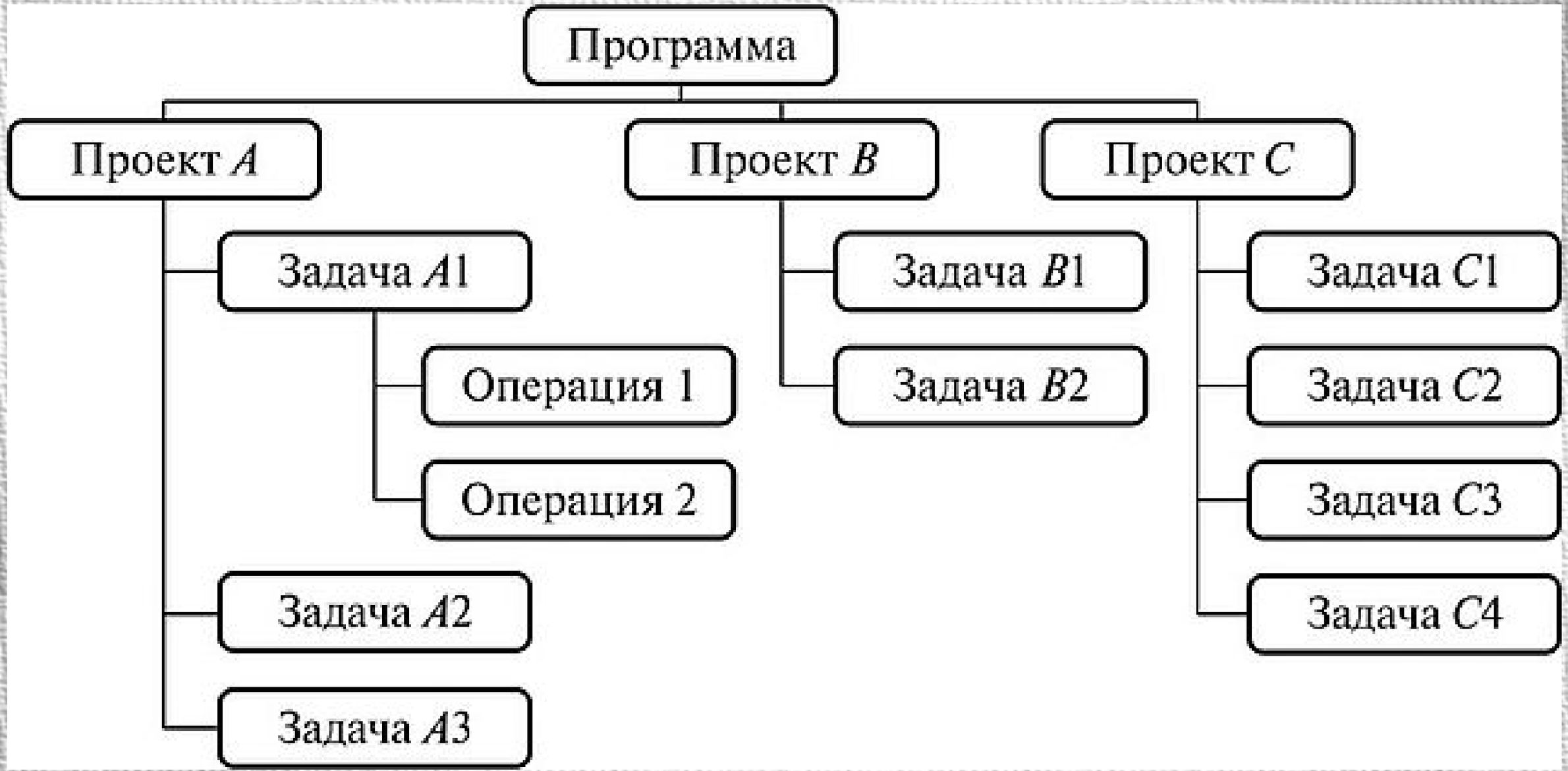
Выделение компонентов, составляющих программный продукт, — это элемент высокоуровневого проектирования, которое должно выполняться на фазе планирования проекта, не дожидаясь проработки всех функциональных требований к разрабатываемому ПО.

Компонентами могут быть как прикладные подсистемы, так и инфраструктурные или ядерные, например подсистема аутентификации, безопасности, библиотека визуальных компонентов графического интерфейса (GUI). При составлении базового плана работ не стоит стремиться максимально детализировать все работы ИСР, достаточно 3—5 уровней.

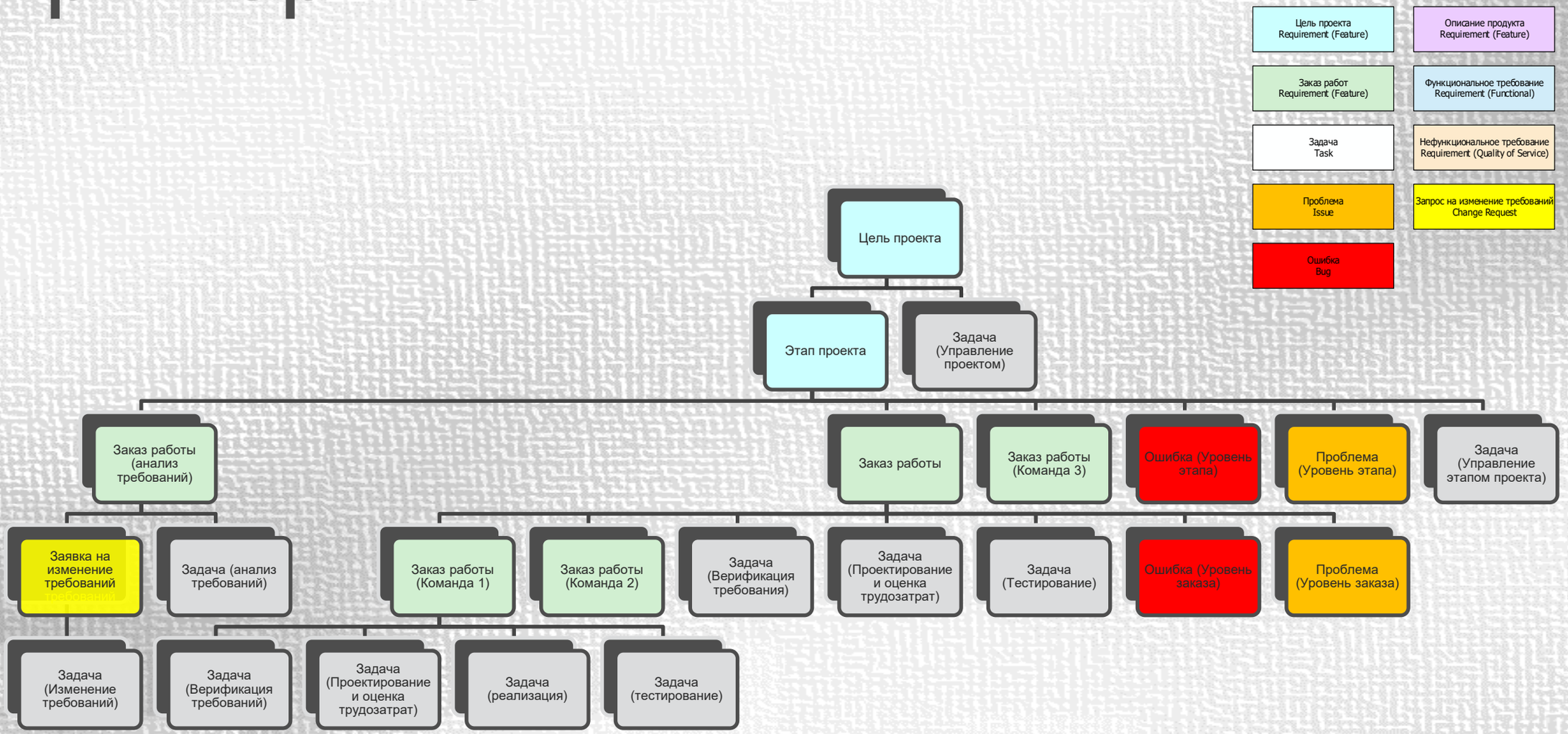
В контексте детализации часто используются термины «задачи» и «работы» как взаимозаменяемые. Все же корректнее говорить, что задача определяет достижение промежуточного результата, а работы являются комплексами действий для достижения этих результатов.

Так как любая задача требует проведения определенных работ при заданных ограничениях, безусловно, можно говорить о несомненном «отображении» (mapping) задач на работы и наоборот. Это и есть причина взаимозаменяемости терминов в повседневной жизни. В то же время понимание отличий между этими понятиями позволяет почувствовать нюансы процессного взгляда на создание продукта, а следовательно, и на жизненный цикл проектов, в том числе и проектов программных систем.

В общем случае можно говорить о детализации: «программа — проект — задача — операция». На рисунке представлен пример такой детализации (показаны только операции Задачи А).


Программа
Проект A
Проект B
Проект C
Задача A1
Операция 1
Операция 2
Задача A2
Задача A3
Задача B1
Задача B2
Задача C1
Задача C2
Задача C3
Задача C4

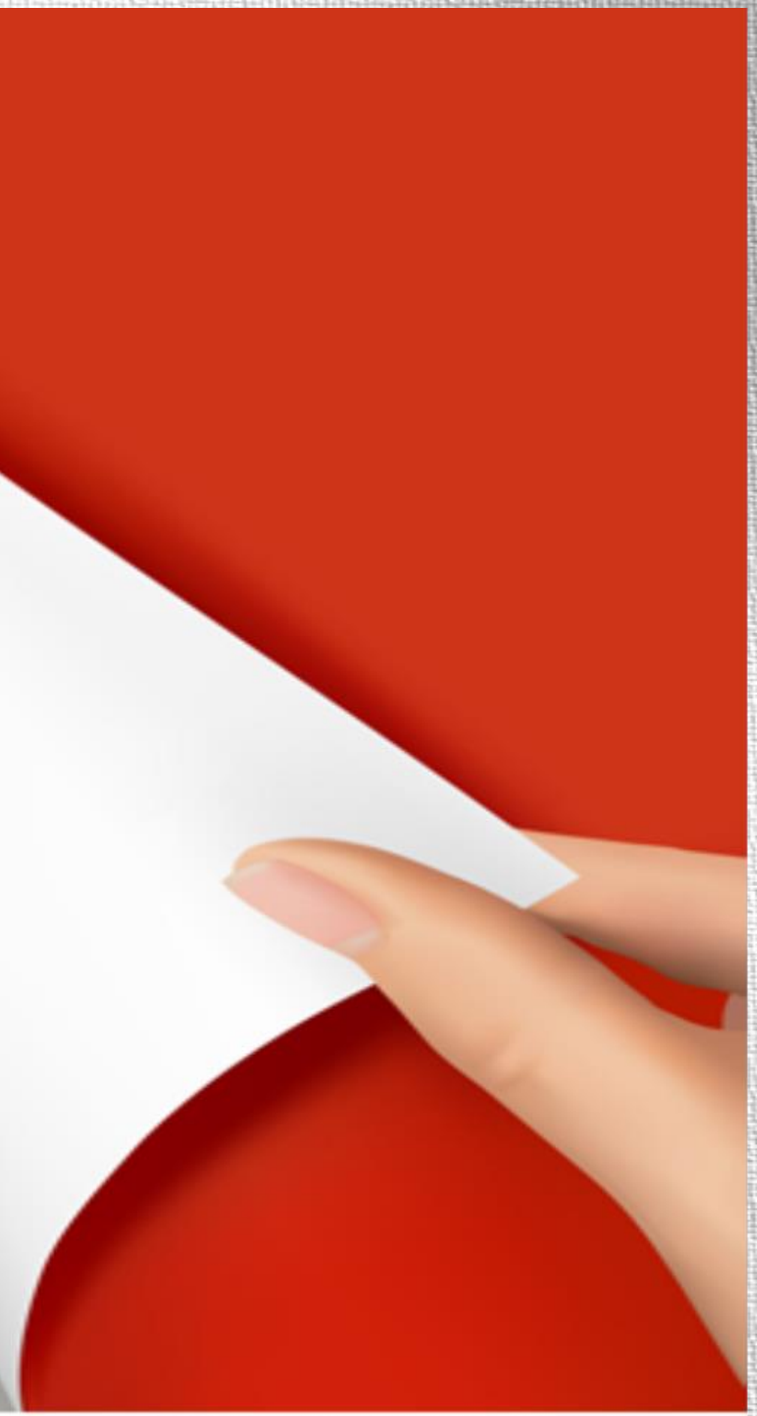
Пример ИСР


Цель проекта Requirement (Feature)
Описание продукта Requirement (Feature)
Заказ работ Requirement (Feature)
Функциональное требование Requirement (Functional)
Задача Task
Нефункциональное требование Requirement (Quality of Service)
Проблема Issue
Запрос на изменение требований Change Request
Ошибка Bug
Цель проекта
Этап проекта
Задача (Управление проектом)
Заказ работы (анализ требований)
Заказ работы
Заказ работы (Команда 3)
Ошибка (Уровень этапа)
Проблема (Уровень этапа)
Задача (Управление этапом проекта)
Заявка на изменение требований
Задача (анализ требований)
Заказ работы (Команда 1)
Заказ работы (Команда 2)
Задача (Верификация требования)
Задача (Проектирование и оценка трудозатрат)
Задача (Тестирование)
Ошибка (Уровень заказа)
Проблема (Уровень заказа)
Задача (Изменение требований)
Задача (Верификация требований)
Задача (Проектирование и оценка трудозатрат)
Задача (реализация)
Задача (тестирование)

Подведем итоги

- Иерархическая структура работ — способ оценить объем работ по проекту, разбив его элементы на более измеримые части.
- ИСР описывает все работы, которые должны быть выполнены в проекте. Все, что не отражено — не делается в проекте.
- Если напротив каждого элемента ИСР написать фамилию ответственного, то вовлеченность исполнителя будет выше в 3 раза
- Способов декомпозиции несколько, выбирай любой
- Глубина декомпозиции должна позволять оценить объем пакета работ (остальное можно детализировать в ходе проекта)
- Корректная ИСР — это структура без дублирования, с примерно одинаковыми по объему работ элементами и понятная заказчику

Методологии и технологии проектирования ПС



Основа проекта ПС

- Методологии
- Технологии
- Инструментальные средства проектирования (CASE-средства)

составляют основу проекта любой ПС.

Проект ПС = Методология + Технологии + CASE

Модель разработки ПО и методология

- **Модель процесса разработки ПО –** формализованное представление процесса разработки ПО. Часто при описании процессов вместо слова модель употребляется термин методология, что приводит к неоправданному расширению данного понятия.
- Но в отечественном техническом сообществе уже устоялся термин методология для описания процессов разработки ПО…

Методы и методологии

- **Метод** – совокупность приемов или операций для получения искомого результата.
- **Методология** – совокупность методов, применяемых в какой-либо науке.

Методология реализуется через конкретные технологии и поддерживающие их стандарты, практики, методики и инструментальные средства, которые обеспечивают выполнение процессов ЖЦ. Другими словами, **методология соединяет вместе технологии, стандарты и инструментальные средства в единое пространство проектирования.** Каждая методология делает это по-разному.

КРАТКО!

МЕТОД - логическая последовательность действий, приводящая к достижению поставленной цели.

СПОСОБ – средства + методы + формы деятельности

МЕТОДИКА - совокупность способов.

ТЕХНОЛОГИЯ – методика, дающая гарантированный результат

ЧТО ТАКОЕ МЕТОДОЛОГИЯ

Методология – СОВОКУПНОСТЬ МЕТОДОВ, применяемых в отдельных науках;

Методология – учение о структуре, логической организации, методах и средствах деятельности.

Методология - наука, обобщающая методики в рамках единой и общей цели.

- Создание программных продуктов – **технологический процесс**.
- При этом применяются различные **технологии**.
- Эти технологии относятся не только к кодированию, но и к анализу, проектированию, управлению проектом, разработке документации...

Технологии ...

- технологии программирования
- технологии проектирования
- технологии управления проектом
- технологии анализа
- технологии разработки
- технологии сопровождения
- ...

Технологии программирования...

Начнем разговор с того, что нам должно быть известно, с **технологий программирования**.

Что такое технология?

Технология – совокупность производственных процессов в определенной отрасли производства, а также научное описание способов производства*.

* *Источник*: **С.И. Ожегов. Словарь русского языка. - М.: Советская энциклопедия, 1975.**

ТЕХНОЛОГИИ ПРОГРАММИРОВАНИЯ

Технология программирования – совокупность методов, приемов и средств для сокращения стоимости и повышения качества разработки программных систем.

ТЕХНОЛОГИЯ ПРОЕКТИРОВАНИЯ

Технология проектирования определяется как совокупность трех составляющих:

1. ***пошаговой процедуры***, определяющей последовательность технологических операций проектирования;
2. ***критериев и правил***, используемых для оценки результатов выполнения технологических операций;
3. ***нотаций*** (графических и текстовых средств), используемых для описания проектируемой системы.

Технологическая операция



Стандарты

Реальное **применение любой технологии** проектирования, разработки и сопровождения ИС в конкретной организации и конкретном проекте **невозможно без выработки ряда стандартов** (правил, соглашений), которые должны соблюдаться всеми участниками проекта. К таким стандартам относятся следующие:

- стандарт проектирования;
- стандарт оформления проектной документации;
- стандарт пользовательского интерфейса.

Стандарт проектирования

Должен устанавливать:

- набор необходимых моделей (диаграмм) на каждой стадии проектирования и степень их детализации;
- правила фиксации проектных решений на диаграммах, в том числе: правила именования объектов (включая соглашения по терминологии), набор атрибутов для всех объектов и правила их заполнения на каждой стадии, правила оформления диаграмм, включая требования к форме и размерам объектов, и т. д.;
- требования к конфигурации рабочих мест разработчиков, включая настройки операционной системы, настройки CASE-средств, общие настройки проекта и т. д.;

- механизм обеспечения совместной работы над проектом, в том числе: правила интеграции подсистем проекта, правила поддержания проекта в одинаковом для всех разработчиков состоянии (регламент обмена проектной информацией, механизм фиксации общих объектов и т.д.), правила проверки проектных решений на непротиворечивость и т. д.

Стандарт оформления проектной документации

Должен устанавливать:

- комплектность, состав и структуру документации на каждой стадии проектирования;
- требования к ее оформлению (включая требования к содержанию разделов, подразделов, пунктов, таблиц и т.д.),
- правила подготовки, рассмотрения, согласования и утверждения документации с указанием предельных сроков для каждой стадии;
- требования к настройке издательской системы, используемой в качестве встроенного средства подготовки документации;
- требования к настройке CASE-средств для обеспечения подготовки документации в соответствии с установленными требованиями.

Стандарт пользовательского интерфейса

Стандарт интерфейса пользователя должен устанавливать:

- правила оформления экранов (шрифты и цветовая палитра), состав и расположение окон и элементов управления;
- правила использования клавиатуры и мыши;
- правила оформления текстов помощи;
- перечень стандартных сообщений;
- правила обработки реакции пользователя.

CASE-технологии

Назначение

- Применяются прежде всего на этапах анализа и проектирования
- Обеспечивают качество принимаемых технических решений
- Поддерживают подготовку проектной документации
- Применение средств визуального (наглядного) представления информации
  - построение структурных и иных диаграмм (моделей) в реальном масштабе времени
  - использование многообразной цветовой палитры
  - сквозная проверка синтаксических правил

- Структурное программирование
- Модульное программирование
- Объектно-ориентированное программирование
- Компонентное программирование
- Автоматное программирование

Технологии программирования



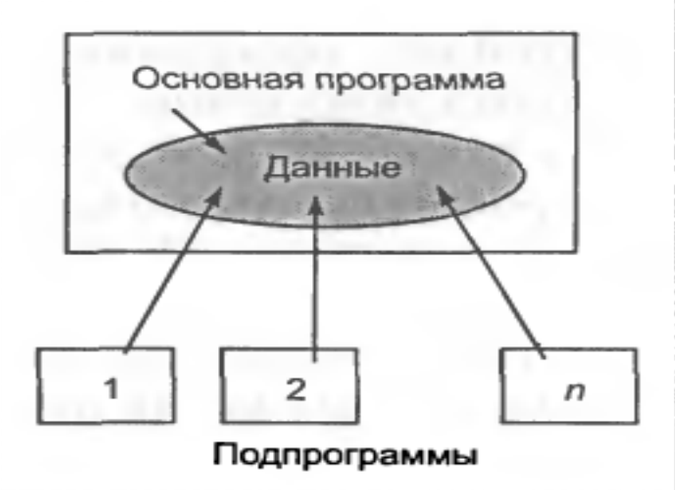
Добавить отсюда:
https://github.com/kolei/OAP/blob/master/articles/t2l2.md

Основные этапы развития технологии разработки

- **Первый этап** – «стихийное» программирование.
- **Второй этап** – структурный подход к программированию (60 – 70-е годы XX в.).
- **Третий этап** – объектный подход к программированию (с середины 80-х годов до нашего времени).
- **Четвертый этап** – компонентный подход и CASE-технологии (с середины 90-х годов до нашего времени).
- **Пятый этап** – разработка, ориентированная на архитектуру и CASE-технологии (с начала XXI в. до нашего времени).

1 этап
Стихийное программирование

- Использовалась до середины 60-х годов.
- Отсутствуют технологии, программирование – это искусство.



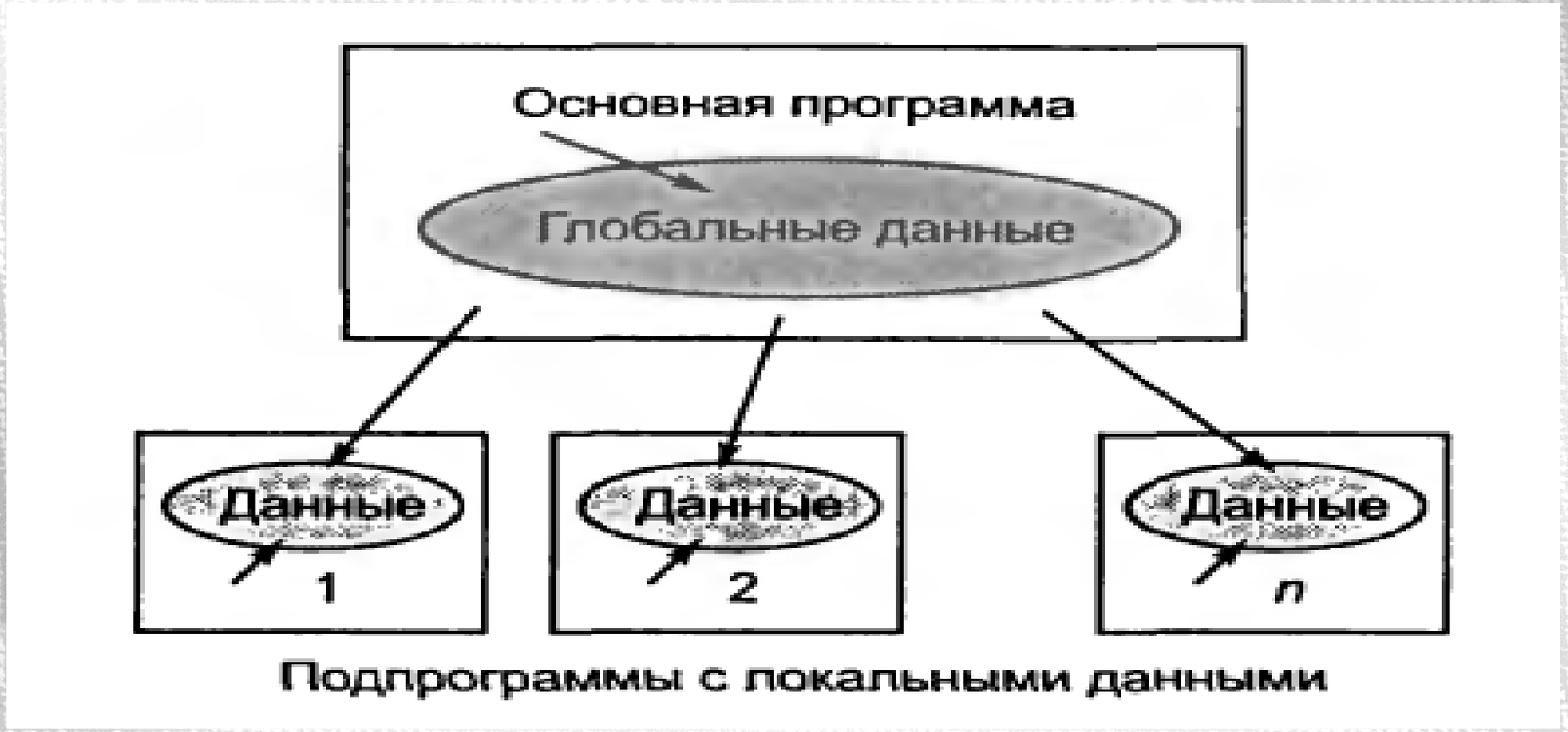
- Типичная программа состояла из основной программы, области глобальных данных, и набора подпрограмм, выполнявших обработку всех данных или их части.

Стихийное программирование

- Самое слабое место – при увеличении количества подпрограмм возрастала вероятность искажения части глобальных данных какой-либо подпрограммой.
- Чтобы сократить количество таких ошибок в подпрограммах, было предложено применять локальные данные.

Стихийное программирование.

Архитектура программы, использующей подпрограммы с локальными данными.



Подпрограммы с покальными данными

Переход к следующей технологии

- В начале 60-х годов разразился кризис программирования.
- Это было вызвано несовершенством технологии программирования. Стихийно использовалась разработка «снизу-вверх»

2 этап

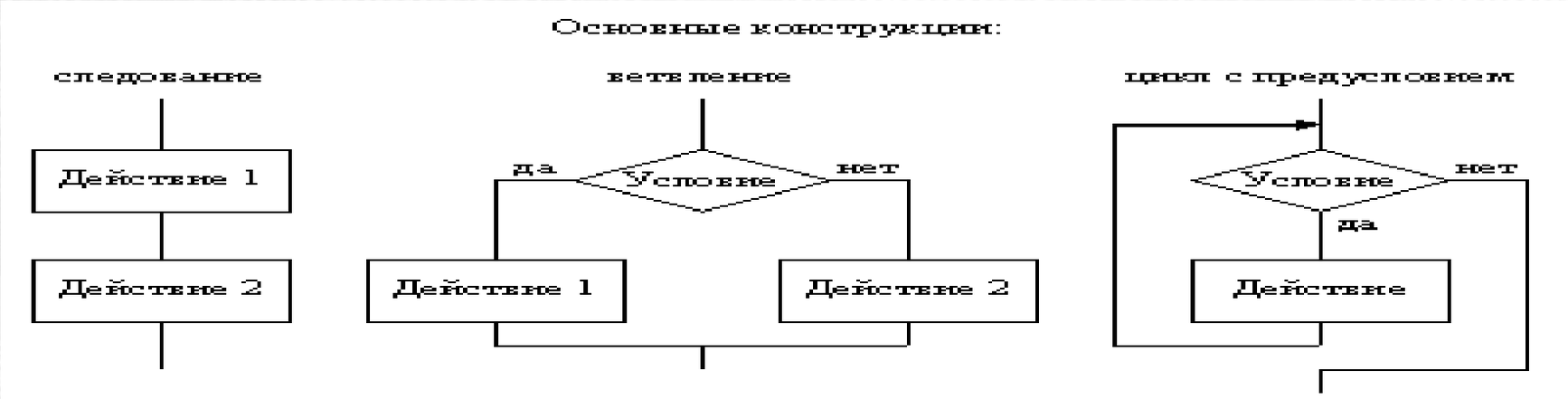
Структурное программирование (60-70 годы)

В основе СП лежит *процедурная декомпозиция* сложных систем с целью последующей реализации в виде отдельных небольших подпрограмм. Такой подход требовал представления задачи в виде иерархии подзадач простейшей структуры. Таким образом проектирование осуществлялось «сверху вниз» и подразумевало реализацию общей идеи с проработкой интерфейсов подпрограмм.

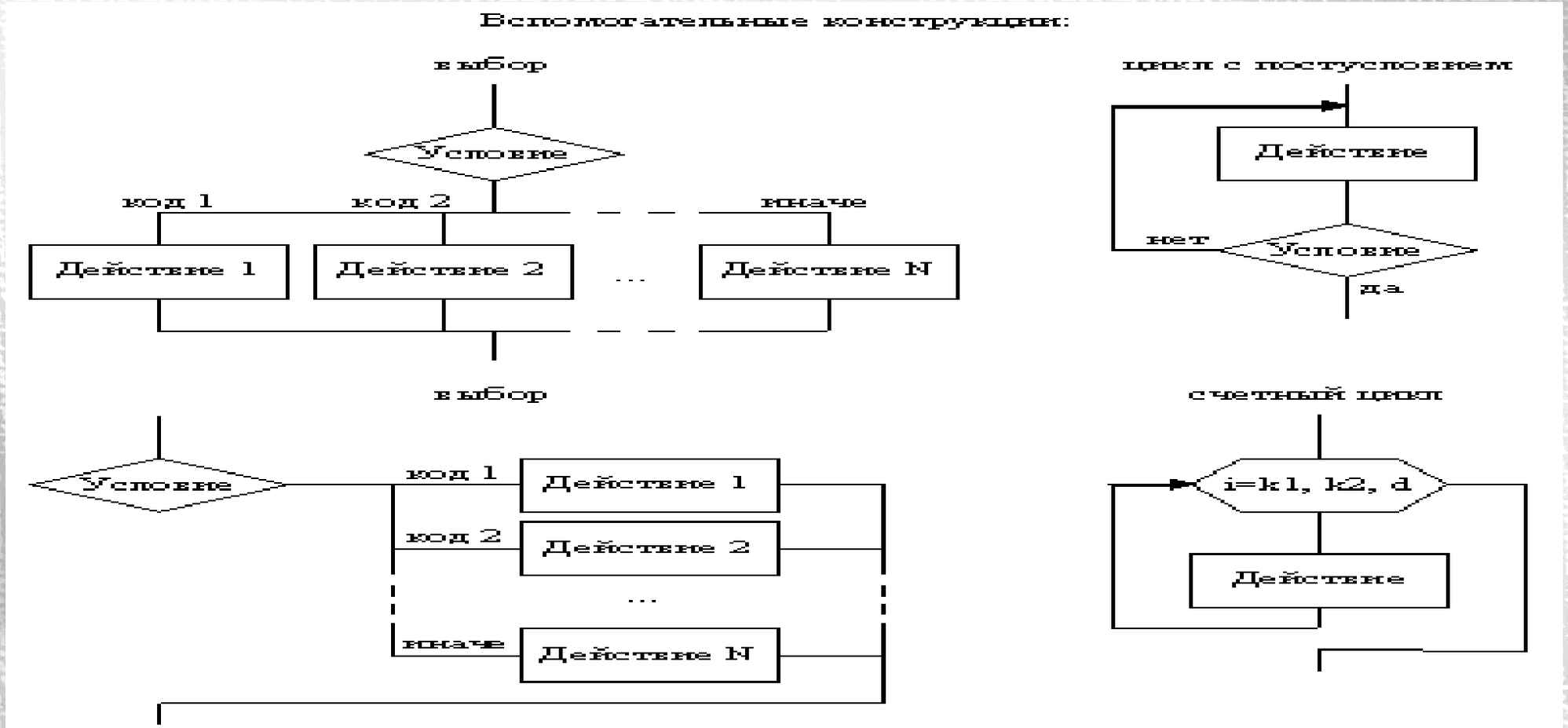
Структурное программирование

Центральный технологический принцип структурного программирования состоит в том, что формулировку алгоритма и его запись в виде программы рекомендуется выполнять на основе базиса из трех алгоритмических конструкций, применяя при необходимости их суперпозицию. Результатом последовательного применения этого принципа будет более ясная структура программы

Блок-схемы базисных алгоритмических конструкций



Дополнительные структуры алгоритмов



а – выбор, б – цикл до, в – цикл с заданным числом повторений

Переход к следующей технологии

Дальнейший рост размеров и сложности разрабатываемого ПО потребовал развития *структурирования данных*. Появляется возможность определения пользовательских типов данных. Одновременно усиливается стремление разграничить доступ к глобальным данным программы.

В результате появилась и начала развиваться технология модульного программирования.

Модульное программирование

МП предполагает выделение групп подпрограмм, использующих одни и те же глобальные данные, в отдельно компилируемые модули (библиотеки подпрограмм). Например, модуль графических ресурсов, модуль подпрограмм вывода на принтер. Связи между модулями осуществляются через **специальный интерфейс**, в то время как доступ к реализации модуля (телам подпрограмм и внутренним переменным) **запрещен**.

Эту технологию поддерживают версии современных языков.

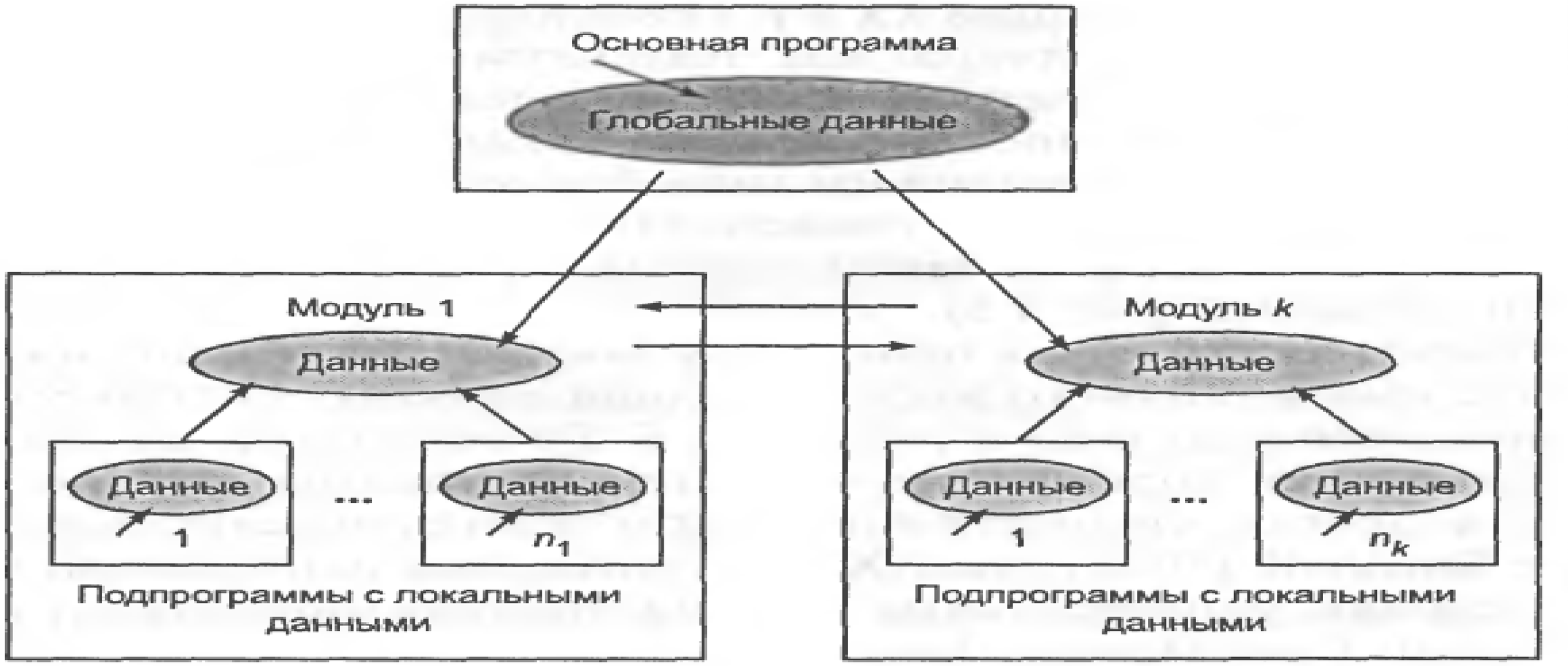
Модульное программирование

Если концепция структурного программирования предлагает некоторый универсальный алгоритмический базис, то модульное программирование состоит в разработке под конкретную задачу или круг задач (предметную область) собственного базиса в виде набора модулей, позволяющего наиболее эффективно по целому ряду критериев построить программный комплекс. Модули, входящие в базис, это целые программы (в отличие от примитивов структурного программирования), решающие некоторые подзадачи основных задач.

Модульное программирование

С применением модульного программирования появляются возможности коллективной разработки программ как набора «независимых» частей, последовательного уменьшения сложности методом разбиения сложной задачи на более простые подзадачи, наконец, возможности повторного использования созданного ранее кода.

Архитектура программы, состоящей из модулей



Модули с локальными данными и подпрограммами

Переход к следующей технологии

- Практика показала, что структурный подход в сочетании с модульным программированием позволяет получать надежные программы, размер которых не превышает 100 000 операторов.
- Узким местом модульного программирования является то, что ошибка в интерфейсе при вызове подпрограммы выявляется только при выполнении программы. Из-за раздельной компиляции модулей обнаружить эти ошибки раньше невозможно.
- При увеличении размера программы обычно возрастает сложность межмодульных интерфейсов и с некоторого момента предусмотреть взаимовлияние отдельных частей программы становится невозможно.

Структурный подход в сочетании с модульным программированием будет ниже рассмотрен нами достаточно подробно

3 этап

Объектно-ориентированное программирование

(с середины 80 до конца 90)

В основе технологии лежат объектная модель и объектная декомпозиция.
К основные принципам объектной модели часто относят следующие:

- **абстракция;**
- **инкапсуляция;**
- **иерархия (наследование, агрегация);**
- **полиморфизм;**
- **модульность.**

Суть объектной декомпозиции состоит в выделении в предметной области классов и объектов, а также связей между ними, и лишь потом данных и алгоритмов, которыми характеризуется каждый класс. Таким образом, именно классы становятся основным «строительным блоком» в ООП, тогда как ранее таковыми блоками являлись алгоритмы.

Основные методы и концепции ООП

- **Метод объектно-ориентированной декомпозиции** – заключается в выделении объектов и связей между ними. Метод поддерживается концепциями инкапсуляции, наследования и полиморфизма.
- **Метод абстрактных типов данных** – метод, лежащий в основе инкапсуляции. Поддерживается концепцией абстрактных типов данных.
- **Метод пересылки сообщений** – заключается в описании поведения системы в терминах обмена сообщениями между объектами. Поддерживается концепцией сообщения.

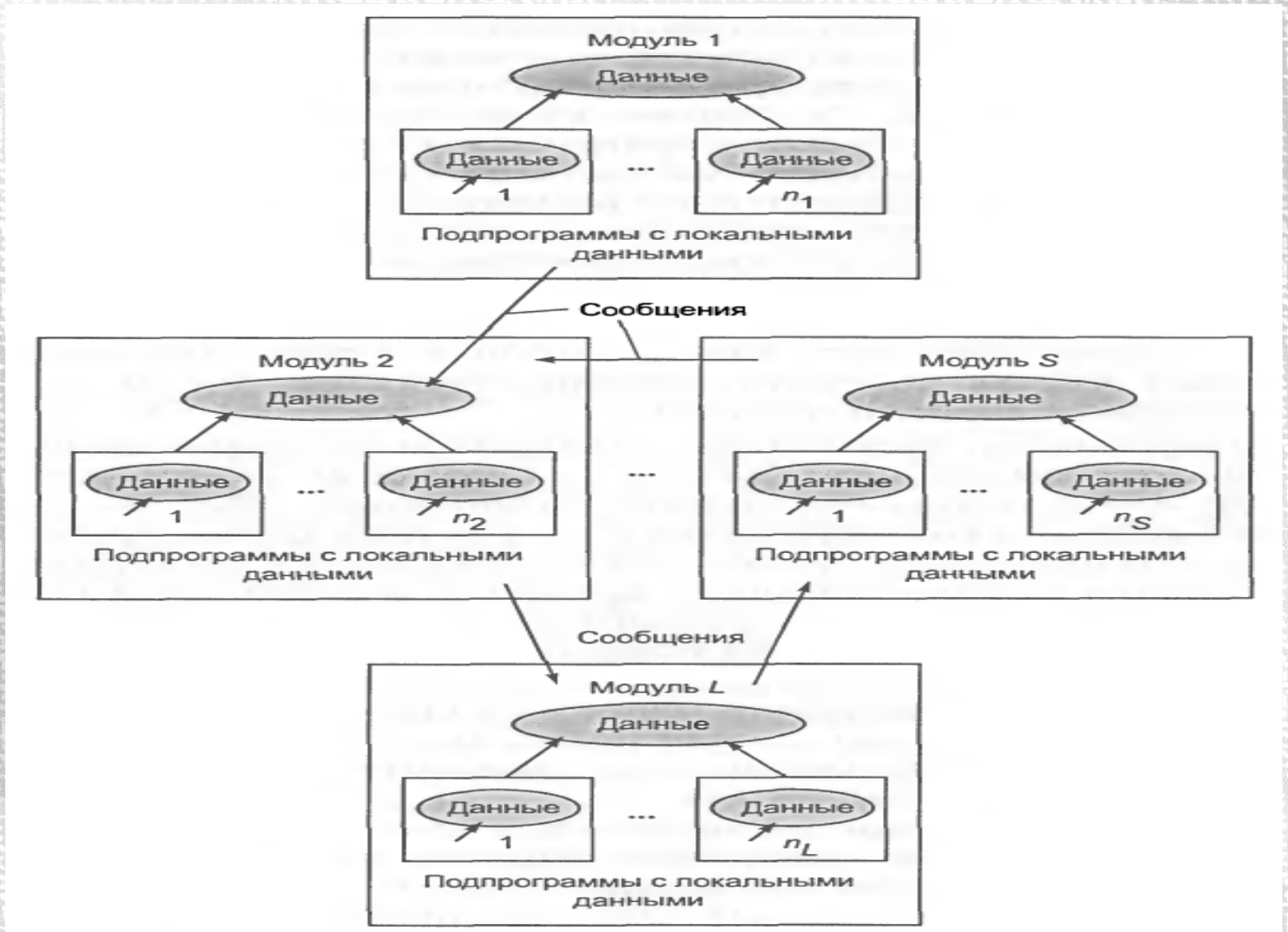
Вычислительная модель чистого ООП поддерживает только одну операцию – посылку сообщения объекту. Сообщения могут иметь параметры, являющиеся объектами. Само сообщение тоже является объектом.

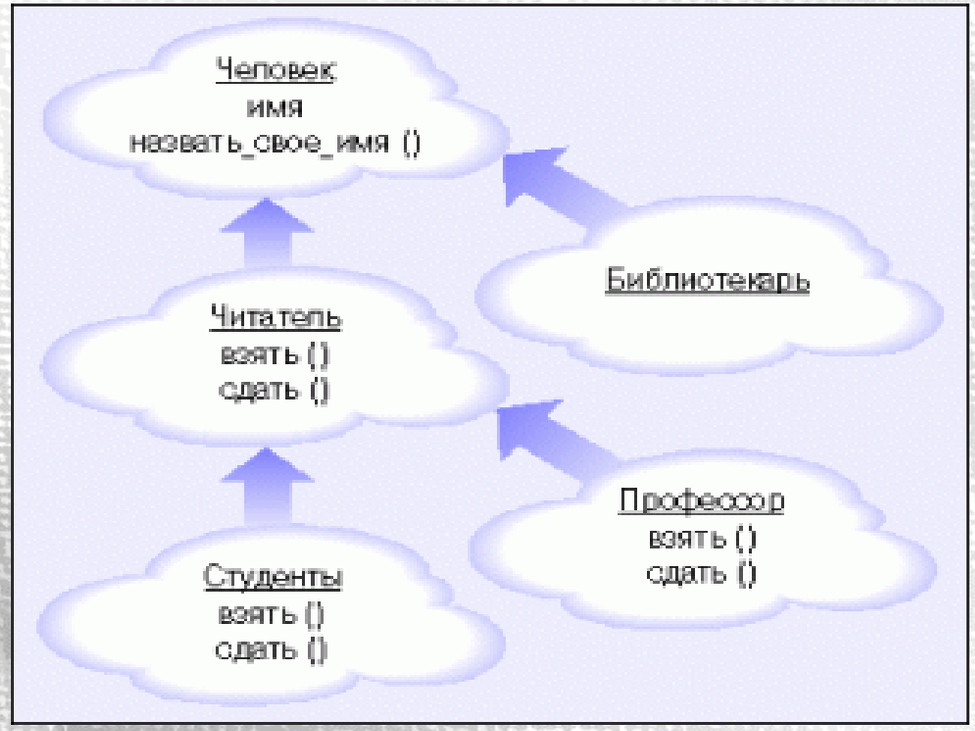
Объект имеет **набор обработчиков сообщений** (набор методов). У объекта есть поля – персональные переменные данного объекта, значениями которых являются ссылки на другие объекты. В одном из полей объекта хранится ссылка на объект-предок, которому переадресуются все сообщения, не обработанные данным объектом. Структуры, описывающие обработку и переадресацию сообщений, обычно выделяются в отдельный объект, называемый классом данного объекта. Сам объект называется экземпляром указанного класса.

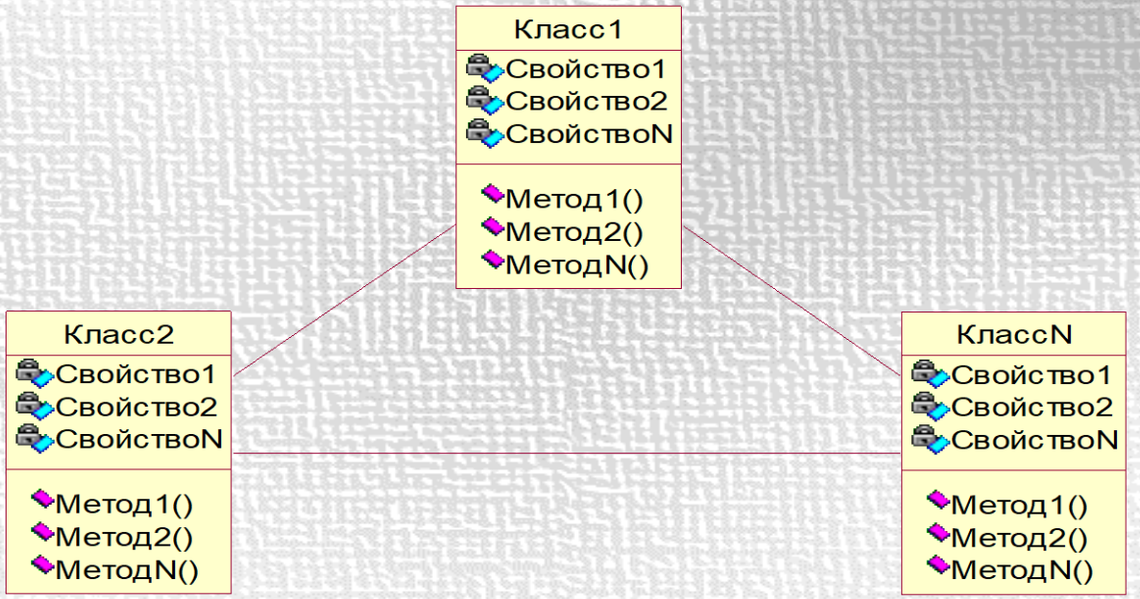
У каждого объекта есть ссылка на класс, к которому он относится. При приеме сообщения объект обращается к классу для обработки данного сообщения. Сообщение может быть передано вверх по иерархии наследования, если сам класс не располагает методом для его обработки. Если обработчик событий для сообщения выбирается динамически, то методы, реализующие обработчиков событий, принято называть **виртуальными**.

Естественным средством структурирования в данной методологии являются классы. Классы определяют, какие поля и методы экземпляра доступны извне, как обрабатывать отдельные сообщения и т. п. В чистых объектно-ориентированных языках извне доступны только методы, а доступ к данным объекта возможен только через его методы.

Архитектура программы при объектно-ориентированном подходе




Человек
имя
назвать_свое_имя ()
Библиотекарь
Читатель
взять ()
сдать ()
Профессор
взять ()
сдать ()
Студенты
взять ()
сдать ()


Класс1
Свойство1
Свойство2
СвойствоN
Метод1()
Метод2()
МетодN()
Класс2
Свойство1
Свойство2
СвойствоN
Метод1()
Метод2()
МетодN()
КлассN
Свойство1
Свойство2
СвойствоN
Метод1()
Метод2()
МетодN()

4 этап

Компонентное программирование

(с середины 90-х и до нашего времени)

Компонентное программирование представляет собой развитие объектно-ориентированной технологии. В отличие от ООП введен следующий уровень абстракции – классы объединяются в компоненты.

Компонент:

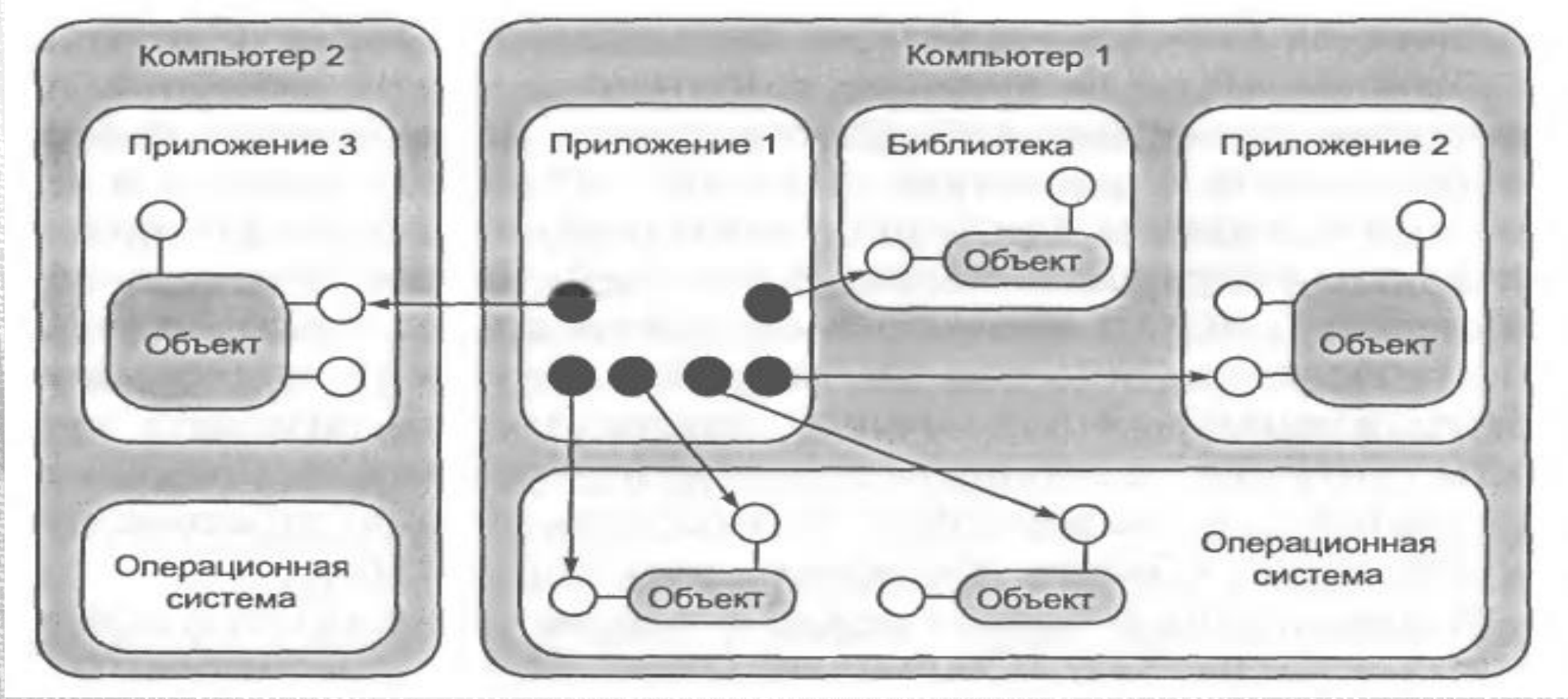
- программный код в виде самостоятельного модуля;
- может быть использован в неизменном виде;
- может допускать настройку;
- обладает поведением (функциональностью).

Основной принцип компонентного программирования: сборка приложения из готовых компонент, в общем случае написанных на разных языках.

Компонент изолирован от внешнего мира своим интерфейсом – набором методов (их *сигнатурами*).

Компонентная программа – набор независимых компонент, связанных друг с другом посредством интерфейсов.

Взаимодействие программных компонентов различных типов



Технологии компонентного подхода

- Технологии на базе COM (DCOM)

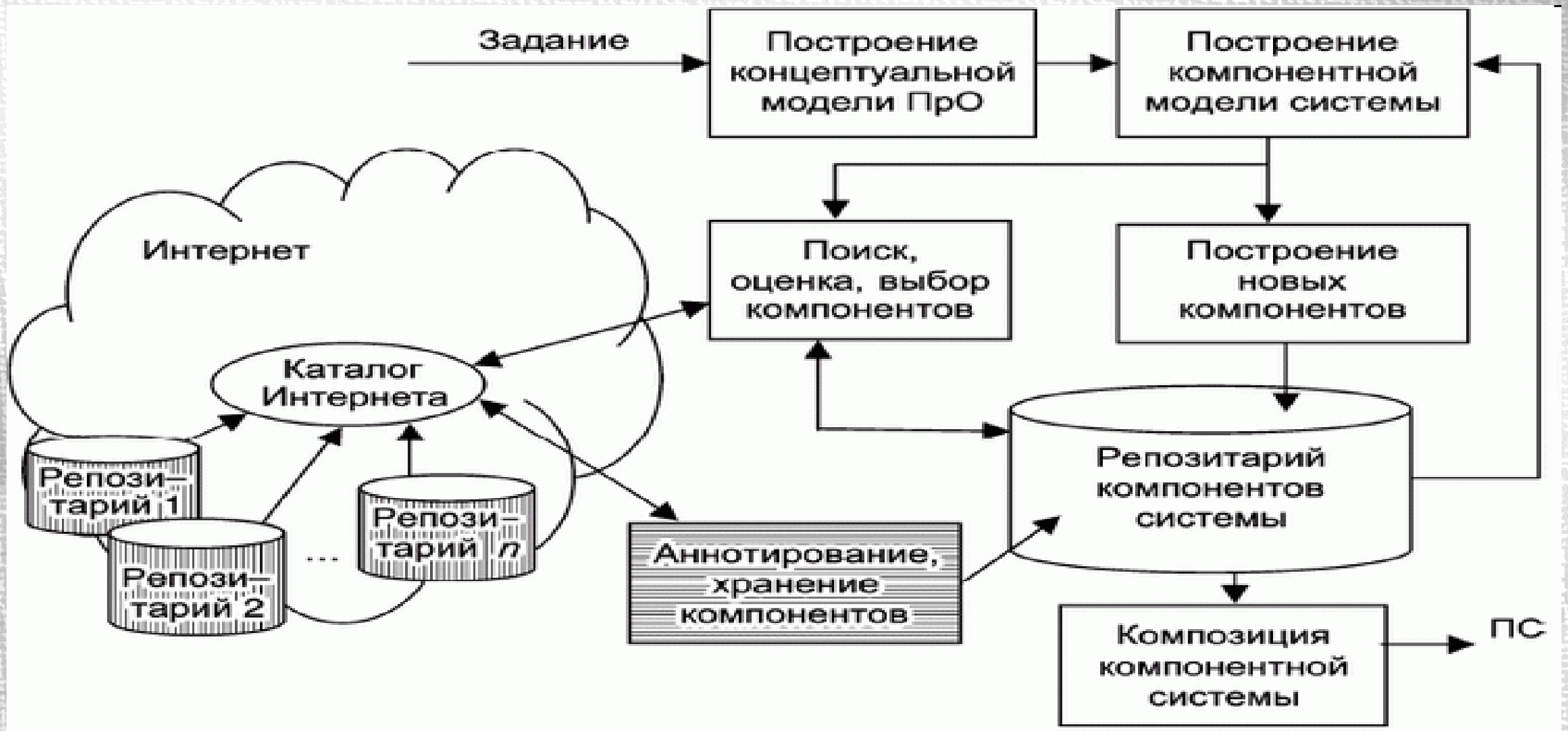
(компонентная модель объекта)

  - OLE-automation
  - ActiveX
  - MTS (сервер управления транзакциями)
  - MIDAS (сервер многозвенных распределенных приложений)
- Технология создания распределенных приложений CORBA

- **Компонентное сборочное программирование** — объектно ориентированное сборочное программирование, основанное на распространении классов в бинарном виде и предоставление доступа к методам класса через строго определенные интерфейсы. Компонентное сборочное программирование поддерживают технологические подходы COM, CORBA, .Net.
- **Аспектно-ориентированное сборочное программирование** — разновидность сборочного программирования, основанная на сборке полнофункциональных приложений из многоаспектных компонентов, инкапсулирующих различные варианты реализации.

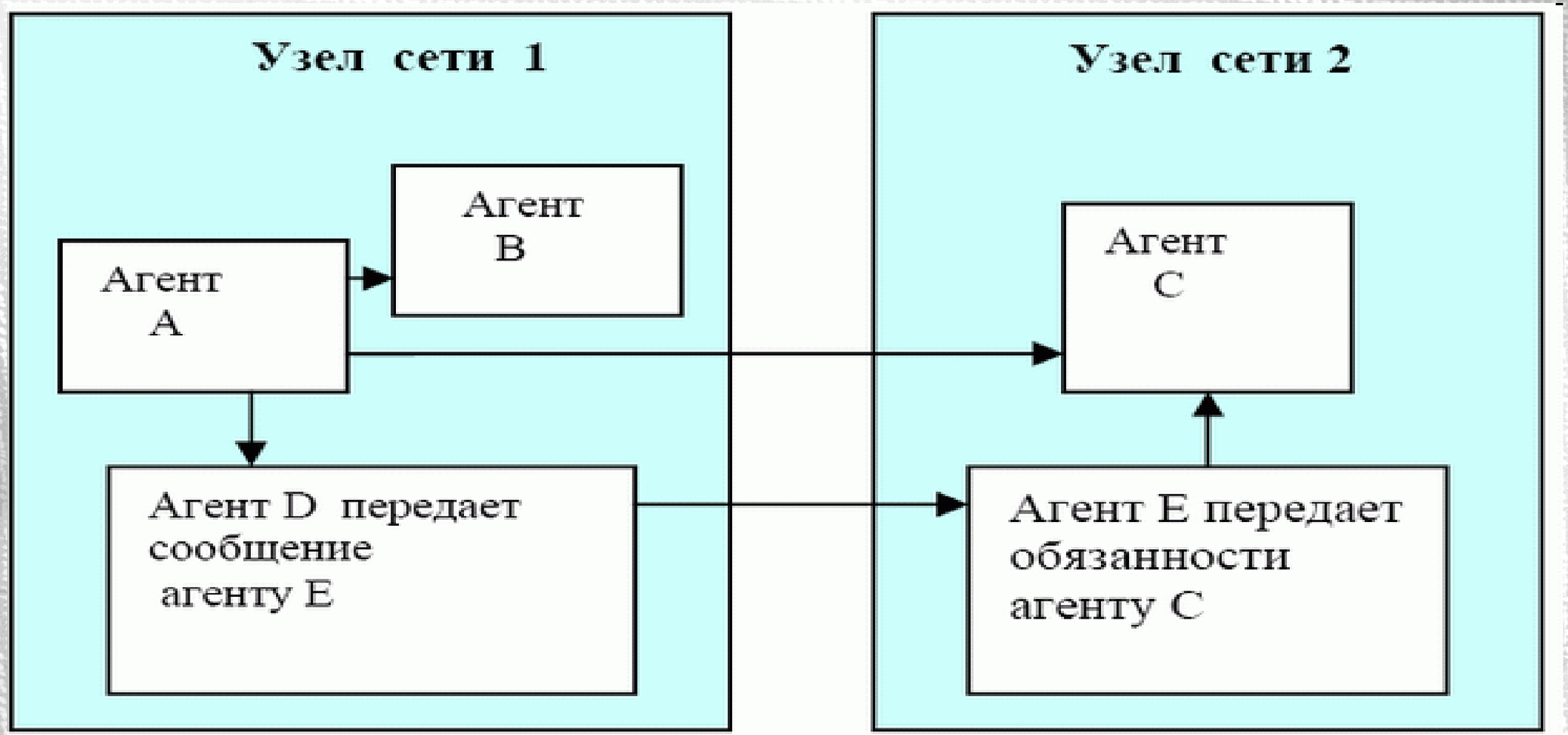
- **Объектно-ориентированное сборочное программирование** — разновидность сборочного программирования: основанная на методологии объектно ориентированного программирования; и предполагающая распространение библиотек классов в виде исходного кода или упаковку классов в динамически компонуемую библиотеку…. …
- **Модульное сборочное программирование** — разновидность сборочного программирования, основанная на процедурах и функциях методологии структурного императивного программирования.

Концептуальна схема построения ПС из компонентов в среде Интернет



Агентное программирование

Пример взаимодействия агентов в разных средах



5 этап

Разработка, управляемая моделями и CASE-технологии (с начала XXI в. до нашего времени)

Разработка, управляемая моделями, (*model-driven development*) — это стиль разработки программного обеспечения, когда модели становятся основными артефактами разработки, из которых генерируется код и другие артефакты.

Модели описанные на предметно-ориентированном языке программирования могут быть использованы, как точки расширения каркасов.

Наиболее известными MDE-инициативами являются:

- концепция модельно-ориентированного подхода к разработке программного обеспечения (**Model Driven Architecture**), разрабатываемая консорциумом OMG.
- **Eclipse Modeling Framework** — свободный фреймворк, основанный на Eclipse, для генерации кода, инструментов и прочих приложений на основе структурированной модели данных (метамодели).

Model Driven Architecture — модельно-ориентированный подход к разработке программного обеспечения

Для конструирования программного приложения **должна быть построена подробная, формально точная модель, из которой потом может быть автоматически генерирован исполняемый программный код приложения**.

Основные шаги разработки:

- Сначала разрабатывается модель предметной области проектируемого приложения, полностью независимая от имплементирующей технологии. Она называется Platform Independent Model (PIM).
- Затем PIM автоматически трансформируется специальным инструментом в платформо-зависимую модель (Platform Specifical Model, PSM).

- PSM переводится в исходный код на соответствующем языке программирования.

Такова схема в идеальном OMG-мире. В реальных современных проектах часть бизнес-логики приходится реализовать вручную. Но поскольку этот код отделен от генерированного системой, большой проблемы это не представляет.

Примеры

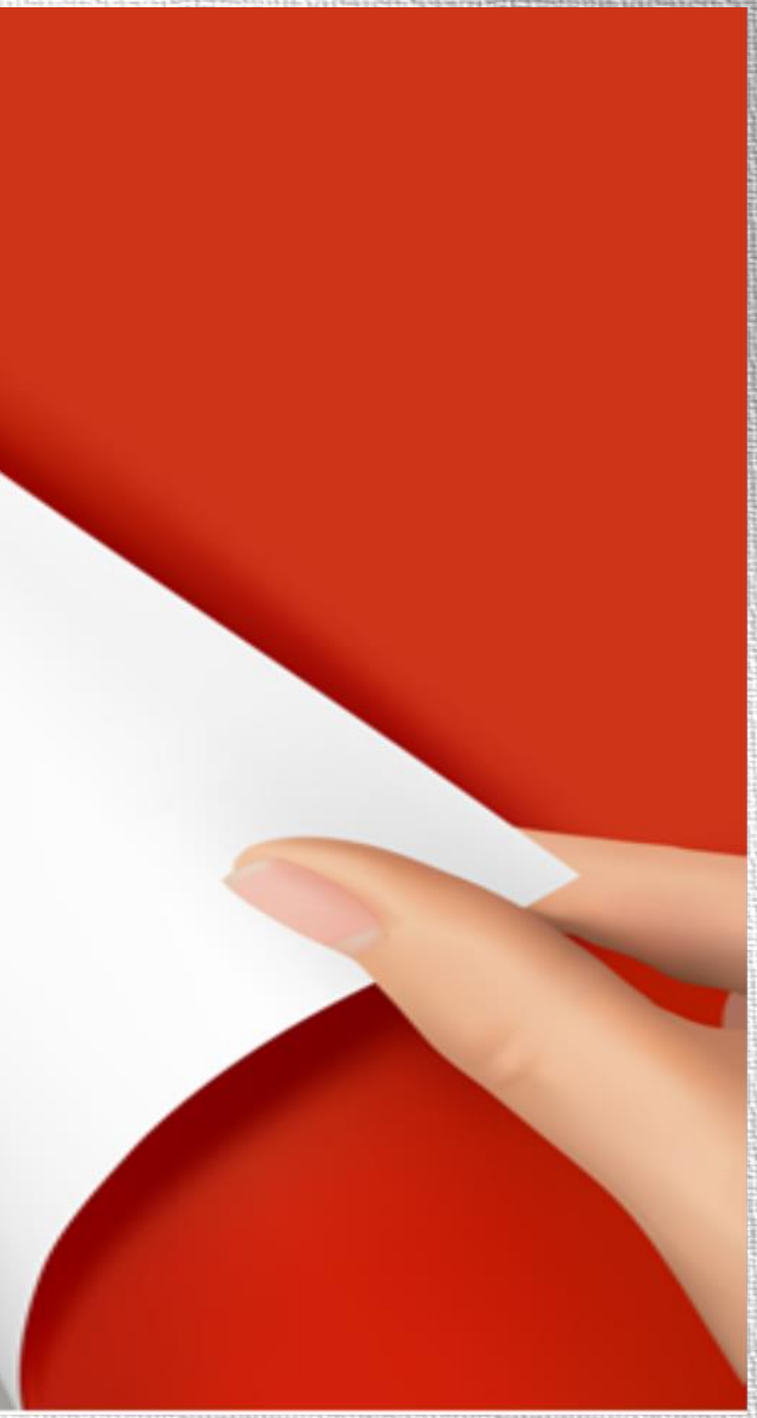
Примером реализации MDA можно считать технологию CASEBERRY и платформу Flexberry, разрабатываемую с её помощью.

В основе разработки лежит UML-модель, создаваемая при помощи собственного инструмента создания UML-диаграмм. Есть возможность генерации как Windows, так и Web-приложений на языке C#. Генерируемый код делится на 2 части: первую часть генератор кода будет перезаписывать при внесении изменений в модель, вторая часть останется для него неприкасаемой. Таким образом, можно наращивать функционал создаваемого приложения параллельно с внесением изменений в модель, при этом функционал не будет утерян при перегенерации кода.

Бизнес-логика вынесена в отдельный проект, для её реализации создаются заготовки, значительно облегчающие её (бизнес-логики) добавление.

Стратегии разработки программной системы

- нисходящая («сверху–вниз»)
- восходящая («снизу–вверх»)
- «расширения ядра» («изнутри к границам»)
- «формирования ядра» («от границ внутрь»)



Нисходящая и восходящая стратегии разработки программной системы

В процессе проектирования и реализации программной системы как правило используется одна из следующих стратегий:

1. **нисходящая («сверху–вниз»),**
2. **восходящая («снизу–вверх»),**
3. **«расширения ядра» («изнутри к границам»),**
4. **«формирования ядра» («от границ внутрь»).**

Нисходящая стратегия разработки программной системы

Нисходящее (сверху вниз) **проектирование** начинается с постановки задачи, которую требуется решить, и развивается далее путем разбиения ее на подзадачи. В конечном счете, это приводит к таким подзадачам, решение которых легко выразить в терминах базовых конструкций. Процесс проектирования включает в себя следующие действия:

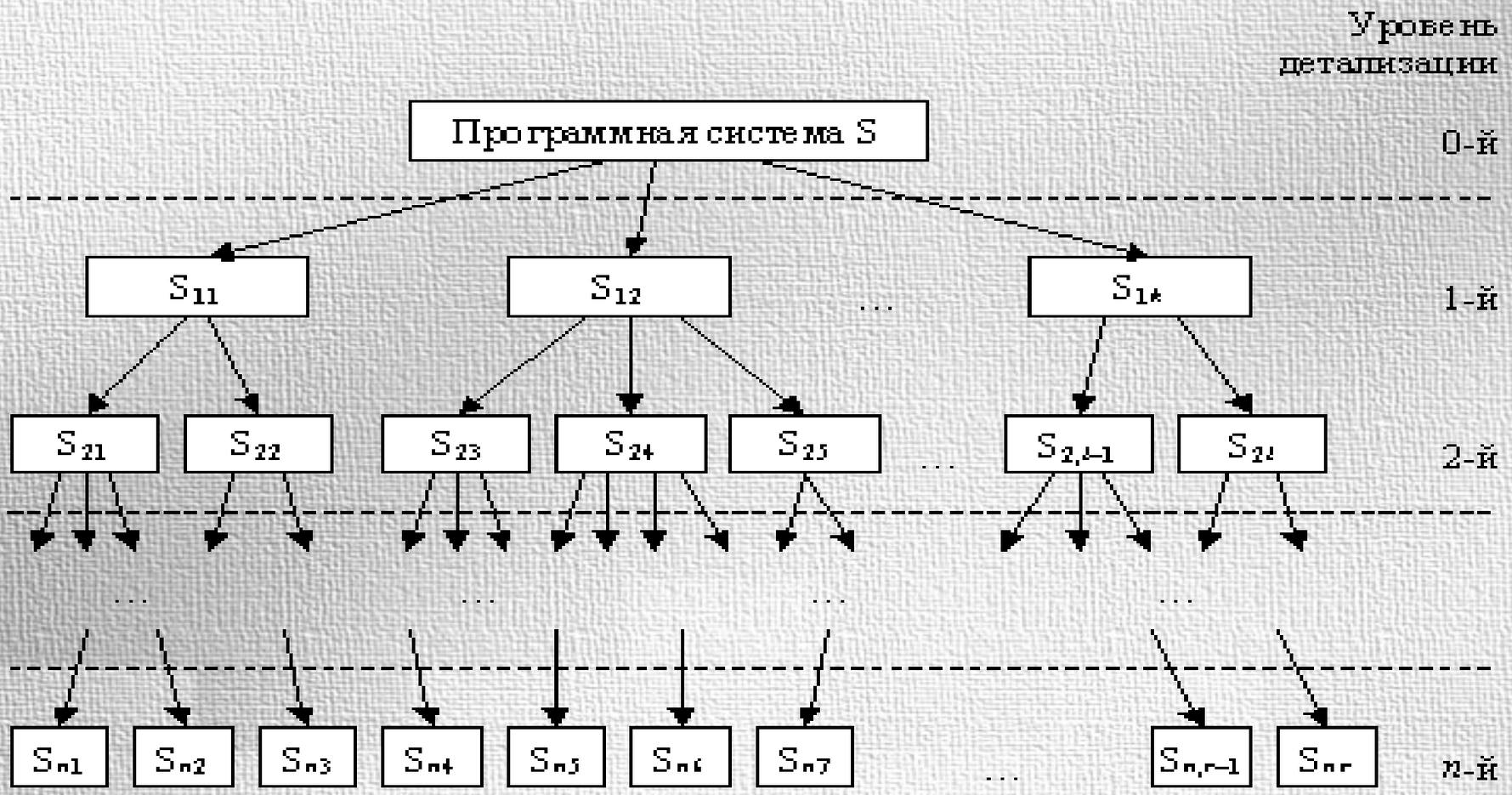
- *производится декомпозиция общей задачи на точно определенные подзадачи;*
- *доказывается, что если каждая подзадача решена корректно и полученные решения взаимосвязаны, то первоначальная задача будет решена корректно.*

Процесс декомпозиции повторяется для подзадач, и доказывается корректность для выполненного разбиения. Этот процесс повторяется до получения подзадач настолько простых, что решение выражается базовыми конструкциями.

Такой способ проектирования алгоритмов и программ в виде иерархической структуры позволяет решать громоздкие и сложные задачи. При нисходящей разработке появляется возможность проверки алгоритма или проверки и отладки программы с самого начала разработки и тем самым создается ситуация выявления ошибок на ранних стадиях проектирования программ.

Так как модули программ разрабатываются сверху вниз, то вместо модулей нижнего уровня используются **программные заглушки**. Они требуются только для того, чтобы позволить модулю верхнего уровня проверить взаимодействие с модулем нижнего уровня. Программная заглушка содержит входные и выходные операторы и выдает сообщения о своем выполнении.

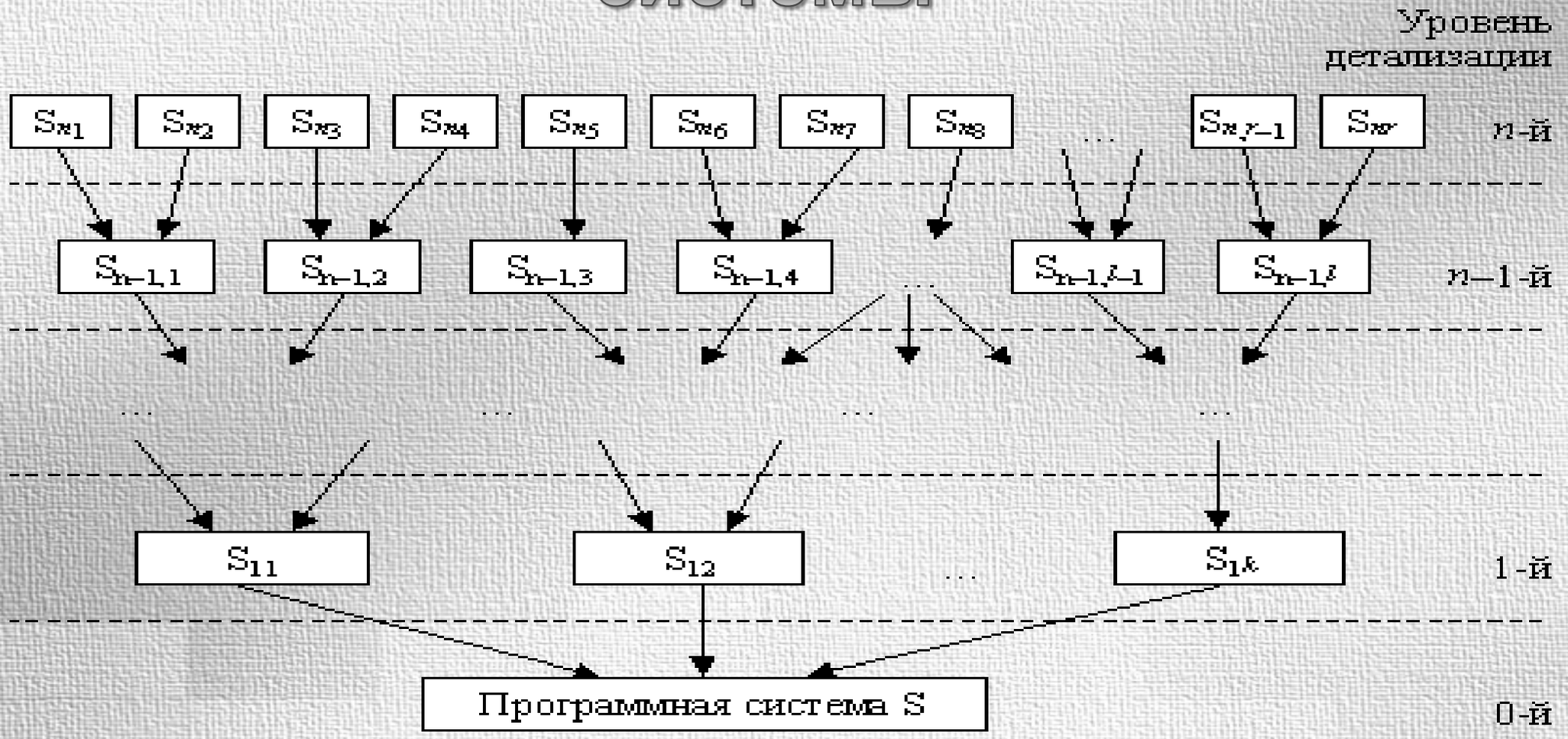
Концептуальная схема нисходящей стратегии разработки программной системы



Восходящая стратегия разработки программной системы

- **Восходящее** (снизу вверх) ***проектирование*** использует идею выделения достаточно крупных подалгоритмов и модулей, каждый из которых реализует определенные функции.
- Выбор модулей определяется различными соображениями: функциональной законченностью, размерами, однородностью структур данных и др. Для выделенных модулей составляются спецификации, определяется связь по входу и выходу. Далее каждый модуль алгоритма автономно разрабатывается и проверяется. После разработки модулей с учетом связей по входу и выходу осуществляется объединение в подсистемы, которые подвергаются контролю. Испытанные подсистемы объединяются в программную систему, которая подвергается комплексной проверке.
- Следовательно, восходящее проектирование начинается с уровня базовых понятий и развивается путем формирования на их основе абстрактных понятий и, наконец, приводит к таким понятиям, в терминах которых выражается решение всей задачи.

Концептуальная схема восходящей стратегии разработки программной системы



Основными недостатками восходящего способа проектирования являются:

- *сложность процесса объединения отдельных модулей в алгоритм;*
- *трудность исправления ошибок, допущенных на ранних стадиях разработки;*
- *сложность контроля разработки и прогноза завершения.*

Реальная стратегия разработки алгоритмов и программ почти всегда представляет собой разумное сочетание нисходящего и восходящего способов проектирования (встречное проектирование).

При восходящем методе проектирование системы начинается с определения отдельных задач внутри системы. Затем изучается, как решение этих задач может использоваться в качестве абстрактных инструментов для решения более общих задач. Многие годы этот подход считался хуже нисходящего метода проектирования. Однако в настоящее время методология восходящего проектирования получила мощную поддержку. Одна из причин этого заключается в том, что нисходящая методология ищет решение, в котором основной модуль использует подмодули, каждый из которых основывается на подмодулях, и т.д. Однако для многих систем наилучший вариант решения лишен подобной иерархической природы.

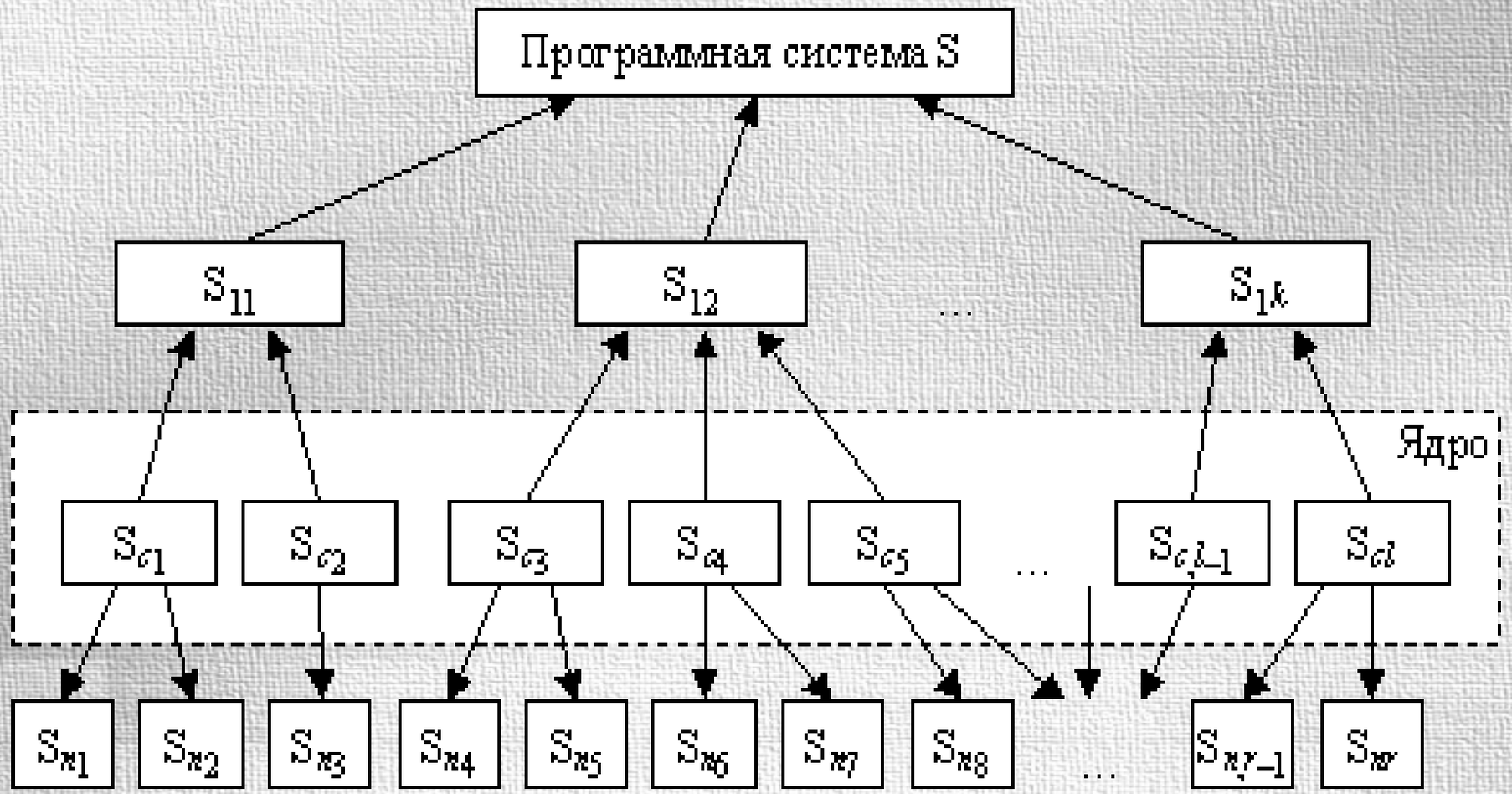
Действительно, при построении приложения по принципам архитектуры "клиент/сервер" или использовании методов параллельной обработки проектное решение, в котором два или несколько модулей взаимодействуют как равноправные партнеры, может оказаться более удачным, чем проект, состоящий из управляющего модуля, обращающегося к модулям подпрограмм для решения стоящих перед ним задач.

Другой причиной такого интереса к восходящему проектированию является то, что оно лучше соответствует задачам построения сложных систем программного обеспечения из предварительно созданных, стандартных, свободно продаваемых компонентов. Однако именно этот подход является наиболее современной тенденцией в технологии разработки программного обеспечения. Подробно мы рассмотрим эту стратегию ниже.

Стратегии «расширения ядра»
(«изнутри к границам»)

Подход к разработке программной системы на основе принципа «расширения ядра» или «изнутри к границам» представляет собой комбинацию двух предыдущих. Он предполагает первоначальное проектирование и разработку некоторой основы системы — её программного ядра, включающего компоненты, реализующие центральные, наиболее значимые функции. В дальнейшем ядро «наращивается» с использованием восходящего и нисходящего подходов. В соответствии с первым из них выполняется синтез новых, более крупных компонентов по принципу «изнури–вверх» с целью построения целостной системы, а нисходящий подход используется для декомпозиции существующих компонентов ядра по принципу «изнутри–вниз» с целью их детальной реализации.

Концептуальная схема стратегии «расширения ядра»

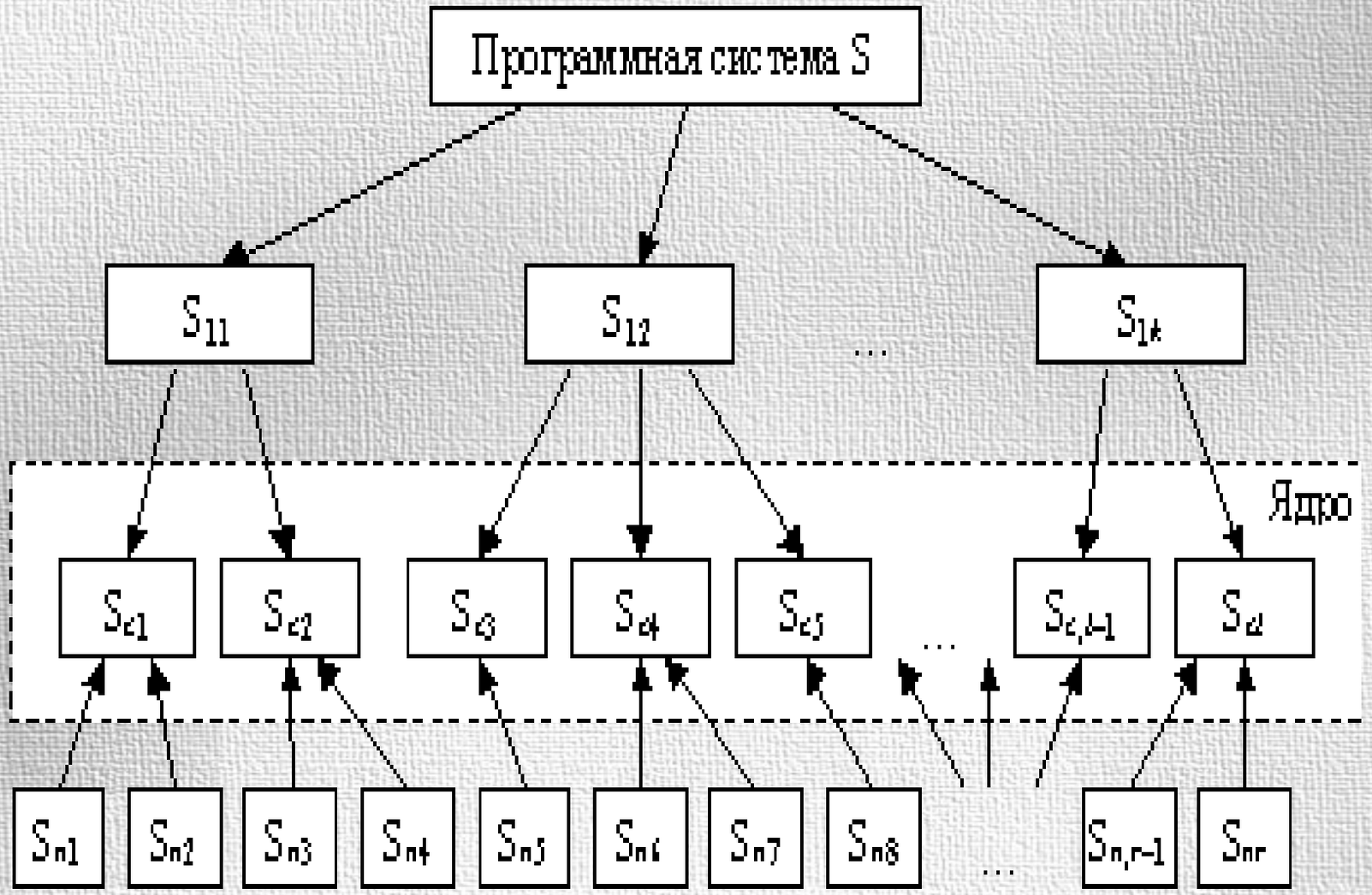


Стратегия «формирования ядра»

(«от границ внутрь»)

Стратегия проектирования и разработки на базе принципа «формирования ядра» или «от границ внутрь» также является комбинацией нисходящего и восходящего подходов. В этом случае в начале опреде-ляются и разрабатываются компоненты на самом верхнем и нижнем уровнях детализации системы. Затем выполняется декомпозиция компонентов на верхнем уровне и синтез компонентов на нижнем уровне, в результате чего формируется промежуточный уровень, состоящий из компонентов, образующих функци-ональное ядро программной системы.

Концептуальная схема стратегии «формирования ядра»



Технологичность программной системы и её характеристики

При индустриальном подходе к разработке и сопровождению программных систем значительную роль начинают играть технологические характеристики создаваемых программ. Для обеспечения требуемых свойств разрабатываемой системы применяются специальные технологические приемы и методики, которые включают в себя способы и правила декомпозиции системы, методы её проектирования, программирования и контроля качества.

Технологичность программной системы характеризует качество её проекта. Обеспечение высокой технологичности проекта особенно важно при разработке программной системы, рассчитанной на многолетнее интенсивное использование, или повышенных требованиях к качеству системы. Технологичность зависит от нескольких критериев.

1. **Проработанность моделей программной системы.** Чем лучше проработана модель создаваемой системы, тем четче определены решаемые подзадачи и используемые структуры данных для хранения входной, промежуточной и выходной информации, тем проще их проектирование и реализация и меньше вероятность ошибок, для исправления которых потребуются существенные изменения программ системы.
2. **Уровень независимости модулей системы.** Чем выше независимость модулей, тем проще они для понимания, разработки, модификации, тестирования и отладки.

3. **Стиль программирования.** Определяет стиль написания и оформления программ. Хороший стиль программирования способствует простоте чтения и понимания программы и снижению количества ошибок программирования.
4. **Степень повторного использования программного кода.** Увеличение степени повторного использования обеспечивается применением ранее созданных библиотек подпрограмм или классов и унификацией программных компонентов в ходе разработки. Однако, если степень повторного использования кода повышается искусственно, то технологичность проекта может существенно снизиться.

ПОДХОДЫ К РАЗРАБОТКЕ ПО

На сегодняшний день в инженерии программного обеспечения существует два основных подхода к разработке программных систем, различие между которыми обусловлено критериями декомпозиции:

1. Первый подход называют функционально-модульным, или структурным. В его основу положен принцип алгоритмической декомпозиции, когда выделяются функциональные элементы системы и устанавливается строгий порядок выполняемых действий.
2. Второй, объектно-ориентированный подход (Object-Oriented Analysis and Design, OOAD) использует объектную декомпозицию. При этом поведение системы описывается в терминах взаимодействия объектов.

Поддержка различных уровней принятия решений со стороны ИС и ИТ

Уровень принятия решений	*Направленность принимаемых решений*	*Типичные приложения ИС*	*Типичные ИТ-решения*
Стратегический	**Стратегии, поддерживающие долгосрочные цели организации**	**Анализ рынка и сбыта; планирование разработки товаров; оценка эффективности**	**Добыча данных (Data mining); управление знаниями**
Тактический	**Стратегии, поддерживающие краткосрочные задачи и распределение ресурсов**	**Анализ бюджета; прогнозирование фонда зарплаты; планирование запасов; обслуживание потребителей**	**Хранилище данных; анализ данных; электронные таблицы**
Оперативный	**Поддержка повседневной деятельности персонала и производство**	**Выплата жалованья; выписка счетов; закупки; бухгалтерский учет**	**Базы данных; обработка транзакций; генераторы приложений**

ПОДХОДЫ К РАЗРАБОТКЕ ПО

На сегодняшний день в инженерии программного обеспечения существует два основных подхода к разработке программных систем, различие между которыми обусловлено критериями декомпозиции:

1. Первый подход называют функционально-модульным, или структурным. В его основу положен принцип алгоритмической декомпозиции, когда выделяются функциональные элементы системы и устанавливается строгий порядок выполняемых действий.
2. Второй, объектно-ориентированный подход (Object-Oriented Analysis and Design, OOAD) использует объектную декомпозицию. При этом поведение системы описывается в терминах взаимодействия объектов.

Поддержка различных уровней принятия решений со стороны ИС и ИТ

Уровень принятия решений	*Направленность принимаемых решений*	*Типичные приложения ИС*	*Типичные ИТ-решения*
Стратегический	**Стратегии, поддерживающие долгосрочные цели организации**	**Анализ рынка и сбыта; планирование разработки товаров; оценка эффективности**	**Добыча данных (Data mining); управление знаниями**
Тактический	**Стратегии, поддерживающие краткосрочные задачи и распределение ресурсов**	**Анализ бюджета; прогнозирование фонда зарплаты; планирование запасов; обслуживание потребителей**	**Хранилище данных; анализ данных; электронные таблицы**
Оперативный	**Поддержка повседневной деятельности персонала и производство**	**Выплата жалованья; выписка счетов; закупки; бухгалтерский учет**	**Базы данных; обработка транзакций; генераторы приложений**